

BÁO CÁO THỰC TẬP CHUYÊN NGÀNH TUẦN 1 & 2

Tìm hiểu về Blockchain

Giảng viên hướng dẫn
TS. Lê Nguyên Khôi

Người thực hiện
Trần Việt Anh

Ngày 26 tháng 8 năm 2021

Mục lục

1	Giới thiệu	4
2	Các khái niệm cơ bản trong Blockchain	4
2.1	Định nghĩa Blockchain	4
2.1.1	Bitcoin và Blockchain	4
2.1.2	Cấu trúc của Blockchain	5
2.1.3	Các hoạt động cơ bản	9
2.1.4	Những sự phát triển từ Bitcoin	9
2.2	Ethereum Blockchain	10
2.2.1	Smart Contracts - Hợp đồng thông minh	10
2.2.2	Cấu trúc của Ethereum	11
2.2.3	Các hoạt động của Ethereum	12
2.2.4	Mô hình khuyến khích	13
2.3	Các thuật toán & kỹ thuật được sử dụng	13
2.3.1	Mã hóa khóa công khai	13
2.3.2	Mã băm - Hashing	14
2.3.3	Tính toàn vẹn của giao dịch	14
2.3.4	Bảo mật Blockchain	14
2.4	Những yếu tố thiết yếu trong thiết lập niềm tin	15
2.4.1	Hệ thống phi tập trung	15
2.4.2	Giao thức đồng thuận	15
2.4.3	Tin tưởng về Robustness	15
2.4.4	Forks	16
3	Smart Contracts	16
3.1	Những khái niệm cơ bản về Smart Contracts - hợp đồng thông minh	16
3.1.1	Tại sao nên sử dụng Smart Contracts?	16
3.1.2	Định nghĩa Smart Contracts	16
3.1.3	Xử lý Smart Contracts	18
3.1.4	Triển khai Smart Contracts	20
3.2	Ngôn ngữ Solidity	20
3.2.1	Cấu trúc của Solidity	20
3.2.2	Kiểu dữ liệu và luồng điều khiển cơ bản	20
3.2.3	Các kiểu dữ liệu đặc thù	21
3.2.4	Cấu trúc dữ liệu	22
3.3	Ứng dụng thực tiễn	23
3.3.1	Phát triển Smart Contracts	23
3.3.2	Yếu tố thời gian	24
3.3.3	Thẩm định và kiểm thử	24
3.3.4	Ứng dụng client	24
3.4	Best practices	24
3.4.1	Đánh giá và thiết kế Smart Contracts	25
4	Decentralized Applications	25
4.1	Giới thiệu	25
4.1.1	Blockchain Server	25
4.1.2	Định nghĩa Dapp	26
4.1.3	Ethereum APIs	27
4.2	Truffle	28
4.2.1	Truffle IDE	28
4.2.2	Phát triển theo kiểm thử	30
4.2.3	Giao diện web và kiểm thử	31
4.3	Cải thiện thiết kế	32
4.3.1	Các tính năng của Solidity	33
4.3.2	Điều khiển sự kiện	34
4.3.3	Oraclize	34
4.4	Mô hình và Tiêu chuẩn cho ứng dụng	37
4.4.1	Mô hình của DApp	37
4.4.2	Tiêu chuẩn cho DApp	37

5	Blockchain Platform	37
5.1	Blockchain cấp quyền	38
5.1.1	Hyperledger	38
5.1.2	Dịch vụ của Fabric	38
5.1.3	Mô hình và chức năng của Fabric	39
5.1.4	Composer	41
5.1.5	Microsoft Azure	42
5.2	Nền tảng ứng dụng phi tập trung	42
5.2.1	Augur	42
5.2.2	Grid+	43
5.3	Các thách thức và giải pháp	44
5.3.1	Tính đồng thuận	44
5.3.2	Tính mở rộng	44
5.3.3	Tính bảo mật và riêng tư	45
5.3.4	Escrow & Multi-sig	45
5.4	Các giải pháp phi tập trung khác	46
5.4.1	Hệ thống file liên kết	46
5.4.2	Hashgraph	46
6	Kết luận - Nhu cầu xã hội	48
7	Tài liệu tham khảo	48

1 Giới thiệu

Trong báo cáo này, em xin trình bày những tìm hiểu của em về Blockchain - một công nghệ mang tính cách mạng cho phép truyền tải sản phẩm kỹ thuật số ngang hàng (peer-to-peer) mà không cần bất kỳ trung gian nào và được dự đoán là sẽ có tác động mạnh mẽ như Internet.

Qua báo cáo ta sẽ được cung cấp:

- sự hiểu biết và kiến thức làm việc về các khái niệm Blockchain cơ bản.
- Bộ kỹ năng để thiết kế và thực hiện các Smart Contracts (hợp đồng thông minh)
- Các phương pháp để phát triển các ứng dụng phi tập trung (Decentralized Application) trên Blockchain
- Thông tin về các Blockchain Framework đang được phát triển và sử dụng rộng rãi toàn ngành.

Báo cáo sẽ bao gồm một loạt các vấn đề thiết yếu, từ nền tảng mật mã của công nghệ Blockchain đến việc sử dụng các ứng dụng phi tập trung trên nền tảng Blockchain Ethereum riêng tư.

2 Các khái niệm cơ bản trong Blockchain

2.1 Định nghĩa Blockchain

2.1.1 Bitcoin và Blockchain

Bitcoin là một loại đồng tiền ảo với hai đóng góp lớn: một hệ thống tiền tệ kỹ thuật số hoạt động liên tục và một mô hình cho công nghệ ứng dụng phi tập trung tự động được gọi là blockchain. Mặc dù trọng tâm của báo cáo là về một blockchain nói chung, nhưng ta phải hiểu hoạt động của công nghệ đằng sau Bitcoin để đánh giá đầy đủ sự đổi mới của blockchain.

Bitcoin được một người hoặc nhiều người bí ẩn tự xưng là Satoshi Nakamoto vào khoảng năm 2008, 2009. Bitcoin tạo nên một nền tảng sáng tạo để giao dịch ngang hàng mà không cần đến một cơ quan thẩm định trung ương nào. Như vậy thì làm thế nào để Bitcoin có được sự tin tưởng và bảo mật trong giao dịch? Bằng cách cài đặt các chương trình phần mềm dành cho kiểm chứng, xác thực, đồng thuận trong một cơ sở hạ tầng mới gọi là **Blockchain**.

Blockchain (chuỗi khối) là một chuỗi các block, có cấu trúc tương tự như một danh sách liên kết (linked list), mỗi block chứa thông tin của nhiều giao dịch.

Khác với những phương pháp truyền thống, khi mà 2 người giao dịch với nhau phải thông qua các bên trung gian như ngân hàng, đại lý thẻ tín dụng,... thì Blockchain cho phép 2 người giao dịch ngang hàng (peer-to-peer) những loại tài sản kỹ thuật số mà không cần tới các bên trung gian này.

Công nghệ Blockchain hỗ trợ các phương pháp cho:

- Một mạng phi tập trung ngang hàng (decentralized peer-to-peer network)
- Một mô hình tin cậy tập thể giữa các cá thể chưa biết
- Một sổ cái phân tán bất biến (distributed immutable ledger) của các bản ghi của các giao dịch.

Ta sẽ tìm hiểu sâu về các khái niệm vừa mới được nêu ở trên:

Phi tập trung có nghĩa là mạng sẽ hoạt động trên hình thức peer-to-peer (ngang hàng) giữa 2 người trực tiếp (user-to-user). Để dễ hiểu hơn ta sẽ lấy một ví dụ so sánh mạng tập trung (centralized) với phi tập trung (decentralized). Với phương pháp truyền thống đã nêu ở trên, khi 2 người muốn giao dịch với nhau sẽ đều phải thông qua một bên trung gian thứ 3 là ngân hàng, và rất nhiều người khác cũng sẽ phải làm tương tự nếu muốn giao dịch. Vì vậy đây sẽ là một mạng tập trung khi mọi giao dịch của tất cả các user đều tập trung hướng về ngân hàng để đến được bên đích người nhận.

Giờ ta sẽ so sánh sự khác biệt với mạng phi tập trung. Trong mạng này các bên sẽ giao dịch trực tiếp với nhau mà vị trí của họ đang ở đâu không quan trọng. Các chức năng của các bên trung gian được chuyển sang ngoại vi cho những người tham gia ngang hàng khác trong cơ sở hạ tầng blockchain. Những người tham gia không nhất thiết phải biết nhau. Khi đó ta có thể thắc mắc thiết lập sự tin tưởng lẫn nhau như thế nào khi các bên không biết nhau?

Blockchain có cách để làm vậy với một quy trình để xác thực, xác minh và xác nhận các giao dịch. Ghi lại giao dịch trong sổ cái phân tán của các khối, tạo bản ghi chống giả mạo các khối, chuỗi khối và triển khai giao thức đồng thuận để thỏa thuận về khối sẽ được thêm vào chuỗi. Vì vậy, xác thực, xác minh, đồng thuận và bản ghi bất biến sẽ hướng đến sự tin cậy và bảo mật của blockchain. Đây là lý do tại sao blockchain hỗ trợ mô hình tin cậy tập thể giữa các cá thể chưa biết.

Một **sổ cái phân tán bất biến** nghĩa là dữ liệu không nằm ở một server lớn duy nhất mà có mặt ở trải khắp nhiều nơi và dữ liệu trong sổ cái không thể bị xóa hoặc thay đổi.

Blockchain được tạo ra để hỗ trợ bitcoin và nhiều loại tiền ảo khác, nhưng ứng dụng của nó có thể tác động tích cực rất mạnh tới nhiều lĩnh vực trong nhiều ngành công nghiệp khác, bao gồm tài chính, chăm sóc sức khỏe, chính phủ, chế tạo và phân phối. Cụ thể có nhiều ứng dụng như:

- Vận chuyển hàng hóa, ví dụ: chuỗi cung ứng.

- Chuyển giao đa phương tiện kỹ thuật số, ví dụ: giao bán đấu giá tác phẩm nghệ thuật.
- Cung cấp dịch vụ từ xa, ví dụ: Du lịch.
- Nền tảng cho logic kinh doanh, ví dụ: Di chuyển công đoạn tính toán xử lý ngay tại nguồn lưu trữ dữ liệu.
- Trí thông minh phân tán (Distributed Intelligence), ví dụ: Chứng chỉ giáo dục.
- Tài nguyên phân tán, ví dụ: cung cấp và phân phối điện năng.
- Huy động vốn từ cộng đồng, ví dụ: huy động vốn cho startup.
- Hoạt động cộng đồng, ví dụ: bầu cử.
- Quản lý danh tính, ví dụ: một ID cho mọi hoạt động trong cuộc sống.
- Hồ sơ công khai của chính phủ và quản lý mở.

2.1.2 Cấu trúc của Blockchain

Giao dịch là phần tử cơ bản nhất của Bitcoin Blockchain, các giao dịch được kiểm chứng và truyền đi (broadcast). Nhiều giao dịch sẽ tạo thành 1 khối (block). Nhiều khối tạo thành một chuỗi thông qua các liên kết dữ liệu kỹ thuật số.

Các khối sẽ cần phải thông qua một **quy trình đồng thuận (Consensus process)** để chọn một khối tiếp theo sẽ được thêm vào chuỗi. Khối được chọn sẽ được xác thực và thêm vào chuỗi đang có hiện tại. Kiểm chứng và quy trình đồng thuận được thực hiện bởi các nút ngang hàng đặc biệt gọi là **thợ đào (miner)** - đây là những máy tính công suất cực cao thực thi những phần mềm được định nghĩa bởi giao thức Blockchain.

Giờ ta sẽ đi vào tìm hiểu một giao dịch của bitcoin. Một khái niệm cơ bản của một mạng bitcoin là **Unspent Transaction Output (UTXO)**, tập hợp tất cả các UTXOs trên mạng bitcoin sẽ xác định chung trạng thái của Bitcoin Blockchain. UTXOs có thể được hiểu là **đầu vào (input)** và **đầu ra (output)** của một giao dịch. Đầu ra của giao dịch này có thể làm đầu vào của giao dịch khác.

Ta có thể xem một ví dụ đơn giản: Người A chuyển cho người B 2 bitcoin, giao dịch chuyển 1 bitcoin này sẽ tạo ra 3 đầu ra (3 UTXOs) cho B. Ban đầu khi B chưa tiêu thụ 2 bitcoin này, 3 UTXOs đầu ra của giao dịch vừa rồi vẫn sẽ nằm yên trong cơ sở dữ liệu và chưa thay đổi. Cho tới khi B chuyển tiếp 2 bitcoin này cho một bên khác để tiêu thụ, thì 3 UTXOs nói trên là trở thành đầu vào (input) của một giao dịch mới. Đây là lý do tại sao nó được gọi là Unspent Transactions Output (nó sẽ chỉ là đầu ra được giữ nguyên khi mà lượng tiền vẫn chưa được tiêu - unspent).

Cấu trúc của một UTXO rất đơn giản. Nó chứa một định danh duy nhất của giao dịch đã tạo ra nó, một chỉ số nêu vị trí của nó trong danh sách đầu ra của giao dịch và giá trị hoặc lượng tiền nó đại diện. Cuối cùng là một script không bắt buộc phải có, để nêu ra điều kiện cần thỏa mãn thì đầu ra này mới được sử dụng tiếp.

Bản thân giao dịch chứa một số tham chiếu tới chính nó, tham chiếu tới một hoặc nhiều hoặc không có UTXO đầu vào nào, tham chiếu tới một hoặc nhiều hoặc không có UTXO đầu ra được tạo bởi nó, cuối cùng là lượng tiền đầu vào và đầu ra. Các người tham gia trên mạng có thể kiểm chứng các nội dung của giao dịch - đầu vào UTXO của giao dịch này có thực sự tồn tại trên trạng thái mạng bây giờ không? Đây chỉ là 1 trong rất nhiều điều kiện để kiểm chứng giao dịch. Thao tác này giống như Alice cho Amy vay 10000\$ và Amy nhờ Kevin đếm tiền để kiểm chứng xem có thật sự là 10000\$ hay không - trên mạng Blockchain thì giao dịch sẽ có rất nhiều "Kevin" để kiểm chứng (rất nhiều thợ đào) vì vậy rất khó để gian lận trong giao dịch, từ đó sự an toàn, tin tưởng và bảo mật được thiết lập trong blockchain.

Bitcoin và nhiều giao thức dựa trên UTXO lưu trữ dữ liệu, giao dịch và số dư tài khoản người dùng trong dạng Unspent Transactions Output, được hiểu như là danh sách lượng bitcoin được gửi tới người dùng mà "chưa tiêu" (unspent). Tổng các đầu ra này sẽ tạo thành số dư của người dùng. Trên blockchain, chúng được coi như là tập hợp các lượng bitcoin trên nhiều địa chỉ khác nhau, và vai trò của ví điện tử (wallet) ở đây là xác định xem những địa chỉ nào người dùng có quyền truy cập tới (có khóa của địa chỉ). Một bitcoin độc lập được theo dõi dễ dàng vì chúng được ký gửi từ người này sang người khác. Một giao dịch được coi là hợp lệ nếu một người có thể chứng minh quyền sở hữu của những bitcoin mà họ đang cố gửi đi.

Cách thức này khác với mô hình tài khoản của Ethereum, nơi lưu trữ toàn bộ thông tin số dư của tài khoản người dùng. Người dùng gửi và nhận token (tiền) từ tài khoản của họ và các đồng tiền ETHs khó theo dõi hơn vì chúng chỉ được cộng và trừ khỏi số dư. Một giao dịch được coi là hợp lệ nếu người dùng chứng minh được số dư tài khoản của họ đủ cao để thực hiện.

Hệ thống UTXO có thể coi là tái hiện lại nền kinh tế tiền mặt. Amy cho Bob 1 BTC (bitcoin), hệ thống giờ nhận thấy có 1 BTC được ký giao lại cho Bob mà anh ấy chưa đưa lại tiếp cho ai khác. Nếu Bob đã có sẵn 1 BTC, thì giờ số dư của anh ấy trên Blockchain sẽ là $1 \text{ BTC} + 1 \text{ BTC} = 2 \text{ BTC}$. Số dư tài khoản Bitcoin của Bob là tổng của tất cả bitcoin được ký cho anh ấy, tương tự như chiếc ví thật của Bob chứa tất cả các tờ tiền mặt mà Bob kiếm được. Nếu Bob muốn gộp 2 BTC riêng biệt mà anh ấy đang có, Bob phải làm vậy trong một giao dịch mới, tương tự như việc gộp một tờ 10\$ và một tờ 5\$ để mua một món hàng 15\$.

Khác biệt với cách trên thì mô hình tài khoản như của Ethereum giống với một tài khoản ngân hàng tự động duy trì số dư khách hàng. Khi Alice cho Bob 1 ETH, hệ thống nhận thấy số dư của Bob tăng lên 1 và số dư Alice giảm đi 1. Nếu Bob đã có sẵn 1 ETH trước đó, thì giờ số dư của anh ấy là 2 ETH. Bob không cần một giao dịch khác để gộp 2 đồng tiền trong tài khoản của anh ấy.

This transaction was first broadcast to the Bitcoin network on July 14, 2021 at 11:17 AM GMT+7. The transaction is currently unconfirmed by the network. At the time of this transaction, 0.09032339 BTC was sent with a value of \$2,881.05. The current value of this transaction is now \$2,883.54. Learn more about [how transactions work](#).

Hash

9256908f89c11698cdef1c6a199ee727ecf49eb6dcab7ba1696b2...

2021-07-14 11:17

bc1qa4j243ey3sa9f3kvvf7p8ygzf6vq078tkc43...

0.06284702 BTC

➔

3JSzdzqGYKqZefqLbE71RSAsmgBHU2FA7n1

0.07379755 BTC

bc1q5funhmqyh80gtvxvhjsyff0kz0jfu22mh0z6ak

0.02748171 BTC

bc1qe60tfvg7r37gmz8ly0l474q685nrvur59504qt

0.01652584 BTC

Fee

0.00000534 BTC
(1.439 sat/B - 0.639 sat/WU - 371 bytes)
(2.555 sat/vByte - 209 virtual bytes)

0.09032339 BTC

UNCONFIRMED

Details ⓘ

Hash	9256908f89c11698cdef1c6a199ee727ecf49eb6dcab7ba1696b2ad1bea4ba60
Status	Unconfirmed
Received Time	2021-07-14 11:17
Size	371 bytes
Weight	836
Included in Block	Mempool
Confirmations	0
Total Input	0.09032873 BTC
Total Output	0.09032339 BTC
Fees	0.00000534 BTC
Fee per byte	1.439 sat/B

Hình 1: Thông tin về 1 giao dịch Bitcoin

Trên đây là thông tin về một giao dịch Bitcoin với 2 UTXO đầu vào và 2 UTXO đầu ra, ta có thể gọi là tổng lượng tiền trong 2 đầu vào được sử dụng để sinh ra 2 đầu ra.

Giờ ta sẽ quan sát một block cụ thể. Mỗi block được tạo ra bao gồm phần header chứa thông tin về khối đó, và một tập hợp các giao dịch hợp lên trong khối.

This block was mined on July 14, 2021 at 10:27 AM GMT+7 by [Unknown](#). It currently has 5 confirmations on the Bitcoin blockchain.

The miner(s) of this block earned a total reward of 6.25000000 BTC (\$199,528.75). The reward consisted of a base reward of 6.25000000 BTC (\$199,528.75) with an additional 0.26527600 BTC (\$8,468.83) reward paid as fees of the 2259 transactions which were included in the block. The Block rewards, also known as the Coinbase reward, were sent to this [address](#).

A total of 22,394.61770228 BTC (\$714,939,212.30) were sent in the block with the average transaction being 9.91350939 BTC (\$316,484.82). [Learn more about how blocks work](#).

Hash	00000000000000000000000040a9bde370494814e7b4c33631ff916cebbb1b2411c55 
Confirmations	5
Timestamp	2021-07-14 10:27
Height	690918
Miner	Unknown
Number of Transactions	2,259
Difficulty	14,363,025,673,659.97
Merkle root	cdb3296fd97500e6bc87e813b22df393566c11d8dc69634251f9b7e464162ccf
Version	0x20400004
Bits	387,160,270
Weight	3,993,296 WU
Size	1,376,690 bytes
Nonce	1,189,379,644
Transaction Volume	22394.61770228 BTC
Block Reward	6.25000000 BTC
Fee Reward	0.26527600 BTC

Hình 2: Thông tin về một khối trên Bitcoin Blockchain được lấy bởi web BlockchainExplorer

Hash	cda503e82d97ad784915447e1fcb036beda0d82961ef4c54ef935bd...		2021-07-14 10:27
	COINBASE (Newly Generated Coins)	➡	1Bf9sZvBHPFGVPX71WX2njhd1NXKv5y7v5 OP_RETURN OP_RETURN 6.51527600 BTC 0.00000000 BTC 0.00000000 BTC
Fee	0.00000000 BTC (0.000 sat/B - 0.000 sat/WU - 293 bytes) (0.000 sat/vByte - 266 virtual bytes)		6.51527600 BTC 5 Confirmations
Hash	bcbec77f6386f1b6500d9e045f76ca211ca4bb9a245b1cbea694771... 358mMRwcxuCSkKheuVWaXHJBGKrXo3f6JW 28.78222867 BTC		2021-07-14 10:23
		➡	3JedszLiYHSY1jKhNbT5Pjd2nzbEpsEWaJ 358mMRwcxuCSkKheuVWaXHJBGKrXo3f6JW 18.42800000 BTC 10.35332867 BTC
Fee	0.00090000 BTC (362.903 sat/B - 135.952 sat/WU - 248 bytes) (542.169 sat/vByte - 166 virtual bytes)		28.78132867 BTC 5 Confirmations
Hash	ffb81136b89b8d2ebde8a62428000e07bc2b1f96eee469de600492... bc1qwqdg6squsna38e46795at95yu9atm8azz... 3.98260000 BTC		2021-07-14 10:23
		➡	1NKgp5DAfGiksJXFFdgRjJ6cdLX2ffX18q bc1qwqdg6squsna38e46795at95yu9atm8azzm... 3.56058750 BTC 0.42111250 BTC
Fee	0.00090000 BTC (235.602 sat/B - 117.493 sat/WU - 382 bytes) (468.750 sat/vByte - 192 virtual bytes)		3.98170000 BTC 5 Confirmations
Hash	271a3e58f1bb66bfb349c3487d0d7b0d294115defd12e027b09575f... bc1qdl753ur9ucwa3cgfrud2nqv7k69dykk3cw... 2.96120000 BTC		2021-07-14 10:21
		➡	112FXXHrCjHAQUshe3Vr1hAH2jBRqmRk59 bc1qwqdg6squsna38e46795at95yu9atm8azz... 2.72955380 BTC 0.23074620 BTC
Fee	0.00090000 BTC (235.602 sat/B - 117.493 sat/WU - 382 bytes) (468.750 sat/vByte - 192 virtual bytes)		2.96030000 BTC 5 Confirmations

Hình 3: Thông tin về các giao dịch trong một block

Giờ ta có thể hiểu cấu trúc liên kết của Blockchain bằng cách nhìn vào 3 khối số 488867, 488868 và 488869. Khối 488868 ở giữa có mã băm của khối 488867 làm previous hash (mã băm khối liền trước) của nó. Đồng thời khối 488868 cũng có mã băm của 488869 làm next block hash (mã băm khối liền sau). Thông tin này áp dụng tương tự cho 2 khối 2 bên là 488867 và 488869. Điều này tạo liên kết trong Blockchain.

Block #488867

Summary	
Number Of Transactions	258
Output Total	2,237.10736771 BTC
Estimated Transaction Volume	780.44955721 BTC
Transaction Fees	0.15507913 BTC

Hashes	
Hash	0000000000000000081dbaadbccbd44f3f9c37f7727d9351b841d84b3c92dd1
Previous Block	000000000000000000c0e14634086cd854f8f9fd32094f9b2f849b558522a14
Next Block(s)	000000000000000000bcaafbee83a84d2f86968e5c13ddb303c8ef1cfe83bb
Merkle Root	3f2cf55ebc5b6280871306240a72e863e3263b08dba8efacc39cf14de1a2c0eba

Block #488868

Summary	
Number Of Transactions	1936
Output Total	7,969.11120067 BTC
Estimated Transaction Volume	323.01697332 BTC
Transaction Fees	0.51852372 BTC

Hashes	
Hash	000000000000000000bcaafbee83a84d2f86968e5c13ddb303c8ef1cfe83bb
Previous Block	0000000000000000081dbaadbccbd44f3f9c37f7727d9351b841d84b3c92dd1
Next Block(s)	000000000000000000b0dbdf8f1566fb7a21b8859da92ef932e150ae6738303b
Merkle Root	e9c51dd8270518856ac17eeecfaa8031e063c9d17a8d7985539914379d034500e

Block #488869

Summary	
Number Of Transactions	2000
Output Total	16,565.68401578 BTC
Estimated Transaction Volume	1,246.96280447 BTC
Transaction Fees	1.23389765 BTC

Hashes	
Hash	000000000000000000b0dbdf9f1566fb7a21b8859da92ef932e150ae6738303b
Previous Block	000000000000000000bcaafbee83a84d2f86968e5c13ddb303c8ef1cfe83bb
Next Block(s)	0000000000000000000033e2da91f11168aa6c0c273a6b3f8720e6b2a0198e2f12
Merkle Root	c79f2b8e7ee4f8f3dc014ebb9f583bc43547855fb05a6ffa4d426f0ab04efa

Hình 4: Thông tin hiển thị sự liên kết giữa 3 block liền nhau thông qua các trường thuộc tính hash

2.1.3 Các hoạt động cơ bản

Các hoạt động trong một mạng phi tập trung là trách nhiệm của những người tham gia ngang hàng và nút tính toán tương ứng của họ - ví dụ như laptop, máy tính bàn và các máy chủ. Những hoạt động này bao gồm kiểm chứng giao dịch, thu thập các giao dịch cho một khối, truyền đi giao dịch được bầu chọn trong khối, và đồng thuận trong việc tạo khối nào để thêm tiếp vào chuỗi tạo thành một bản ghi bất biến.

Đầu tiên ta tìm hiểu về phương diện những người tham gia. Có 2 vai trò chính cho những người này. Đầu tiên là những người khởi tạo việc truyền giao giá trị bằng cách tạo giao dịch, và tiếp đến là những người gọi là thợ đào - họ thực hiện công việc tính toán bổ sung để xác minh giao dịch, truyền phát giao dịch, cạnh tranh để yêu cầu quyền tạo khối, làm việc để cùng đạt được sự đồng thuận bằng cách xác thực khối, phát khối mới được tạo và xác nhận giao dịch. Những người thợ đào sẽ được thưởng bitcoin để khuyến khích họ trong việc quản lý blockchain.

Việc kiểm chứng giao dịch được thực hiện độc lập bởi tất cả các thợ đào. Quá trình này liên quan đến việc thẩm định bằng hơn 20 tiêu chí, bao gồm kích thước, cú pháp, v.v. Một số tiêu chí sau đây là: UTXO hợp lệ, UTXO đầu ra tham chiếu là chính xác, lượng đầu vào tham chiếu và lượng đầu ra khớp đủ. Giao dịch không hợp lệ sẽ bị từ chối và sẽ không được phát sóng. Tất cả các giao dịch hợp lệ được thêm vào một nhóm giao dịch. Các thợ đào chọn một tập hợp các giao dịch từ nhóm này để tạo một khối.

Việc này sẽ tạo ra một thách thức. Nếu mỗi thợ đào thêm một khối vào chuỗi thì chuỗi này sẽ bị tách ra thành rất nhiều nhánh, tạo thành một trạng thái không nhất quán. Mà ta nhớ lại blockchain là một chuỗi liên kết nhất quán duy nhất của dòng chảy. Chúng ta cần một hệ thống để giải quyết vấn đề này. Giải pháp là các thợ đào sẽ cạnh tranh với nhau để giải một bài toán đố (puzzle) để quyết định xem ai sẽ là người có quyền tạo khối tiếp theo. Những puzzle này cần công suất xử lý cao (CPU intensive). Khi một thợ đào giải được bài toán, một thông báo sẽ được truyền lên mạng và khối thợ này tạo ra cũng được truyền lên. Sau đó, những người tham gia còn lại xác thực khối mới này. Mọi người tham gia đã đạt được sự đồng thuận (consensus) về việc thêm khối mới này vào chuỗi. Khối mới này cũng được thêm vào bản sao chuỗi cục bộ của họ. Từ đó, một tập hợp các giao dịch được ghi lại và xác nhận. Thuật toán để đạt được sự đồng thuận này được gọi là Proof-Of-Work protocol (Giao thức bằng chứng công việc) - vì nó có công việc tính toán công suất cao để giải được bài toán đố và chiếm quyền tạo ra khối tiếp theo. Giao dịch không (Transaction Zero), chỉ số 0 của khối vừa được xác nhận được tạo bởi thợ đào tạo ra khối. Giao dịch này có một UTXO đặc biệt và không có bất kỳ UTXO đầu vào nào. Nó được gọi là giao dịch Coinbase mà tạo ra tiền thưởng cho người tạo khối - hay còn gọi là phí giao dịch. Hiện tại, tiền thưởng này đang là 4.519\$ BTC cho một bitcoin.

2.1.4 Những sự phát triển từ Bitcoin

Bitcoin blockchain là mã nguồn mở và toàn bộ code có trên Github. Vào những năm ban đầu quanh 2009, mã nguồn mở này được phát triển để tạo ra nhiều đồng tiền ảo khác. Bitcoin hỗ trợ một tính năng đặc biệt và tùy chọn gọi là scripts để chuyển giao giá trị có điều kiện. Ethereum Blockchain đã mở rộng tính năng scripting này thành một nền tảng thực thi

code mạnh mẽ gọi là **Smart Contracts** - Hợp đồng thông minh. Một hợp đồng thông minh cung cấp khả năng chạy code cho logic kinh doanh nhúng trên Blockchain cực kỳ mạnh mẽ. Dựa trên những khả năng đó, 3 loại chính của Blockchain đã phát triển từ nền tảng Bitcoin:

- Loại một xử lý đồng tiền trong chuỗi tiền tệ ảo, ví dụ: Bitcoin
- Loại hai hỗ trợ tiền ảo và một lớp logic kinh doanh được hỗ trợ bởi việc thực thi code, ví dụ: Ethereum
- Loại ba không liên quan tới tiền ảo nhưng có hỗ trợ thực thi phần mềm cho logic kinh doanh, ví dụ: Linux Foundation’s Hyperleger.

Với việc bổ sung thực thi mã, ta cần phải xem xét nghiêm túc về quyền truy cập công khai vào blockchain, phân loại các blockchain **công khai, riêng tư và được cấp phép** dựa trên giới hạn truy cập.

Bitcoin là một ví dụ rõ ràng cho blockchain công khai, ai cũng có thể tham gia và rời đi tùy theo họ muốn. Các khối giao dịch và blockchain có thể được quan sát công khai cho dù những người tham gia là ẩn danh. Bitcoin là mã nguồn mở, ta có thể tạo ra đồng tiền mã hóa mới bằng cách thay đổi mã nguồn của Bitcoin.

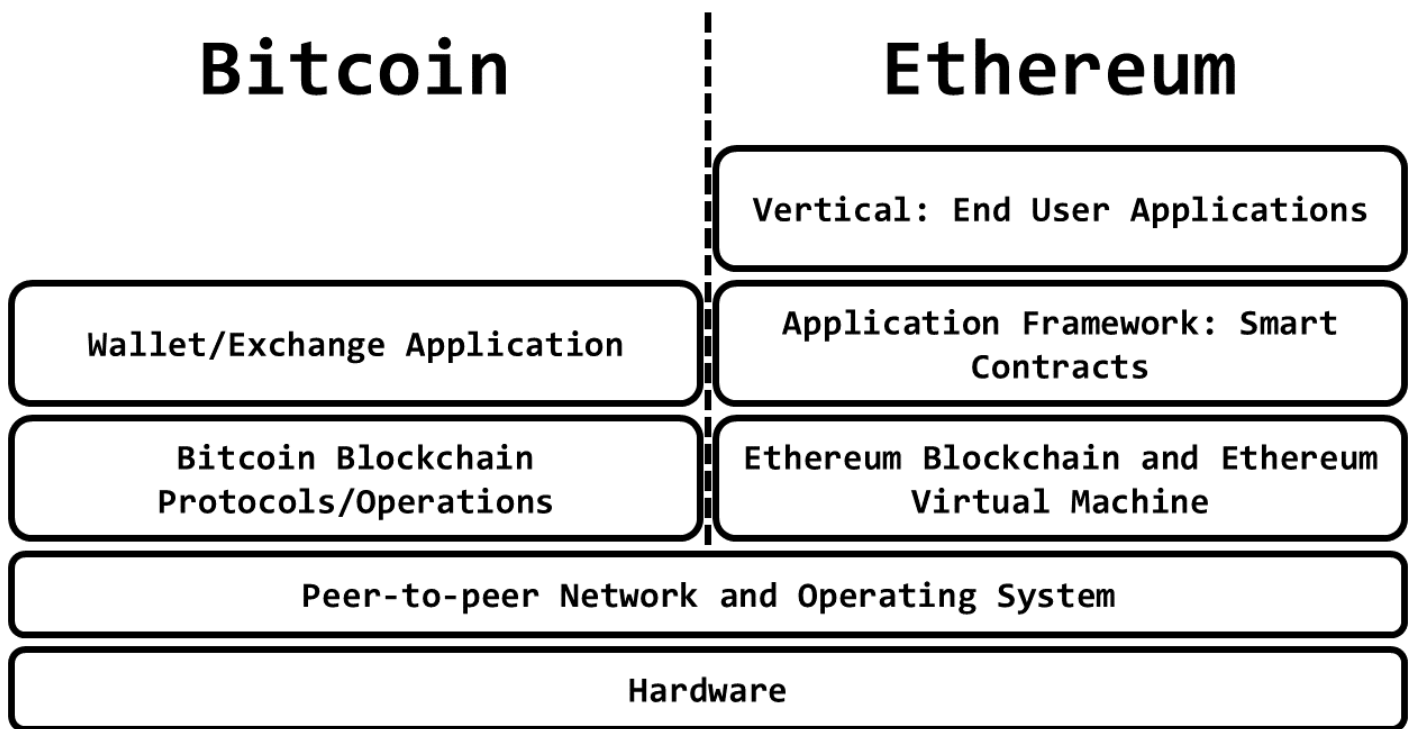
Trong một blockchain riêng tư, quyền truy cập bị giới hạn và chỉ dành cho những người được chọn / cấp phép, ví dụ như những người cùng trong một tổ chức. Sự hạn chế này giúp đơn giản hóa những hoạt động thông thường như tạo khối và mô hình dự phòng.

Loại thứ ba gọi là blockchain được cấp phép hay còn gọi là blockchain liên hợp. Nó được dành cho một tập đoàn gồm nhiều bên hợp tác để giao dịch trên một blockchain cho dễ quản lý, xuất xứ và trách nhiệm giải trình, ví dụ: một tập đoàn gồm tất cả các công ty ô tô hoặc các tổ chức chăm sóc sức khỏe. Blockchain được cấp phép có những lợi ích của một blockchain công khai với việc chỉ cho phép những người dùng có quyền cộng tác và giao dịch.

2.2 Ethereum Blockchain

2.2.1 Smart Contracts - Hợp đồng thông minh

Bitcoin Blockchain là cha đẻ của mọi blockchain. Nó được dùng cho giao dịch ngang hàng và nó thực hiện tốt. Vào năm 2013, một nền tảng dành cho việc thực thi code được giới thiệu bởi những nhà sáng lập Ethereum. Điểm nhấn nổi bật của Ethereum chính là Smart Contracts.



Hình 5: So sánh cấu trúc giữa Bitcoin Blockchain và Ethereum Blockchain

Sơ đồ trên so sánh Bitcoin blockchain và Ethereum blockchain . Ở bên trái là Bitcoin và một ứng dụng ví để bắt đầu giao dịch. Ở bên phải là Ethereum đã thực hiện một bước quan trọng trong việc chuyển đổi blockchain thành một framework tính toán mở ra rất nhiều cơ hội trong lĩnh vực phi tập trung. Ethereum hỗ trợ các hợp đồng thông minh và máy ảo trên đó các hợp đồng thông minh thực thi. Các hợp đồng thông minh sau đó cho phép ứng dụng phi tập trung thực hiện nhiều hơn việc chuyển giao giá trị, tự động hóa hiệu quả các ứng dụng phi tập trung như chuỗi cung ứng.

Smart Contracts với định nghĩa đơn giản nhất chính là một đoạn mã nguồn được triển khai trên Blockchain node. Lệnh thực thi smart contract được gửi bằng tin nhắn đính kèm trong giao dịch. Giao dịch đồng tiền kỹ thuật số chỉ cần yêu cầu lệnh cộng và trừ. Đôi khi giao dịch ngoài thực tế cần nhiều logic và điều kiện phức tạp hơn, ví dụ như hẹn đến một thời điểm cụ thể mới bắt đầu giao dịch, độ tuổi người thực hiện trên 18 mới chấp nhận giao dịch,... Từ đó Smart Contracts có thể thỏa mãn những yêu cầu này

Về mặt cấu trúc, một smart contract giống như một class trong thiết kế hướng đối tượng. Nó có dữ liệu, functions hoặc methods với modifiers như là public hoặc private, đi kèm với các hàm get hoặc set để lấy giá trị biến. Một ngôn ngữ lập trình cụ thể đã được thiết kế dành riêng cho việc viết smart contracts. Solidity là ví dụ phổ biến nhất.

```
1  pragma solidity ^0.4.0;
2  // Imagine a big integer that the whole world could share
3  contract SimpleStorage {
4      uint storedData;
5
6      function set(uint x) public {
7          storedData = x;
8      }
9
10     function get() constant public returns (uint) {
11         return storedData;
12     }
13
14     function increment (uint n) public {
15         storedData = storedData + n;
16         return;
17     }
18
19     function decrement (uint n) public {
20         storedData = storedData - n;
21         return;
22     }
23 }
24
```

Listing 1: Mã nguồn ví dụ cho một Smart Contracts

Trong mã nguồn trên, dòng đầu tiên với từ khóa **pragma** chỉ định phiên bản của ngôn ngữ Solidity. Smart Contracts này có mục đích lưu trữ một biến số nguyên. Dữ liệu của biến được định nghĩa với kiểu uint. Đồng thời hai hàm cũng được định nghĩa để đọc và viết dữ liệu - get and set, đi cùng với 2 hàm tăng và giảm giá trị của biến.

Chúng ta cần một cơ sở hạ tầng tính toán để thực thi bất kỳ mã code tùy ý nào. Mọi nút trong mạng Ethereum sẽ có thể thực thi mã bất kể loại phần cứng hoặc hệ điều hành, nhờ vào Ethereum Virtual Machine, EVM. EVM cung cấp một lớp trừu tượng chạy ở bất kỳ đâu cho Smart Contract Code. Một hợp đồng thông minh được viết bằng ngôn ngữ lập trình cấp cao được dịch sang mã byte EVM và sau đó, được triển khai trên Máy ảo Ethereum, EVM. Mỗi nút sẽ host các mã Smart Contract giống nhau trên EVM.

2.2.2 Cấu trúc của Ethereum

Ethereum giới thiệu khái niệm tài khoản như một phần của giao thức Ethereum blockchain. Tài khoản là người khởi tạo và cũng là mục tiêu của một giao dịch. Một giao dịch cập nhật trực tiếp số dư tài khoản thay vì duy trì trạng thái như trong các UTXO của Bitcoin. Nó cho phép truyền giá trị, tin nhắn và dữ liệu giữa các tài khoản có thể dẫn đến chuyển đổi trạng thái. Các sự chuyển đổi này được thực hiện bằng các giao dịch.

Có lại loại tài khoản, Externally Owned Account (EOA) được điều khiển bởi khóa bí mật. Contracts Account (CA) được điều khiển bởi mã nguồn và chỉ có thể kích hoạt bởi EOA. Ta cần phải có một EOA để tham gia vào mạng Ethereum. Nó tương tác với blockchain bằng các giao dịch. Một CA đại diện cho một Smart Contract. Mọi tài khoản đều có một số dư. Một nút tham gia trên mạng có thể gửi giao dịch để chuyển tiền Ethereum hoặc giao dịch để kích hoạt mã nguồn Smart Contract hoặc làm cả hai. Cả hai loại giao dịch này yêu cầu trả phí. Một tài khoản phải có đủ số dư để chi trả phí giao dịch cần kích hoạt. Phí được thanh toán trong đơn vị Wei, Wei là mệnh giá thấp hơn Ether. Một Ether bằng 10^{18} Weis.

Một giao dịch trong Ethereum bao gồm:

- Người nhận tin nhắn
- Chữ ký kỹ thuật số của người gửi ủy quyền chuyển
- Số tiền Wei để chuyển
- Trường dữ liệu tùy chọn hoặc gọi là tải trọng chứa thông báo cho hợp đồng
- STARTGAS: giá trị đại diện cho số lượng tối đa các bước tính toán mà giao dịch được phép chạy.
- GASPRICE: giá trị đại diện chi phí người gửi phải trả cho việc tính toán.

⑦ Block Height:	12824025 < >
⑦ Timestamp:	⌚ 1 min ago (Jul-14-2021 08:16:39 AM +UTC)
⑦ Transactions:	168 transactions and 46 contract internal transactions in this block
⑦ Mined by:	0xea674fde714fd979de3edf0f56aa9716b898ec8 (Ethermine) in 4 secs
⑦ Block Reward:	2.48933716352450873 Ether (2 + 0.48933716352450873)
⑦ Uncles Reward:	0
⑦ Difficulty:	6,534,621,754,238,006
⑦ Total Difficulty:	27,503,571,605,250,840,727,361
⑦ Size:	48,244 bytes
⑦ Gas Used:	14,755,807 (98.42%)
⑦ Gas Limit:	14,992,411
⑦ Extra Data:	ethermine-us-east1 (Hex:0x65746865726d696e652d75732d6561737431)

Hình 6: Thông tin về một khối trong Ethereum Blockchain

⑦ Transaction Hash:	0x97711e234fa7e21f5fc3bc214be3ace8713a0c1d22aedfa5c77c8be76f5e6ac0 ⓘ
⑦ Status:	✔ Success
⑦ Block:	12824030 1 Block Confirmation
⑦ Timestamp:	⌚ 19 secs ago (Jul-14-2021 08:17:11 AM +UTC) ⌚ Confirmed within 30 secs
⑦ From:	0xb10fe1db47ce0a5297c5c7360621541274e281b4 ⓘ
⑦ Interacted With (To):	Contract 0xc4ad29ba4b3c580e6d59105fff484999997675ff ✔ ⓘ
⑦ Tokens Transferred:	➤ From 0xb10fe1db47ce0... To 0x7a16ff8270133f... For 9.565928767966581593 ⚙ Curve.fi USD... (crv3cr...)
⑦ Value:	0 Ether (\$0.00)
⑦ Transaction Fee:	0.000953128125 Ether (\$1.81)
⑦ Gas Price:	0.000000028125 Ether (28.125 Gwei)
Click to see More ↓	
⑦ Private Note:	To access the Private Note feature, you must be Logged In

Hình 7: Thông tin về các giao dịch trong một khối Ethereum

2.2.3 Các hoạt động của Ethereum

Với một giao dịch Ether đơn giản, lượng tiền chuyển đi cùng địa chỉ đích đến được chỉ rõ cùng với phí hay gọi là gas points. Lượng tiền cùng phí được chuyển tới các tài khoản tương ứng. Như hình vẽ dưới, trong một giao dịch gửi đi 100 Ether, 21,000 gas points sẽ phải được chi trả cho thợ đào - người đã thêm khối giao dịch vào trong blockchain.

Một nút Ethereum là một hệ thống tính toán đại diện cho một thực thể kinh doanh hoặc một người tham gia cá nhân. Một nút Ethereum đầy đủ lưu trữ phần mềm cần thiết để bắt đầu giao dịch, xác thực, khai thác, tạo khối, thực hiện hợp đồng thông minh và chạy Máy ảo Ethereum, EVM. Hợp đồng thông minh được thiết kế, phát triển, biên dịch và triển khai trên EVM. Có thể có nhiều nhiều hợp đồng thông minh cùng chạy trên một máy ảo. Khi địa chỉ đích trong giao dịch là hợp đồng thông minh, mã thực thi tương ứng với hợp đồng thông minh sẽ được kích hoạt và thực thi. Đầu vào cần có cho việc thực hiện này được trích xuất từ trường trọng tải (payload field) của giao dịch. Trạng thái hiện tại của hợp đồng thông minh là các giá trị của các biến được xác định trong đó. Trạng thái của hợp đồng thông minh có thể được cập nhật bằng cách thực thi này. Kết quả của việc thực hiện này được cho biết trong biên lai giao dịch. Một blockchain duy trì cả mã băm trạng thái và mã băm biên nhận. Tất cả các giao dịch được tạo đều được xác thực. Xác thực giao dịch bao gồm việc kiểm tra dấu thời gian, kiểm tra tổ hợp nonce (nonce có thể hiểu là con số có được khi thợ đào giải bài toán đố) hợp lệ và kiểm tra người gửi đủ phí để thực hiện. Các nút thợ đào trong mạng nhận, xác minh, thu thập và thực hiện các giao dịch. Các mã hợp đồng thông minh đang hoạt động được thực thi bởi tất cả các thợ đào. Các giao dịch đã được xác thực được phát sóng và thu thập để tạo khối.

2.2.4 Mô hình khuyến khích

Mọi hành động trong Ethereum cần fuel (nhiên liệu) mã hoá hay gọi là gas (xăng). Lượng gas được sử dụng để chi khoản phí tính bằng Ether, để hoàn thành tính toán bằng các giá trị chuẩn. Ether, là một đồng tiền mã hoá, sẽ thay đổi giá trị theo biến động thị trường, nhưng lượng gas thì không. Ethereum định rõ lượng gas cụ thể cho từng loại hoạt động. Quá trình khai thác (mining) tính toán lượng gas cần thiết để thực hiện giao dịch. Nếu phí được chỉ rõ và lượng gas trong giao dịch không đủ, giao dịch này sẽ bị từ chối. Tương tự như gửi một bức thư mà không đủ phí bưu chính. Bức thư sẽ không được gửi nếu nó không được trả đủ phí. Lượng gas cần thiết để thực hiện giao dịch cần phải có đủ trong tài khoản cho để đảm bảo việc thực thi. Nếu còn gas thừa sau khi thực hiện giao dịch, khoản này sẽ được chuyển trả về tài khoản ban đầu.

Giới hạn gas là lượng gas có sẵn cho một block để dùng. Ví dụ, nếu một block chỉ định giới hạn 1.500.000 gas và phí giao dịch Ether cơ bản là 21.000, Block Ethereum cụ thể này có thể dùng cho khoảng 70 giao dịch Ether thuần túy. Nếu chúng ta thêm hợp đồng thông minh vào block này, thường sẽ cần nhiều gas hơn, và số lượng giao dịch cho block này có thể sẽ giảm xuống. Lượng gas đã sử dụng là lượng gas thực tế sử dụng để hoàn thành việc tạo ra block.

Bây giờ ta nhìn vào Mô hình Khuyến khích khai thác khá giống với bên Bitcoin Blockchain. Thợ đào thắng cuộc thi giải câu đố sẽ tạo ra một block mới, được khuyến khích thưởng với mức phí được trả là 3 Ether, và tất cả phí giao dịch trong Ethereum blockchain. Thợ đào chiến thắng cũng sẽ được thưởng phí và lượng gas cho loại giao dịch hợp đồng thông minh.

Sẽ có những người thợ đào giải được câu đố, nhưng không giành chiến thắng khối này được gọi là Ommer. Các block được tạo ra bởi họ được gọi là Ommer block. Chúng được thêm dưới dạng khối Ommer block hoặc gọi là block phụ, vào chuỗi chính. Những người thợ đào Ommer cũng nhận được một số phần trăm nhỏ của tổng số lượng gas giống như một sự an ủi và cho việc bảo mật mạng.

2.3 Các thuật toán & kỹ thuật được sử dụng

2.3.1 Mã hóa khóa công khai

Hai kỹ thuật chủ yếu được sử dụng để bảo mật chuỗi và thẩm định, xác minh một cách hiệu quả là Hashing (băm) và mã hoá với khoá bất đối xứng (asymmetric key encryption). Những kỹ thuật này dựa trên một số thuật toán phức tạp đã được chứng minh. Báo cáo sẽ cung cấp tổng quát về việc áp dụng các thuật toán này, và vai trò quan trọng của chúng trong việc bảo mật chuỗi phi tập trung, cụ thể là: Mã khoá công khai, hashing an toàn (secure hashing), toàn vẹn của giao dịch và toàn vẹn blockchain. Ta bắt đầu phần này với thảo luận các khái niệm mã hoá với khóa bất đối xứng, sau đó ta sẽ định nghĩa khái niệm hashing, tiếp sau là các thuật toán sử dụng cho các nhu cầu hashing khác nhau trong giao thức blockchain. Sau đó ta sẽ giải thích các kỹ thuật sử dụng các thuật toán này để quản lý sự toàn vẹn của giao dịch và các block trong chuỗi.

Nhắc lại rằng các thành viên trong mạng phi tập trung blockchain không nhất thiết phải biết đến nhau. Không thể kiểm tra uỷ quyền bằng các phương pháp thông thường ví dụ như xác minh bạn là ai thông qua bằng lái xe của bạn. Các thành viên có thể tham gia và rời khỏi chuỗi bất cứ khi nào họ muốn. Mọi người hoạt động vượt ra ngoài ranh giới của niềm tin. Với hoàn cảnh này, làm thế nào để nhận dạng những thành viên trong mạng ngang hàng? Làm thế nào để ta uỷ quyền và ký các giao dịch? Làm sao để phát hiện các giao dịch giả mạo hoặc bị lỗi? Chúng ta làm được điều này bằng cách sử dụng giải thuật mã hoá với khóa công khai. Cùng một khóa được sử dụng để mã hoá và giải mã, do đó nó được gọi là khóa đối xứng. Lưu ý rằng các khóa và chức năng mã hoá và giải mã thường phức tạp hơn nhiều trong một ứng dụng thực tế. Lưu ý rằng mã hoá với khóa đối xứng cũng có một số vấn đề. Thứ nhất, người ta có thể dễ dàng lấy được khóa bí mật từ dữ liệu được mã hoá. Thứ hai là việc phân phối khóa, làm thế nào để ta có thể truyền khóa cho người tham gia để giao dịch? Những vấn đề này gây ra nhiều khó chịu trong một mạng lưới blockchain phi tập trung nơi thành viên không biết nhau. Giờ ta xem xét cách mã hoá bằng khóa công khai xử lý các vấn đề này. Thay vì một mã khóa bí mật duy nhất, ta sử dụng một cặp khóa để xử lý cả hai vấn đề của mã hoá đối xứng. Giả sử chữ b thường B hoa là cặp khóa cho thành viên ở Buffalo, New York, Hoa Kỳ. Đặt chữ k thường và K hoa là cặp khóa cho thành viên ở Kathmandu, Nepal. Khóa công khai (public key) được chia sẻ công khai, khóa riêng (khóa bí mật - private key) được giữ an toàn và bị khóa. Thường mật khẩu và cặp khóa hoạt động như sau: hàm mã hoá giữ hai thuộc tính với một cặp khóa. Khóa công khai và khóa bí mật có đặc điểm độc nhất là mặc dù dữ liệu được mã hoá bằng khóa riêng, nó có thể được giải bằng khóa công khai tương ứng và ngược lại.

Giờ hãy xem xét một ví dụ, xác thực người gửi và người nhận. Chúng ta sẽ xem một cách sử dụng phổ biến của một mã hoá đối xứng. Giả sử thành viên ở Buffalo muốn giao dịch với thành viên ở Kathmandu. Thay vì chỉ gửi một thông điệp đơn giản, thành viên ở Buffalo gửi một dữ liệu giao dịch được mã hoá bằng khóa riêng của Buffalo, và sau đó thông điệp được mã hóa bằng khóa công khai của Kathmandu. Kathmandu trước tiên sẽ giải mã dữ liệu bằng khóa bí mật của họ, sau đó sử dụng khóa công khai của Buffalo để giải mã lần nữa dữ liệu giao dịch được giao. Điều này đảm bảo rằng chỉ có Kathmandu mới có thể giải mã và nhận dữ liệu và cũng chỉ có Buffalo mới có thể gửi dữ liệu.

Một cài đặt phổ biến sử dụng khóa công khai, khóa bí mật là thuật toán Rivest Shamir Adleman (RSA). Ứng dụng phổ biến của RSA là xác thực người dùng không dùng mật khẩu, ví dụ để truy cập một máy ảo trên đám mây của Amazon. Mặc dù RSA được sử dụng rất phổ biến ở nhiều ứng dụng, blockchain vẫn cần một thuật toán hiệu quả và mạnh hơn. Hiệu quả là một yêu cầu quan trọng vì cặp khóa công khai thường được sử dụng trong nhiều hoạt động khác nhau trong giao thức blockchain. Mã hoá Đường cong Elip, họ thuật toán ECC được sử dụng trong Bitcoin cũng như trong Ethereum blockchain để sinh ra cặp khóa. Tại sao là ECC chứ không phải RSA? ECC mạnh hơn RSA khi dùng trên cùng một số bit. Một cặp khóa ECC 256 bit tương đương với khóa RSA với 3072 bit. Cả Bitcoin và Ethereum đều sử dụng các thuật toán ECC cho việc mã hoá.

2.3.2 Mã băm - Hashing

Ta cần phải bảo vệ khoá riêng để giữ an toàn cho tài sản của mình trong blockchain. Hashing là gì? Hàm hash hay **chuyển đổi và ánh xạ dữ liệu đầu vào độ dài tùy ý thành một giá trị có độ dài cố định**. Dữ liệu đầu vào có thể là tài liệu, cây dữ liệu hoặc khối dữ liệu. Ngay cả một sự khác biệt nhỏ trong dữ liệu đầu vào cũng sẽ tạo ra một giá trị đầu ra của hash hoàn toàn khác nhau. Sau đây là hai yêu cầu cơ bản của hàm hash. Thuật toán được chọn cho hàm hash phải là hàm một chiều và nó phải tránh được trùng lặp, hoặc có tỉ lệ trùng lặp rất thấp. Yêu cầu đầu tiên là đảm bảo rằng không ai có thể lấy được tài liệu gốc từ giá trị hash. Ta không thể tạo được khoai tây từ khoai tây nghiền nát. Yêu cầu thứ hai là đảm bảo rằng giá trị hash là duy nhất đại diện cho tài liệu gốc. Sẽ có xác suất cực kỳ thấp là hai bộ dữ liệu khác nhau cho cùng một giá trị hash. Những yêu cầu này đạt được bằng cách chọn một thuật toán mạnh như hash bảo mật (secure hash), và bằng cách sử dụng số lượng bit lớn thích hợp cho giá trị hash. Kích thước hash phổ biến nhất hiện nay là 256 bit và các hàm hash phổ biến nhất là SHA-3, SHA-256 và Keccak. Về giá trị hash, mức băm với 256 bit mạnh đến mức nào? Dung lượng giá trị hash là 256 bit thực sự rất lớn. Nó là giá trị tổ hợp của 2^{256} tương đương 10 lũy thừa của 77. Đó là số 10 với 77 chữ số 0 theo sau. Tỷ lệ của một thiên thạch rơi trúng nhà còn cao hơn việc tạo giá trị băm trùng nhau với 256 bit khi áp dụng thuật toán này. Ta tiếp tục khám phá một số kỹ thuật nữa. Ta sẽ so sánh hai phương pháp tiếp cận hash khác nhau dựa trên cách các thành phần cấu thành được tổ chức: phương pháp hash đơn giản (simple hash) và Merkle tree hash. Trong phương pháp hash đơn giản, tất cả các mục dữ liệu được sắp xếp tuyến tính và được băm. Trong cách tiếp cận với cấu trúc cây, dữ liệu nằm ở mức lá của cây, các lá được hash theo cặp để đi đến cùng một giá trị hash như phương pháp hash đơn giản. Khi chúng ta có một số mục cố định cần băm, chẳng hạn như các mục trong phần header của block, và chúng ta xác minh tính toàn vẹn của block tổng hợp chứ không phải tính toàn vẹn của từng mục, chúng ta dùng simple hash. Khi số lượng các mục khác nhau giữa các block, ví dụ, số lượng giao dịch, số lượng trạng thái, số lượng biên lai, ta sử dụng cấu trúc cây để tính toán giá trị hash. Trạng thái là một biến có thể sửa đổi bằng việc chạy hợp đồng thông minh, và kết quả thực hiện có thể được trả lại trong biên lai. Cấu trúc cây rất hiệu quả cho các hoạt động lặp lại, như sửa đổi giao dịch và thay đổi trạng thái ở các block khác nhau. $O(\log N)$ so với $O(N)$. Tóm lại, trong Ethereum, hàm hash được dùng để sinh ra địa chỉ tài khoản, chữ ký số, hash các: giao dịch, trạng thái, biên nhận và header của block. SHA-3, SHA-256, Keccak-256 là một vài thuật toán thường được sử dụng cho hash trong blockchain.

2.3.3 Tính toàn vẹn của giao dịch

Để quản lý tính toàn vẹn của giao dịch, đầu tiên chúng ta cần bảo mật một địa chỉ tài khoản duy nhất. Ta cần một phương pháp chuẩn để xác định một cách duy nhất thành viên trong mạng phi tập trung. Thứ hai, uỷ quyền giao dịch của người gửi thông qua chữ ký số. Và thứ ba, xác minh rằng nội dung của giao dịch đó không bị sửa đổi. Ta sử dụng tổ hợp của hash và mã hoá với khóa công khai để giải quyết những vấn đề này. Địa chỉ của tài khoản được sinh bằng cách dùng cặp khóa công khai/bí mật. Bước 1, một số độ dài 256 bit được sinh ngẫu nhiên, và được chỉ định làm khóa bí mật/khoá riêng. Giữ khóa này an toàn và dùng mật khẩu để bảo mật. Bước 2, thuật toán ECC được áp dụng cho khóa riêng, để có được khóa công khai duy nhất. Đây là một cặp khóa công khai-bí mật. Bước 3, sau đó áp dụng hàm băm cho khóa công khai để có được địa chỉ tài khoản. Địa chỉ có kích thước ngắn hơn, chỉ 20 byte hoặc 160 bit. Bây giờ ta đã có địa chỉ tài khoản, hãy xem giao dịch khởi phát từ tài khoản này. Giao dịch chuyển tài sản sẽ phải được uỷ quyền, không thể từ chối và không thể sửa đổi được. Đầu tiên ta xem quá trình ký điện tử, sau đó áp dụng nó vào giao dịch. Dữ liệu được băm và mã hoá. Đây chính là chữ ký số. Người nhận nhận dữ liệu gốc cùng secure hash được ký điện tử. Người nhận có thể tính toán lại giá trị băm của dữ liệu gốc đã nhận, và so sánh nó với giá trị băm nhận được để xác minh tính toàn vẹn của tài liệu. Bây giờ, ta xem xét khi giao dịch chính là dữ liệu nói trên. Bước 1, tìm giá trị băm của các trường dữ liệu trong giao dịch. Bước 2, mã hoá giá trị băm đó bằng khóa riêng của người gửi giao dịch. ký điện tử giao dịch để uỷ quyền và thực hiện giao dịch không thể từ chối. Bước 3, giá trị băm này được thêm vào giao dịch. Nó có thể được xác minh bởi những người giải mã nó sử dụng khóa công khai của người gửi trong giao dịch, và tính lại giá trị băm của giao dịch đó. Sau đó, so sánh giá trị băm được tính toán, và giá trị băm nhận được từ chữ ký số. Nếu khớp nhau, đồng ý giao dịch. Ngược lại, hãy từ chối. Hãy lưu ý rằng để xác minh giao dịch hoàn chỉnh, mốc thời gian, nonce, số dư tài khoản và toàn bộ phí cũng được xác minh.

2.3.4 Bảo mật Blockchain

Một số thành phần chính của Block trong Ethereum là header, các giao dịch, bao gồm cả giá trị hash của giao dịch hoặc giao dịch gốc, và state root, giá trị hash của trạng thái hoặc trạng thái gốc. Tính toàn vẹn của block được quản lý bằng cách đảm bảo rằng nội dung trong header của block không bị giả mạo, các giao dịch không bị giả mạo, các chuyển đổi trạng thái được tính toán hiệu quả, được băm và xác minh. Blockchain được cho là một bản ghi bất biến. Trong Ethereum, hash của block là khối của tất cả các thành phần nằm trong header, bao gồm cả các gốc của giao dịch và các hash gốc trạng thái. Nó được tính toán bằng cách áp dụng một biến thể của thuật toán SHA-3 được gọi là Keccak và tất cả các thành phần của header trong block. Một block điển hình có khoảng 2.000 giao dịch với Bitcoin và khoảng 100 giao dịch với Ethereum. Chúng ta cần một cách hiệu quả để phát hiện các giả mạo và thẩm định giao dịch hiệu quả. Hash của giao dịch trong một block được xử lý trong một cấu trúc cây được gọi là Merkle tree hash mà báo cáo đã nói ở trước. Merkle tree hash cũng được sử dụng để tính toán hash gốc trạng thái, vì chỉ băm các chuỗi trạng thái từ block này sang block kia cần phải được tính toán lại. Nó cũng được sử dụng để băm gốc biên nhận. Nếu bất kỳ giao dịch cần được xác minh, chỉ cần kiểm tra một đường dẫn đến cây. Ta không phải trải qua toàn bộ các giao dịch. Hợp đồng thông minh thực thi trong Ethereum dẫn đến thay đổi trạng thái. Mọi thay đổi trạng thái yêu cầu tính toán lại hàm băm gốc trạng thái. Thay vì tính toán băm cho toàn bộ các trạng thái, chỉ có đường dẫn bị ảnh hưởng trong cây Merkle cần phải được tính lại (Chỉ 1 nhánh có lá thay đổi được tính

toán lại, không phải toàn bộ cả cây). Bây giờ, ta chuyển sang tính toán băm block. Băm block trong Ethereum được tính bằng cách tính toán hash gốc trạng thái, gốc giao dịch đầu tiên, sau đó hash gốc biên nhận, được hiển thị ở dưới cùng của header. Những gốc này cùng với tất cả mục khác trong header đều được băm cùng với các nút biến để giải quyết các câu đố Proof-Of-Work. Khối băm phục vụ hai mục đích quan trọng; xác minh tính toàn vẹn của khối và các giao dịch, hình thành liên kết chuỗi bằng cách nhúng khối băm trước đó vào header khối hiện tại. Nếu bất kỳ nút nào của người tham gia giả mạo block, nó thay đổi giá trị băm dẫn đến sự không phù hợp của các giá trị băm và kết xuất chuỗi cục bộ của nút trạng thái không hợp lệ. Bất kỳ khối tương lai nào được khởi tạo bởi nút sẽ bị các miner từ chối do giá trị hash không khớp. Điều này đảm bảo tính bất biến của chuỗi. Tóm lại, một sự kết hợp giữa hashing và mã hoá được dùng để bảo vệ các thành phần các nhau của blockchain. Cặp khoá công khai/bí mật và hashing là các khái niệm nền tảng quan trọng trong các mạng phi tập trung hoạt động ngoài ranh giới của sự tin cậy.

2.4 Những yếu tố thiết yếu trong thiết lập niềm tin

2.4.1 Hệ thống phi tập trung

Trong phần này, ta có thể dễ dàng xác định các yếu tố của sự tin tưởng trong một blockchain, an ninh, xác nhận, xác minh và đồng thuận; tìm hiểu về sự đồng thuận giao thức, một thuật toán tiếp cận để thêm một khối mới và bảo mật chuỗi; giải thích sự tin tưởng và mạnh mẽ của chuỗi chính; minh họa cho sự tin tưởng trong việc quản lý các tình huống đặc biệt như cập nhật phần mềm bắt buộc và cập nhật phần mềm không gây xung đột với phiên bản cũ. Sự tin cậy trong một blockchain phi tập trung liên quan tới việc bảo mật, xác nhận, xác minh và đảm bảo có sẵn các tài nguyên cần thiết để thực hiện giao dịch. Điều này được thực hiện bằng cách bảo đảm rằng chuỗi phải sử dụng một giao thức cụ thể, xác thực giao dịch và chặn giả mạo, xác minh có đủ tài nguyên để thực hiện giao dịch, và thực hiện và xác nhận các giao dịch. "Con đường của sự tin cậy" được thiết lập bởi các hoạt động sau: xác thực giao dịch, xác minh nhiên liệu và tài nguyên, thu thập các giao dịch, thực hiện giao dịch để có được một trạng thái mới, tạo khối, làm việc theo sự đồng thuận, hoàn tất khối bởi người thắng, mọi người đều thêm khối mới đó vào chuỗi của họ và xác nhận sự giao dịch. Chúng ta sẽ kiểm tra từng bước một của các bước này. Bước một và hai là xác nhận giao dịch và kiểm tra các nguồn tài nguyên. Đối với một bitcoin, có khoảng 20 tiêu chuẩn phải được kiểm tra trước khi xác thực một giao dịch như báo cáo đã nói ở phần đầu. Cũng như vậy, đối với một giao dịch Ethereum, cú pháp, chữ ký của giao dịch, thời gian đóng dấu, nonce, số lượng gas tối thiểu, và số dư tài khoản của người gửi trước khi thực hiện giao dịch, nhiên liệu, hoặc số điểm gas, và các tài nguyên khác có sẵn để thực hiện hợp đồng thông minh, cũng được xác thực. Chữ ký giao dịch và hash cũng được xác nhận. Bước thứ ba là thực hiện giao dịch. Hash của cây Merkle của các giao dịch được xác thực được tính toán. Đó là trong Ethereum. Đây là gốc giao dịch của block header. Tất cả các thợ đào đều thực hiện giao dịch cho chuyển khoản cũng như thực hiện cho các hợp đồng thông minh. Trạng thái từ việc thực hiện giao dịch được sử dụng trong việc tính toán hash cây Merkle của các trạng thái, gốc trạng thái của tiêu đề khối. Biên nhận gốc của tiêu đề khối cũng được tính toán.

2.4.2 Giao thức đồng thuận

Chuỗi an toàn (secure chain) là một chuỗi chính duy nhất có trạng thái nhất quán. Mỗi block hợp lệ được thêm vào chuỗi này, sẽ tăng thêm mức độ tin cậy cho chuỗi. Các thợ đào cạnh tranh nhau trong việc thêm block của họ vào chuỗi. Phương thức Bằng chứng Công việc (Proof-Of-Work) được sử dụng để chọn khối tiếp theo được thêm vào chuỗi, trong hoàn cảnh mỗi thợ đào đều đưa ra khối của riêng họ để thêm vào. Proof-Of-Work sử dụng hash. Đây là một ứng dụng nữa của mã băm. Đầu tiên, tính toán giá trị hash của các thành phần trong header khối - một giá trị cố định, và nonce - một biến số. Nếu giá trị hash nhỏ hơn 2^{128} đối với Bitcoin, và nhỏ hơn hàm độ khó đối với Ethereum, câu đố đã được giải. Nếu không giải được, lặp lại quy trình sau khi thay đổi số nonce. Nếu câu đố được giải, thợ đào phát tán (broadcast) block chiến thắng, khối mà sẽ được xác minh bởi các thợ đào khác. Các thợ đào không chiến thắng thêm block mới vào bản sao cục bộ của chuỗi, và tiếp tục làm việc trên block tiếp theo. Người chiến thắng nhận phần thưởng cho việc tạo block. Proof-Of-Work là giao thức đồng thuận được dùng bởi Bitcoin blockchain và phiên bản hiện tại của Ethereum. Giao thức có thể giống nhau, nhưng cách triển khai trong hai blockchain này là khác nhau. Có nhiều cách tiếp cận khác như Bằng chứng Cổ phần (PoS - Proof-Of-Stake), Bằng chứng Thời gian (Proof of Elapsed Time) đã qua được đề xuất. Đây là một đề tài được tranh luận sôi nổi bởi các nhà phát triển blockchain.

2.4.3 Tin tưởng về Robustness

Trust không chỉ là về việc thực hiện các hoạt động thông thường chính xác mà còn là **quản lý những trường hợp ngoại lệ thoả đáng**. Robustness là khả năng quản lý thoả đáng các tình huống ngoại lệ. Yếu tố này rất quan trọng trong một mạng tự hành phi tập trung như blockchain, nơi không có trung gian giám sát nào. Điều gì sẽ xảy ra khi có hơn một thợ đào giải được câu đố đồng thuận, trong khoảng thời gian rất gần nhau? Điều gì nếu có hơn một giao dịch tham chiếu là đầu vào của cùng một tài sản số? Tình huống này được gọi là tiêu 2 lần - double spending. Xử lý các tình huống ngoại lệ đó thoả đáng là cực kỳ quan trọng để đảm bảo tính bảo mật của blockchain. Với trường hợp đầu xảy ra, giao thức Bitcoin cho phép chuỗi này chia thành hai chuỗi ở chu kỳ tiếp theo. Một chuỗi được dẫn dắt bởi các block cạnh tranh. Xác suất block tiếp theo sẽ xảy ra cùng một lúc trong chuỗi này là cực kỳ thấp. Nên sẽ có người thắng chu kỳ tiếp theo của việc tạo block hợp nhất các chuỗi và chuỗi đó trở thành chuỗi được chấp thuận. Trường hợp này, block mới nhất được thêm vào chuỗi chính. Giờ chuỗi này là dài nhất và là chuỗi chính hợp lệ. Giao dịch trong các block khác được trả lại vào pool chưa được xác nhận. Tóm lại với một xác suất cực thấp, chuỗi chính có thể tách ra, nhưng nếu có, giao thức Bitcoin có các cách để hợp

nhất nó thành một chuỗi duy nhất trong vòng một chu kỳ. Về phía Ethereum, giao thức cho phép dùng Ommer hoặc khối về nhì và phân một phần khuyến khích nhỏ thưởng cho các khối về nhì này. Mô hình khuyến khích này giúp giữ an toàn cho chuỗi. Khối mới chỉ được thêm vào chuỗi chính chứ không phải chuỗi về nhì. Những khối về nhì này được giữ thêm 6 block nữa sau khi được thêm vào. Giờ ta nói đến vấn đề tiêu 2 lần. Có khả năng là tiền số và hàng tiêu dùng khác là tài sản số có thể được tái sử dụng có chủ đích hoặc vô ý trong giao dịch. Việc này giống như hãng hàng không đặt vé hai lần cho một ghế trên máy bay. Trong trường hợp này, phi hành đoàn đã giải quyết vấn đề này bằng cách dùng phương thức ad-hoc (tức thời) như hỏi xem có ai sẵn sàng huỷ đặt chỗ của họ và nhận lại tiền, v.v... Trong mạng phi tập trung, như blockchain, không có trung gian. Ta cần một chính sách và một cách xác định tự động để xử lý tình huống này. Chính sách xử lý giao dịch và thanh toán hai lần trong Bitcoin cho phép giao dịch đầu tiên tham chiếu đến tài sản số và từ chối giao dịch còn lại tham chiếu đến cùng một tài sản đó. Trong Ethereum, sự kết hợp của số tài khoản và một nonce toàn cục được dùng để giải quyết vấn đề thanh toán hai lần. Mỗi khi một giao dịch được bắt đầu bởi một tài khoản, số nonce toàn cục sẽ được thêm vào giao dịch. Sau đó, số nonce được tăng lên. Mốc thời gian trên nonce trong giao dịch phải là duy nhất và được xác minh để tránh việc dùng hai lần một tài sản số.

2.4.4 Forks

"Tin tưởng vào nhánh (fork)", "một nhánh được đóng gói", "Rẽ nhánh", "rẽ nhánh cứng" (hard fork), "rẽ nhánh mềm" (soft fork), là những cụm từ phổ biến nhất được sử dụng trong một blockchain. Chúng ta thấy, khi khối Ethereum đạt mức 4,7 triệu nó tiến hành một rẽ nhánh cứng. Rẽ nhánh là quá trình bình thường trên con đường tiến hoá của công nghệ non trẻ sản sinh ra blockchain. Forks là những cơ chế làm tăng độ mạnh mẽ (tăng robustness) của nền tảng Blockchain. Những lần rẽ nhánh được quản lý chặt chẽ giúp xây dựng uy tín trong blockchain bằng cách cung cấp các phương pháp tiếp cận để quản lý lỗi không mong muốn và cải tiến theo kế hoạch. Rẽ nhánh cứng và rẽ nhánh mềm trong thế giới Blockchain giống như đưa các bản vá phần mềm và những phiên bản hệ điều hành mới tương ứng.

Rẽ nhánh mềm là rẽ nhánh nơi các phiên bản cập nhật của giao thức tương thích ngược với những phiên bản trước.

Rẽ nhánh cứng là một sự thay đổi của giao thức không tương thích ngược với các phiên bản cũ hơn của khách hàng. Những người tham gia chắc chắn cần phải nâng cấp phần mềm của họ để nhận ra các khối mới.

3 Smart Contracts

3.1 Những khái niệm cơ bản về Smart Contracts - hợp đồng thông minh

3.1.1 Tại sao nên sử dụng Smart Contracts?

Hợp đồng thông minh là một tính năng cực kỳ mạnh mẽ của Blockchain, nó là phần tử tính toán của chuỗi khối. Nhắc lại từ phần trước của báo cáo, Bitcoin có một tính năng gọi là script để chứa những luật lệ và chính sách (rule and policies). Bitcoin sử dụng script được đính kèm với giao dịch để thêm những điều kiện cần thực hiện khi chuyển giao tài sản. Những script này khá đơn giản và có khả năng giới hạn. Sau đó ta có Ethereum được tiến hóa từ Bitcoin. Một sự đóng góp to lớn của Ethereum chính là một lớp hợp đồng thông minh nơi hỗ trợ thực thi bất kỳ code nào trên Blockchain. Từ đó nó sẽ giúp người dùng thực hiện những hoạt động có độ phức tạp lớn hơn, tăng khả năng tương thích của Ethereum Blockchain để trở thành một hệ thống tính toán phi tập trung mạnh mẽ.

Như đã nói ở phần trước, giao dịch tiền ảo nhiều lúc không phải cứ đưa tiền là xong. Chúng ta muốn có thể hẹn giờ tự động giao dịch, hoặc trả góp cho một món hàng hàng tháng với một lãi suất cụ thể. Những việc này mang đến những điều kiện, điều luật, chính sách mà những giao thức chuyển giao tiền mã hóa cần phải xử lý được. Hợp đồng thông minh đáp ứng nhu cầu này cho những ứng dụng xác thực cụ thể cho Blockchain. Hợp đồng thông minh có một số điểm mạnh bao gồm: hợp đồng thông minh tạo điều kiện thuận lợi cho giao dịch chuyển các tài sản khác ngoài tiền điện tử, hợp đồng thông minh cho phép đặc tả các quy tắc, luật lệ cho một hoạt động trên blockchain. Nó tạo điều kiện cho việc thực hiện các chính sách chuyển giao tài sản trong một mạng lưới phi tập trung. Nó cũng bổ sung khả năng lập trình và trí thông minh cho Blockchain.

Một hợp đồng thông minh đại diện cho một lớp business logic, với một logic cụ thể được lập trình trong một ngôn ngữ bậc cao. Một hợp đồng thông minh chứa những hàm có thể kích hoạt chạy bằng những tin nhắn hoạt động như function calls. Những tin nhắn này cùng tham số dành cho hàm chứa trong tin nhắn được nêu cụ thể trong các giao dịch.

Ta có thể so sánh giao dịch Bitcoin với một giao dịch hợp đồng thông minh. Trong bitcoin, mọi giao dịch đều chỉ là về chuyển tiền. Trong trường hợp một blockchain có hỗ trợ hợp đồng thông minh, giao dịch tại blockchain này có thể chạy nhiều hàm được cài đặt bởi smart contracts, cung cấp một lớp tính toán logic có thể thực thi được. Từ đó tăng tính đa dụng, linh hoạt và mạnh mẽ của giao dịch để có thể ứng dụng vào nhiều lĩnh vực, bài toán cụ thể. Ví dụ, một giao dịch kinh doanh có thể liên quan đến các quy tắc, chính sách, luật, quy định và bối cảnh quản lý. Hợp đồng thông minh cho phép các ràng buộc khác trong thế giới thực này được thực hiện trên blockchain, do đó, hợp đồng thông minh cho phép nhiều ứng dụng phi tập trung có độ phức tạp tùy ý được triển khai trên blockchain.

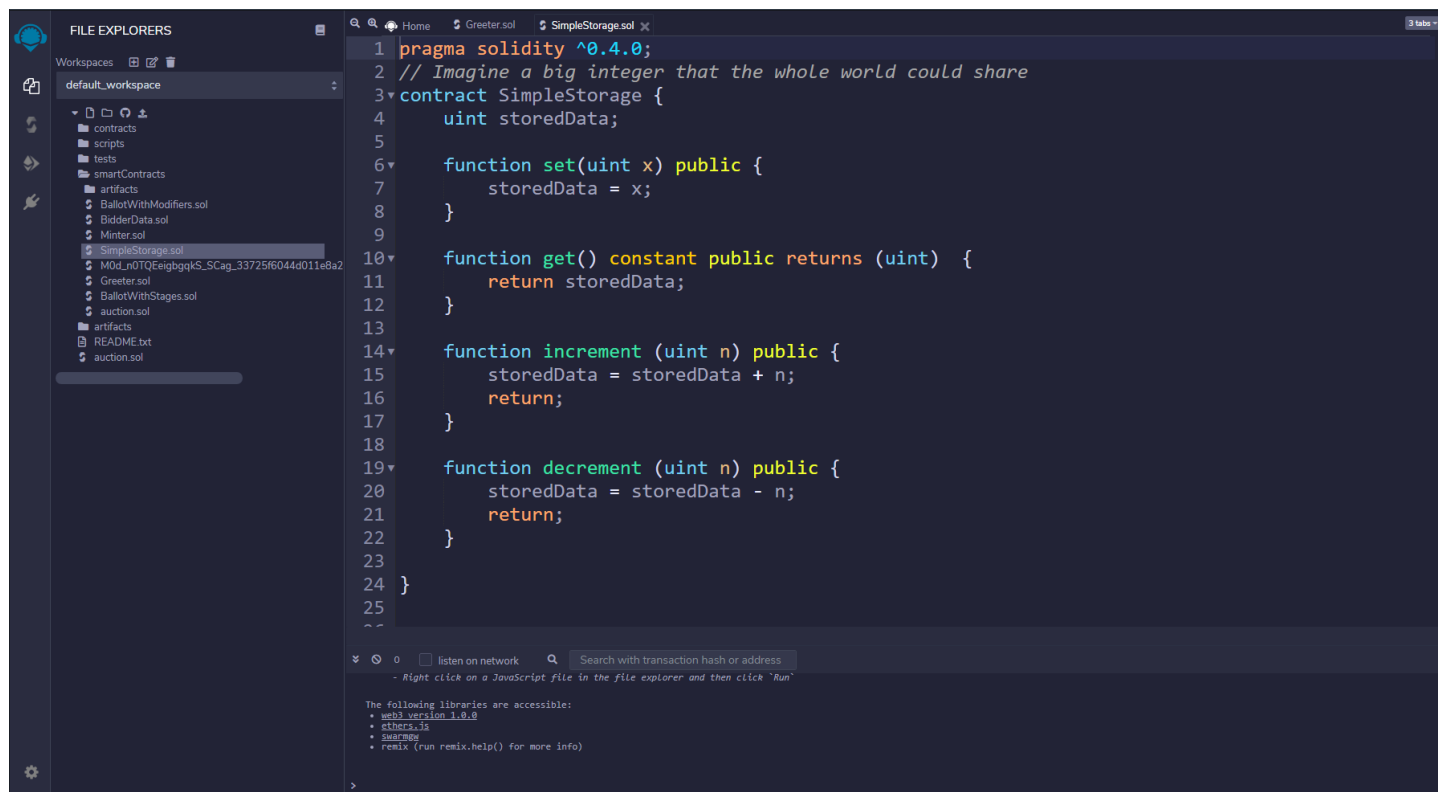
3.1.2 Định nghĩa Smart Contracts

Khi đã được triển khai trên Blockchain, hợp đồng thông minh là **một đoạn code không thể thay đổi**. Chúng ta sẽ cần phải triển khai lại một hợp đồng thông minh mới, hoặc bằng cách nào đó chuyển hướng tin nhắn gọi từ hợp đồng cũ sang hợp đồng mới.

Hợp đồng thông minh có thể chứa các biến bên trong nó và gọi là các biến trạng thái. Ta có thể lấy dữ liệu các biến này và quan sát các biến thay đổi qua các block như thế nào. Một hợp đồng thông minh của Ethereum Blockchain có những thành phần sau:

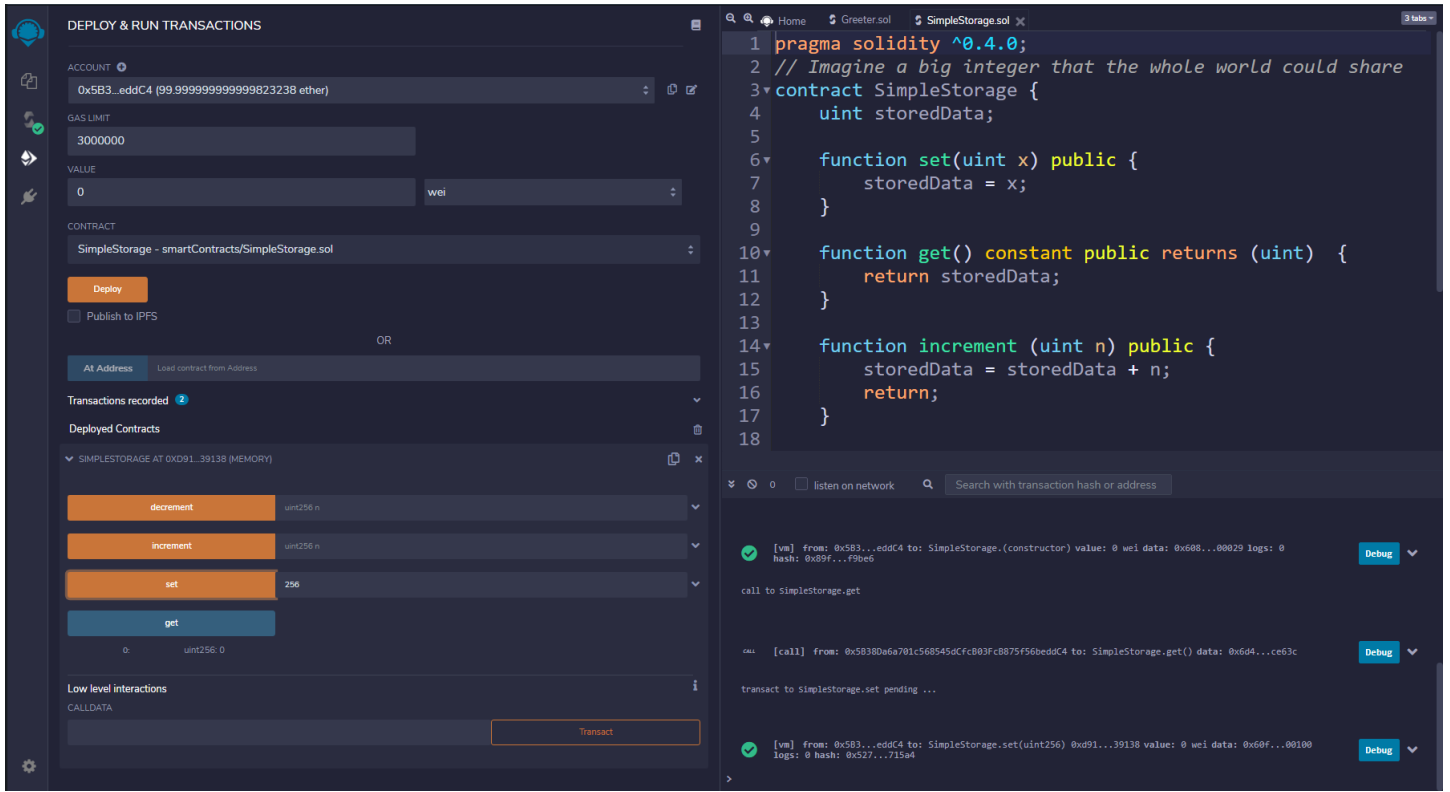
- Pragma directive để chỉ phiên bản ngôn ngữ Solidity cho Compiler
- Tên của hợp đồng
- Dữ liệu hoặc các biến trạng thái tạo nên trạng thái của hợp đồng
- Tập hợp các hàm phục vụ mục đích tạo ra hợp đồng
- Một vài thành phần phụ khác sẽ được nhắc tới ở những phần sau

Ta sẽ sử dụng Remix IDE, một môi trường phát triển phổ biến nhất dành cho ngôn ngữ Solidity và phát triển các hợp đồng thông minh.



Hình 8: Giao diện của Remix IDE, có thể sử dụng tại địa chỉ remix.ethereum.org

Ở side pane của Remix IDE có các tab File Explorer, Compile, Deploy, Plugin Manager. Vì vậy ta có thể làm việc dễ dàng kể cả khi đây là một webapp IDE, ta có thể tải xuống và chạy local không cần mạng với việc cài đặt qua npm.



Hình 9: Chạy thử smart contract ngay trên Remix IDE

Sau khi đã dịch chương trình smart contract trên đây, ta có thể triển khai hợp đồng để kiểm thử ngay lập tức. Sẽ có các nút cùng textbox để điền tham số và chạy các hàm (nút màu cam). Kết quả chạy sẽ được hiển thị trên terminal bên dưới code editor. Ta cũng sẽ có các nút để lấy giá trị các biến trạng thái (nút màu xanh).

3.1.3 Xử lý Smart Contracts

Một hợp đồng thông minh có thể được tạo ra với tư cách của một Externally Owned Account, trong một ứng dụng với giao diện Command-line hoặc giao diện web, UI chạy lệnh. Nó cũng có thể được tạo ra bên trong một hợp đồng thông minh khác. Mỗi hợp đồng cần có một địa chỉ để có thể triển khai và chạy hàm. Địa chỉ này được tính bằng cách băm số địa chỉ của Externally Owned Account và nonce.

Khi đã biên dịch xong hợp đồng thông minh, trình biên dịch sẽ tạo ra một số artifacts (có thể gọi là thuộc tính hoặc thông tin của hợp đồng thông minh để sử dụng trên các nền tảng khác).

WEB3DEPLOY



```
var simplestorageContract = new web3.eth.Contract([{"constant":false,"inputs":[{"data: '0x608060405234801561001057600080fd5b50610174806100206000396000f300608arguments: [
]
})).send({
  from: web3.eth.accounts[0],
  gas: '4700000'
}, function (e, contract){
  console.log(e, contract);
  if (typeof contract.address !== 'undefined') {
    console.log('Contract mined! address: ' + contract.address + ' transacti
  }
})
```

METADATAHASH



```
"1c8d13056bc56450316b627edc90ffcd33d0b4ad6f0f291dc42b3702b4cc9dc0"
```

SWARMLOCATION



```
"bzzr://1c8d13056bc56450316b627edc90ffcd33d0b4ad6f0f291dc42b3702b4cc9dc0"
```

FUNCTIONHASHES



```
{ "3a9ebefd": "decrement(uint256)", "6d4ce63c": "get()", "7cf5dab0": "inc
```

Hình 10: Một số artifact sau khi biên dịch hợp đồng thông minh SimpleStorage

Trong ảnh trên, có artifact Web3Deploy cần phải sử dụng nếu ta muốn dùng hợp đồng thông minh trên một ứng dụng Web. Còn có nhiều artifact khác, mỗi thứ được sử dụng cho mỗi mục đích khác nhau như:

- Tên của hợp đồng
- Bytecode đã thực thi để tạo hợp đồng trên máy ảo Ethereum
- ABI - Application Binary Interface, chi tiết về các hàm, tham số và kiểu trả về
- Ước tính lượng gas cần để thực thi các hàm
- Bytecode thực thi thực tế của hợp đồng thông minh

3.1.4 Triển khai Smart Contracts

Đầu tiên, một giải pháp hợp đồng thông minh được viết trên một ngôn ngữ lập trình bậc cao và được dịch thành bytecode. Một ABI cũng sẽ được tạo ra để ứng dụng vào các ngôn ngữ bậc cao khác, ví dụ như ứng dụng web để tương tác với mã nhị phân của hợp đồng thông minh. Máy ảo Ethereum sẽ cung cấp môi trường thực thi cho bytecode của hợp đồng thông minh.

Sau đây là các bước hoàn chỉnh để triển khai hợp đồng thông minh, sử dụng Remix IDE:

1. Viết mã hợp đồng thông minh trên Remix IDE và tiến hành dịch
2. Remix tạo ra nhiều artifacts
3. Để dễ dàng triển khai, Remix cung cấp cho chúng ta script triển khai Web3 - nơi chứa bytecode, ABI và thông tin tài khoản
4. Để triển khai hợp đồng ta chỉ cần thực thi script nói trên
5. Khi quá trình triển khai hoàn thành, địa chỉ hợp đồng được tạo ra bằng cách dùng địa chỉ tài khoản và nonce
6. Để tương tác với hợp đồng thông minh ta sẽ sử dụng địa chỉ của hợp đồng, ABI và mã băm của các hàm.

3.2 Ngôn ngữ Solidity

3.2.1 Cấu trúc của Solidity

Cấu trúc cụ thể của ngôn ngữ Solidity như sau:

- Dữ liệu hoặc các biến trạng thái.
- Hàm: Có nhiều loại hàm được phép sử dụng:
 - Constructor (có thể dùng mặc định hoặc tự định nghĩa, và chỉ có một, có nghĩa là hàm khởi tạo không thể overload - không thể viết đè lên)
 - Các hàm fallback (đây là một tính năng mạnh mẽ của một hàm ẩn danh)
 - Các hàm quan sát
 - Hàm thuần túy (không thay đổi trạng thái, chỉ tính toán và trả lại giá trị - ví dụ: các hàm toán học)
 - Hàm công khai - public (có thể truy cập thông qua giao dịch, thay đổi trạng thái được ghi lại trên blockchain)
 - Hàm riêng tư - private (chỉ có thể truy cập từ bên trong hợp đồng chứa nó)
 - Hàm nội bộ - internal (chỉ có thể truy cập bên trong hợp đồng hiện tại và các hợp đồng kế thừa từ hợp đồng hiện tại)
 - Hàm ngoài - external (chỉ có thể truy cập từ bên ngoài hợp đồng)
- Các struct và enum được người dùng tự định nghĩa
- Các modifier để xét điều kiện thực thi hàm
- Các events - để log thông tin và gửi thông báo về ứng dụng client

Các hàm trong hợp đồng thông minh khá tương đồng với các hàm trong những ngôn ngữ lập trình bậc cao khác. Function header đi kèm với mã của hàm nằm trong cặp ngoặc nhọn. Mã của hàm có thể chứa những dữ liệu cục bộ và lệnh để xử lý dữ liệu và trả về kết quả xử lý.

Từ khóa "function" sẽ luôn bắt đầu ở mọi hàm. Các tham số có thể là một hoặc nhiều cặp (kiểu dữ liệu, tên biến) - ví dụ: UInt count. Giá trị trả về cũng có thể là cặp (kiểu dữ liệu, tên biến) hoặc chỉ có mỗi (kiểu dữ liệu). Khi chỉ có mỗi kiểu dữ liệu được chỉ ra thì ta phải dùng một lệnh return để chỉ rõ là trả về giá trị nào. Ta có thể trả về bao nhiêu giá trị tùy thích.

3.2.2 Kiểu dữ liệu và luồng điều khiển cơ bản

Ta cần nhớ rằng các hoạt động trong mã nguồn hợp đồng thông minh khi được thực thi sẽ tiêu tốn gas - hay bất cứ nhiên liệu tiền mã nào (crypto fuel). Từ đó ta phải biết chọn những cấu trúc dữ liệu phù hợp để tiết kiệm gas và sử dụng hiệu quả.

Bất kỳ những lệnh cơ bản phổ biến nào có trong những ngôn ngữ lập trình bậc cao đều có trong ngôn ngữ Solidity. Các lệnh gán, if-else, for, while, v.v... Ta sẽ lấy ví dụ cụ thể của một hợp đồng thông minh để thấy rõ các thành phần có trong mã:

```

1  pragma solidity ^0.4.0;
2
3  contract Bidder {
4
5      string public name;
6      uint public bidAmount = 20000;
7      bool public eligible;
8      uint constant minBid = 1000;
9
10 }

```

Listing 2: Mã nguồn ví dụ cho một Smart Contracts chỉ có dữ liệu

Ở đây hợp đồng thông minh đang chỉ có mỗi dữ liệu để chúng ta có thể phát triển theo từng bước. Các dữ liệu trong hợp đồng được set ở trạng thái public để các giao diện web hay nhiều loại client khác có thể truy cập vào mà không cần dùng đến hàm.

Tiếp đó ta thiết kế cách hàm cho hợp đồng thông minh này:

```

1  pragma solidity ^0.4.0;
2
3  contract Bidder {
4
5      string public name = "Buffalo";
6      uint public bidAmount;
7      bool public eligible;
8      uint constant minBid = 1000;
9
10     function setName(string nm) public {
11         name = nm;
12     }
13
14     function setBidAmount(uint x) public {
15         bidAmount = x;
16     }
17
18     function determineEligibility() public {
19         if (bidAmount >= minBid) eligible = true;
20         else eligible = false;
21     }
22 }

```

Listing 3: Mã nguồn ví dụ cho một Smart Contracts hoàn chỉnh

Qua đó ta thấy một hợp đồng thông minh có thể chứa 3 thành phần cơ bản chính như sau:

1. Tên của hợp đồng
2. Các dữ liệu/trạng thái
3. Các hàm

Và ta luôn cần lưu ý phải thiết kế trước khi bắt tay vào code. Thiết kế class diagram, nhận biết rõ cần sử dụng những dữ liệu và hàm nào để tối ưu nhiên liệu và hiệu quả. Class diagram cũng hữu hiệu trong việc thiết kế sự thừa kế giữa các hợp đồng thông minh khác nhau.

3.2.3 Các kiểu dữ liệu đặc thù

Ngoài các kiểu dữ liệu chung phổ biến có trong nhiều ngôn ngữ lập trình bậc cao, Solidity còn có những kiểu dữ liệu riêng là **address** (địa chỉ) và **message** (tin nhắn), kèm theo đó là **mapping**.

Address là kiểu dữ liệu chứa 20 byte địa chỉ Ethereum. Address là nền tảng của hợp đồng thông minh.

Mapping giống Map trong C++ hoặc Dictionary của Python. Nó có cấu trúc <key, value> như một hash table, ánh xạ từ 1 key duy nhất tới value của nó.

Message đại diện cho "cuộc gọi" để kích hoạt một hàm trong hợp đồng thông minh. Trong kiểu dữ liệu này hiện tại ta chỉ cần quan tâm tới 2 thuộc tính. Đầu tiên là **msg.sender** chứa địa chỉ của người gửi tin nhắn - người kích hoạt hàm. Thứ hai là **msg.value** chứa giá trị tính bằng Wei cũng được gửi bởi người kích hoạt hàm. Từ đó ta có thể viết các lệnh để kiểm tra điều kiện kích hoạt hàm của thỏa mãn hay không trước khi chạy lệnh trong hàm.

Ta có thể xem mã nguồn ví dụ sau để hiểu hơn về các kiểu dữ liệu riêng này:

```

1  pragma solidity ^0.4.0;
2
3  contract Coin {
4      // The keyword "public" makes those variables
5      // readable from outside.
6      address public minter;
7      mapping (address => uint) public balances;
8
9
10 }

```

```

9 // Events allow light clients to react on
10 // changes efficiently.
11 event Sent(address from, address to, uint amount);
12
13 // This is the constructor whose code is
14 // run only when the contract is created.
15 function Coin() public{
16     minter = msg.sender;
17 }
18
19 function mint(address receiver, uint amount) public {
20     if (msg.sender != minter) return;
21     balances[receiver] += amount;
22 }
23
24 function send(address receiver, uint amount) public {
25     if (balances[msg.sender] < amount) return;
26     balances[msg.sender] -= amount;
27     balances[receiver] += amount;
28     Sent(msg.sender, receiver, amount);
29 }
30 }

```

Listing 4: Mã nguồn ví dụ cho một Smart Contracts với các kiểu dữ liệu đặc thù

Ở đây ta đã sử dụng kiểu `address` cho biến **minter** để lưu địa chỉ người tạo hợp đồng thông minh. Đồng thời một ánh xạ **balances** lưu số dư từng tài khoản cũng được sử dụng. Khi đó ta có thể kết hợp với kiểu message để kiểm tra điều kiện chạy hàm. **if (msg.sender != minter)** kiểm tra xem người kích hoạt hàm có phải là người tạo hợp đồng thông minh hay không, nếu không thì người này không có quyền chạy hàm.

3.2.4 Cấu trúc dữ liệu

Ngoài ra ta có thể tự định nghĩa nhiều kiểu dữ liệu tùy thích cho hợp đồng thông minh. Các cấu trúc đó là **struct** và **enum**.

Struct là kiểu dữ liệu tổng hợp của một nhóm dữ liệu liên quan có thể được tham chiếu bởi một tên tập thể ý nghĩa.

Enum hoặc gọi là kiểu dữ liệu liệt kê cho phép các kiểu dữ liệu do người dùng xác định với tập hợp giới hạn là các giá trị có ý nghĩa.

```

1 pragma solidity ^0.4.0;
2 contract Ballot {
3
4     struct Voter {
5         uint weight;
6         bool voted;
7         uint8 vote;
8         address delegate;
9     }
10    struct Proposal {
11        uint voteCount;
12    }
13    enum Stage {Init,Reg, Vote, Done}
14    Stage public stage = Stage.Init;
15
16    address chairperson;
17    mapping(address => Voter) voters;
18    Proposal[] proposals;
19
20    event votingCompleted();
21
22    uint startTime;
23    //modifiers
24    modifier validStage(Stage reqStage)
25    { require(stage == reqStage);
26      -;
27    }
28
29    /// Create a new ballot with $( _numProposals ) different proposals.
30
31    function Ballot(uint8 _numProposals) public {
32        chairperson = msg.sender;
33        voters[chairperson].weight = 2; // weight is 2 for testing purposes
34        proposals.length = _numProposals;
35        stage = Stage.Reg;
36        startTime = now;
37    }
38
39    /// Give $(toVoter) the right to vote on this ballot.
40    /// May only be called by $(chairperson).

```

```

41 function register(address toVoter) public validStage(Stage.Reg) {
42     //if (stage != Stage.Reg) {return;}
43     if (msg.sender != chairperson || voters[toVoter].voted) return;
44     voters[toVoter].weight = 1;
45     voters[toVoter].voted = false;
46     if (now > (startTime+ 30 seconds)) {stage = Stage.Vote; }
47 }
48
49 /// Give a single vote to proposal $(toProposal).
50 function vote(uint8 toProposal) public validStage(Stage.Vote) {
51     // if (stage != Stage.Vote) {return;}
52     Voter storage sender = voters[msg.sender];
53     if (sender.voted || toProposal >= proposals.length) return;
54     sender.voted = true;
55     sender.vote = toProposal;
56     proposals[toProposal].voteCount += sender.weight;
57     if (now > (startTime+ 30 seconds)) {stage = Stage.Done; votingCompleted();}
58
59 }
60
61 function winningProposal() public validStage(Stage.Done) constant returns (uint8 _winningProposal) {
62     //if(stage != Stage.Done) {return;}
63     uint256 winningVoteCount = 0;
64     for (uint8 prop = 0; prop < proposals.length; prop++)
65         if (proposals[prop].voteCount > winningVoteCount) {
66             winningVoteCount = proposals[prop].voteCount;
67             _winningProposal = prop;
68         }
69     assert (winningVoteCount > 0);
70
71 }
72 }

```

Listing 5: Mã nguồn ví dụ cho một Smart Contracts các cấu trúc dữ liệu riêng của Solidity

Trên đây mã nguồn của hợp đồng thông minh dùng để chạy các hoạt động bầu cử phân tán đã sử dụng struct, enum. Struct **voter** dùng để chứa các trường lưu thông tin của người đi bầu cử. Enum **Stage** dùng để chứa các giá trị của các giai đoạn đợt bầu cử, các giá trị này bản chất chính là số nguyên.

Ngoài ra trong mã nguồn này cũng đã sử dụng array và modifier. Modifier được ví như là "người gác cổng" bảo vệ hàm. Ở hợp đồng thông minh "Coin" phía trên nữa, ta đã sử dụng lệnh if-else để kiểm tra điều kiện. Khi điều kiện không thỏa mãn, các dòng lệnh phía dưới sẽ không chạy. Tuy nhiên hàm sẽ được coi là đã kích hoạt và hoạt động của hàm/giao dịch sẽ được ghi lại trên blockchain. Modifier khắc phục điểm yếu này. Modifier của hàm sẽ ngăn chặn không cho thay đổi trạng thái của hợp đồng thông minh. Hơn nữa, nó được gọi từ chính bên trong hợp đồng thông minh chứ không phải từ giao dịch, vì vậy hoạt động sẽ không bị ghi lại trên blockchain.

Yếu tố cuối cùng ta cần bàn đến đó chính là thời gian. Trên một ứng dụng blockchain, toàn bộ thành viên và các nút đều phải được đồng bộ trên cùng một mốc thời gian. Để phục vụ mục đích này, Blockchain có một máy chủ thời gian để cung cấp Unix Epoch Time - thời gian tính từ ngày 1/1 năm 1970 bằng giây. Mốc thời gian này dùng để gắn nhãn thời gian các khối. Khi một khối được thêm vào blockchain, mọi giao dịch được xác nhận trên khối đều có nhãn thời gian của khối làm thời điểm xác nhận giao dịch. Trong hợp đồng thông minh, ta có thể sử dụng biến **"now"** để lấy nhãn thời gian này. Lưu ý, biến **now** không phải thời điểm giao dịch được tạo, mà là thời điểm giao dịch được xác nhận. Tận dụng biến thời gian này ta có thể cài đặt các điều kiện ràng buộc về thời gian ví dụ như kiểm chứng các giai đoạn bầu cử, hết thời gian được bầu rồi thì người gửi tin nhắn kích hoạt hàm sẽ không được bầu nữa.

3.3 Ứng dụng thực tiễn

3.3.1 Phát triển Smart Contracts

Ở phần này ta sẽ bàn về những phương pháp phát triển hợp đồng thông minh. Bắt đầu bằng việc đặt vấn đề, phân tích bài toán để tìm ra thiết kế cơ bản, những biến trạng thái và các hàm của nó, luôn nhớ rằng cần phải thiết kế trước khi lập trình bằng cách sử dụng class diagram. Dựa vào bài toán, xác định visibility cho các hàm và biến. Dựa trên yêu cầu, định nghĩa các modifier kiểm soát quyền truy cập cho các hàm, định nghĩa cách kiểm chứng các biến đầu vào kết hợp lệnh assert.

Ta có thể sử dụng event để trình bày kết quả hoặc tiến trình đạt được cho ứng dụng client (giao diện người dùng hoặc một phần mềm quản lý theo dõi giao dịch). Kết hợp với event có listener code được dùng để:

- Theo dõi giao dịch
- Nhận kết quả thông qua tham số của event
- Khởi tạo yêu cầu lấy thông tin từ hợp đồng thông minh

Những yếu tố này sẽ được trình bày rõ hơn trong các phần dưới đây.

3.3.2 Yếu tố thời gian

Ta xem lại hợp đồng Ballot cho bầu cử đã phát triển trong phần trước. Trong một tiến trình bầu cử thông thường, các cử tri trước tiên được ghi danh. Thường có một khoảng thời gian cho việc ghi danh, và cũng có khoảng thời gian để bỏ phiếu. Ví dụ như, đối với hầu hết các bang tại Mỹ, ta phải ghi danh 30 ngày trước ngày bầu cử, và việc ghi danh diễn ra vào một ngày cụ thể để mỗi một cử tri tự thực hiện. Nếu đó là vấn đề, việc ghi danh phải hoàn thành trước khi bầu cử. Ta đã thêm enum cho các giai đoạn và logic cho việc điều chỉnh các giai đoạn trong các hàm. Stage gồm 4 giai đoạn/bước khác nhau, Init, Reg, Vote và Done. Các bước được thực hiện từ Init vào lúc triển khai Smart Contract. Sau đó, trong Constructor, Init chuyển sang bước Reg. Sau khi giai đoạn ghi danh hoàn tất, chuyển sang Vote. Sau giai đoạn bỏ phiếu, stage được cài đặt sang Done. Hãy thêm logic này vào smart contract Ballot và dùng cái này để cài đặt các bước của smart contract. Chúng ta cũng sẽ thêm các thành phần thời gian. Solidity định nghĩa sẵn một biến thời gian "now", tức là thời gian khối hiện tại. Ta đã thêm biến uint startTime được khởi tạo giá trị now trong constructor. Sau đó, thay đổi stage của quá trình bầu cử dựa trên thời gian được cài đặt cho bước ghi danh và bỏ phiếu như đã nói. Chúng ta đã thêm biến startTime, và giai đoạn ghi danh trong trường hợp này là 10 ngày. Chúng ta cũng đã thêm thời gian cho giai đoạn bỏ phiếu, trong trường hợp này là 1 ngày. Biến now trong Solidity now là thời gian khối, với hàm block.timestamp, khi mà giao dịch được xác nhận. Thêm nữa, now có thể không thể hiện chính xác thời gian đã trôi qua. Nếu dùng cho các khoảng thời gian gần đúng và cho việc kiểm tra các tình huống đơn giản, now là một thuộc tính thời gian thuận tiện. Trong một ứng dụng thực tế, với những mốc thời gian cụ thể, một giải pháp hữu hiệu hơn sẽ là đặt deadline dưới dạng thời gian epoch trong Constructor thời điểm hợp đồng được tạo ra, rồi so sánh nó với nhãn thời gian block khi cần. Ta cần nhớ là thời gian khối được đại diện bởi biến "now".

3.3.3 Thẩm định và kiểm thử

Mục đích của phần này giải thích về khái niệm hoàn nguyên/khôi phục (revert) một giao dịch dựa trên việc xác định các vấn đề cụ thể (problem-specific validation) và sử dụng từ khóa revert, áp dụng các khái niệm về modifier dành cho hàm, require và assert.

Solidity cung cấp cho chúng ta chức năng revert để khôi phục lại kết quả khi gặp trường hợp ngoại lệ. Xử lý ngoại lệ sẽ hoàn tác tất cả các thay đổi được thực hiện trong trạng thái hiện tại và khôi phục giao dịch và đồng thời cũng gắn cờ lỗi (error flag) đối với người gọi.

Nhắc lại về Modifier cùng lệnh Require và Assert, đây là cách để chúng ta kiểm tra các điều kiện thực thi hàm mà không cần đi vào trong chạy code của hàm đó (không kích hoạt hàm). Từ đó giao dịch hay hoạt động gọi hàm sẽ không bị ghi lại trên blockchain. Ví dụ sử dụng modifier có thể thấy trên ví dụ code hợp đồng bầu cử vừa rồi.

3.3.4 Ứng dụng client

Chúng ta sẽ dùng một tính năng của Solidity gọi là event (sự kiện) để giao tiếp với ứng dụng client. Phần này sẽ giải thích khái niệm của Event: định nghĩa một sự kiện và thúc đẩy một event đến một listener (người nghe) đã đăng ký. Một định dạng chung của event là: từ khóa event, tên của event và các tham số. Ví dụ, event votingCompleted trên hợp đồng bầu cử. Ở đây không có tham số nào. Ta kích hoạt một event sử dụng tên của nó và bất cứ tham số nào. Trong hàm vote(), khi stage thay đổi sang Done, chúng ta triệu gọi event ở cuối giai đoạn bỏ phiếu. Có nhiều lợi ích khi ghi nhật ký sự kiện (hay còn gọi là logging). Thường thì một event là để báo cáo cho một ứng dụng client, giao diện người dùng hoặc một ứng dụng theo dõi giao dịch rằng ta đã có được kết quả hoặc đã đạt tới một mốc quan trọng. Ứng dụng có thể lắng nghe các event được đẩy ra, sử dụng listener code, để theo dõi các giao dịch, để nhận các kết quả thông qua các tham số của event, khởi tạo yêu cầu lấy thông tin từ hợp đồng thông minh. Chúng ta sẽ tìm hiểu những bộ xử lý event này trong các phần tiếp theo về các ứng dụng phi tập trung, DApp.

3.4 Best practices

Blockchain không phải là giải pháp cho mọi ứng dụng. Ta cần biết chắc rằng ứng dụng của mình yêu cầu những tính năng của Blockchain. Nói cách khác, các giải pháp dựa trên blockchain và hợp đồng thông minh không phải là thuốc chữa bách bệnh cho tất cả các vấn đề ta gặp phải. Giải pháp Blockchain thích hợp nhất cho các ứng dụng có những đặc điểm sau:

- Các vấn đề phi tập trung, có nghĩa là những thành viên / người tham gia nắm giữ tài sản đều không được định vị và biết đến.
- Liên quan với giao dịch ngang hàng (peer-to-peer) mà không cần đến bên trung gian.
- Hoạt động vượt ra ngoài giới hạn tin tưởng giữa những thành viên không biết thông tin.
- Yêu cầu kiểm chứng, xác thực và ghi lại những sổ cái toàn cầu bất biến có mốc thời gian.
- Những hoạt động tự động được chỉ dẫn với nhiều luật lệ và chính sách.

Ta cũng cần phải biết chắc rằng ta có cần hợp đồng thông minh trên blockchain cho ứng dụng của mình hay không. Ta hiểu rằng hợp đồng thông minh có thể được xem bởi mọi thành viên trên chuỗi và được thực thi trên toàn bộ các nút. Ta sẽ cần một hợp đồng thông minh khi ta có một tập hợp các thỏa thuận tập thể dựa trên các quy tắc, quy định, chính sách

hoặc quản trị được thực thi và quyết định cũng như nguồn gốc của nó phải được ghi lại. Hợp đồng thông minh không thay thế máy chủ client của chúng ta hay các giải pháp phân tán không trạng thái vốn có.

3.4.1 Đánh giá và thiết kế Smart Contracts

Khi thiết kế ta cần lưu ý giữ cho mã nguồn hợp đồng thông minh đơn giản, mạch lạc và dễ nhìn. Không được thêm các biến dư thừa và những hàm không liên quan, cần thiết. Ta có thể khiến code dễ hình hơn bằng cách sử dụng function modifier tùy biến mà ta tự viết thay vì các lệnh if-else để kiểm tra các điều kiện trước và sau khi thực thi hàm.

Khi viết mã ta nên viết hàm theo một thứ tự nhất định, thứ tự này dựa theo visibility của hàm. Documentation của Solidity đã nêu ra thứ tự sau:

1. Constructor
2. Hàm fallback
3. External
4. Public
5. Internal
6. Private

Còn vài điều khác nên nhớ khi viết mã Solidity. Đó là dùng modifier "payable" khi các hàm gửi giá trị tiền mã hóa. Sử dụng event trong hợp đồng thông minh để thông báo lại cho ứng dụng phân tán và log lại dữ liệu. Sử dụng mã băm an toàn để bảo vệ dữ liệu. Lưu ý tới Console của Remix IDE, Compiler, Transaction Log và Chi tiết phân tích dữ liệu để nhận biết được những vấn đề đang hiện hữu và giải quyết nó.

Blockchain không phải là một kho dữ liệu. Ta chỉ nên giữ những thông tin cần thiết trên chuỗi và trên hợp đồng thông tin. Một cách hay là phân biệt được đâu sẽ là dữ liệu nằm trên chuỗi (on-chain) và nằm ngoài chuỗi (off-chain). Những dữ liệu lớn có thể cho làm off-chain data và trên chuỗi ta sẽ lưu những metadata chỉ rõ địa chỉ hoặc hướng tới những dữ liệu off-chain lớn này.

4 Decentralized Applications

4.1 Giới thiệu

Các ứng dụng phi tập trung hay gọi tắt là DApp, mở ra nhiều tính năng và dịch vụ của Blockchain để chúng ta có thể khai thác tương tác và giải trí. DApp cấp quyền truy cập cho tất cả mọi người, các ứng dụng và hệ thống không nhất thiết phải biết nhau mà vẫn có thể giao dịch ngang hàng. DApp là một ứng dụng đầu cuối trên nền tảng Blockchain. **Ứng dụng phi tập trung giải quyết bài toán mà có yêu cầu các dịch vụ blockchain và cơ sở hạ tầng blockchain để thực hiện mục đích của nó.**

4.1.1 Blockchain Server

Ta bắt đầu phần này với những khái niệm trọng yếu, đầu tiên là máy chủ blockchain.

Ta sẽ dùng thuật ngữ máy chủ blockchain để làm nói về **cơ sở hạ tầng và chức năng mà blockchain cung cấp**. Thuật ngữ này được rút ra từ sự tương đồng trong máy chủ web và ứng dụng web. máy chủ di động, ứng dụng di động, máy chủ dữ liệu, và ứng dụng dữ liệu, v.v... Do đó thuật ngữ "blockchain server" để chỉ toàn bộ chức năng mà nó cung cấp, Máy chủ blockchain và DApp. Ở đây ta có thể nhìn thấy tập hợp các lệnh cài đặt các dịch vụ Ethereum và giao diện lập trình ứng dụng (API) để cho phép tạo nút, phát triển ứng dụng, và các giao dịch trên nền tảng blockchain:

1. `sudo apt-get install software-properties-common`
2. `sudo add-apt-repository -y ppa:ethereum/ethereum`
3. `sudo apt-get update`
4. `sudo apt-get install ethereum`

Thuật ngữ "blockchain server" cũng giúp phân biệt blockchain, sổ cái (ledger) và những dịch vụ giao thức.

Sau khi đã thiết lập được nền tảng, giờ ta có thể tạo các nút trên blockchain server:

1. `geth -datadir ./ account new`
2. `get -datadir ./ init customgenesis.json`
3. `geth -datadir ./ -maxpeers 95 -networkid 15 -port 303xx console`

Các câu lệnh trên sẽ khởi tạo geth client "NODE", kết quả sẽ hiện ra Node 0: Bootnode:: Node, Accounts, EVM cho hợp đồng thông minh. Đây là một nút đơn, Node 0, tại nút nguyên thủy cho máy chủ blockchain. Ta có thể thấy ID mạng của máy chủ blockchain là 15 - đây là một chuỗi cục bộ riêng.

Có nhiều mạng công khai như ID của mạng Ethereum chính bao gồm Metropolis là 1, Ropsten - một mạng thử nghiệm ứng dụng chéo của Ethereum, có ID là 3, ID của Rinleby là 4. Ngoài ra còn có Musicoin - một nền tảng âm nhạc dựa trên blockchain, có ID là 7.762.959. Ta nên lưu ý đến những ID của những mạng khác. ngoài ra, khi ở trong một môi trường riêng tư, ta nên nắm được các ID mạng đang sử dụng nếu có.

Với các lệnh trên, chúng ta đã dùng lệnh geth được cung cấp bởi máy chủ Ethereum đã được cài đặt, ta cũng sử dụng lệnh geth để tạo một nút bằng cách tạo một số tài khoản mới - một EOA. Sau đó, chúng ta khởi tạo node sử dụng một nút khởi nguyên thủy (genesis) tùy chỉnh có đặc tả, là khối đầu tiên của chuỗi. Cuối cùng, chúng ta chỉ định ID mạng và cổng truy cập để vào nút mà giao dịch với blockchain.

Khi chạy xong, ta sẽ nhìn thấy nút Ethereum, địa chỉ enode chỉ định cho nút nguyên thủy hoặc là nút khởi động (bootnode) được tạo. Sau đó địa chỉ enode được sử dụng bởi những nút khác để kết nối với nút khởi động để thiết lập một mạng ngang hàng mà blockchain hoạt động trên đó. Nhắc lại rằng, mạng ngang hàng là một trong những khái niệm nền tảng trong công nghệ blockchain. Ta đã hoàn thành bước tạo lập nút. Bây giờ ta cần tạo thêm ít nhất một nút ngang hàng nữa:

1. `geth -datadir ./ account new`
2. `geth -datadir ./ init customgenesis.json`
3. `geth -datadir ./ -networkid 15 -port 303xy console`
4. `admin.addPeer([ĐỊA CHỈ ENODE])`

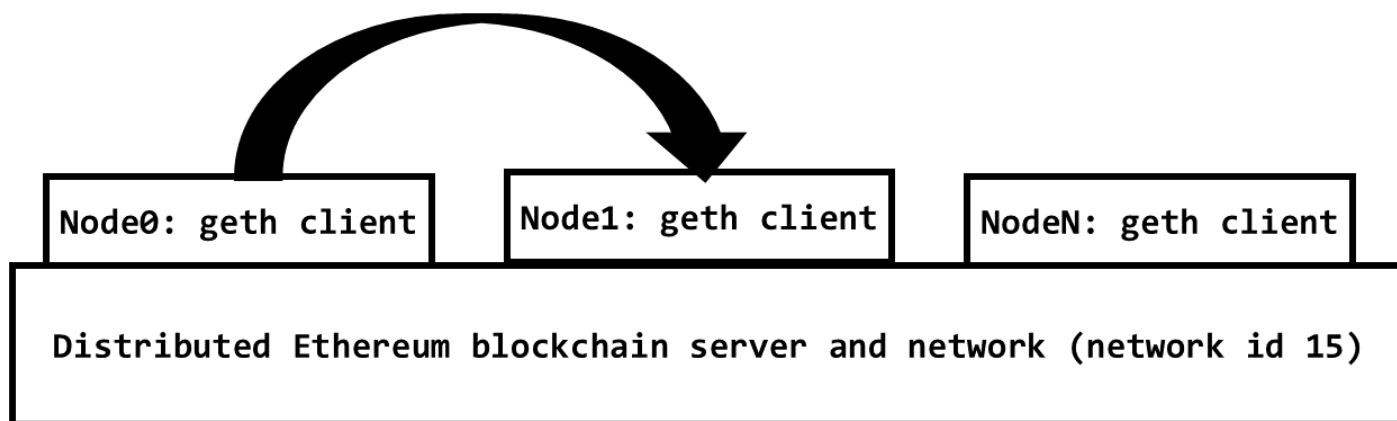
Ở đây chúng ta tạo ra Node 1 và thêm nó như là một nút ngang hàng với nút khởi động mà chúng ta tạo trước đó. Sau khi tạo nút này xong, ta thấy rằng ta đang sử dụng các admin API, lệnh `addpeer` và địa chỉ enode của nút khởi động chúng ta tạo trước đó. Địa chỉ IP và cổng liên kết của nút khởi động là 303xx, nó đại diện cho số cổng của Nút 0 mà chúng ta đã tạo. Đây chính là cách để tạo lập và kết nối nút ngang hàng để tham gia vào mạng lưới blockchain. Ta có thể lặp lại các bước này để tạo liên tục từ node 0 cho tới node N.

Có một vài phương pháp khác nhau để tạo mạng lưới blockchain, ta đã vừa trình bày một trong những cách đó.

4.1.2 Định nghĩa DApp

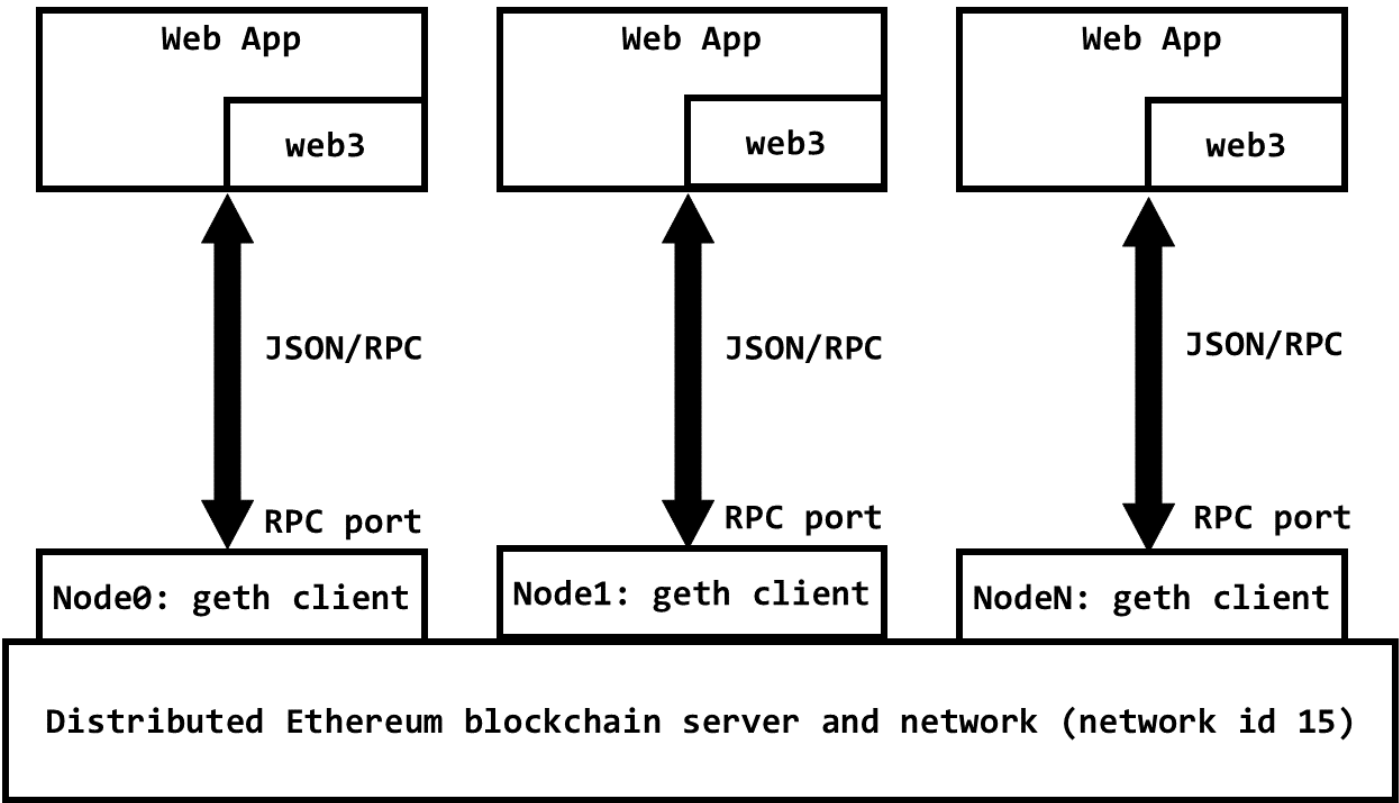
Nhắc lại, Một DApp hay còn gọi ứng dụng phi tập trung giải quyết những vấn đề mà dựa trên dịch vụ và hạ tầng của blockchain để thực hiện hóa nó. Thông thường, một DApp gồm có một ứng dụng web front-end, và một ứng dụng blockchain back-end và được viết mã để kết nối giữa chúng. Trong một kiến trúc như vậy, front-end của DApp sẽ truyền những lệnh kích hoạt bên ngoài từ người dùng tới hạ tầng blockchain và phản hồi lại chúng. Nó khởi tạo các giao dịch để triệu gọi những chức năng trên hợp đồng thông minh. Tiếp đó, ghi lại những giao dịch và chuyển tiếp trạng thái rồi lưu lại trên blockchain. Front-end của một DApp có thể đơn giản chỉ là một giao diện dòng lệnh, nó cũng có thể là một ứng dụng web tinh vi hoặc dễ dàng sử dụng dưới dạng ứng dụng di động. Phát triển front-end có thể liên quan đến phát triển một ứng dụng web với HTML JavaScript và CSS và các tài nguyên web khác hoặc một web framework ví dụ như Express. Ở đây, máy chủ blockchain là một E-node nằm trên cấu trúc cơ sở. Và front-end là một ứng dụng web sử dụng `web3.js`, giao tiếp sử dụng JSON thông qua RPC. Như báo cáo đã nói ở phần 3, script triển khai với `web3.js` là một tạo tác quan trọng được tạo ra bởi trình biên dịch Remix.

Giờ ta sẽ xem xét kiến trúc của một DApp.



Hình 11: Kiến trúc đơn giản của một ứng dụng phân tán

Đây là một kiến trúc DApp đơn giản với các nút mà chúng ta đã tạo ra trước đó với lệnh geth và máy chủ blockchain Ethereum cùng mạng network. Khi các nút đã được khởi tạo với cùng một mạng, ta có thể gửi đi những giao dịch bằng giao diện dòng lệnh. Ngoài ra, ta nên triển khai một hợp đồng thông minh từ dòng lệnh đó. Lưu ý rằng những lệnh này có thể dài và nhiều tham số, có thể kéo dài thành nhiều dòng. Hơn nữa, giao diện dòng lệnh có thể không quen thuộc đối với những người không phải lập trình viên hoặc một người dùng DApp. Vì vậy, ta sẽ dùng web một giao diện web đơn giản và trực quan như một ứng dụng front-end của DApp trên blockchain mà chúng ta định triển khai.



Hình 12: Kiến trúc hoàn thiện đầy đủ của ứng dụng phân tán

Đây là kiến trúc đã được cập nhật cho một DApp. Có rất nhiều nút đầy đủ nhưng ta chỉ hiển thị 3 nút trong số đó: Node1, Node2 và NodeN. Lệnh geth được sử dụng để hiển thị cổng RPC 8544 sử dụng lệnh `geth -rpc -rpcport 8544`. Từ ứng dụng web, các hàm được triển khai trong hợp đồng thông minh và gọi các hàm đã được làm rõ ràng thông qua mô-đun `web3.js`

4.1.3 Ethereum APIs

API, hoặc Application Programming Interface (giao diện lập trình ứng dụng) là một phương pháp và tiêu chuẩn để cung cấp một bộ các hàm/chức năng liên quan tới một tập dữ liệu và các dịch vụ cụ thể.

Các API cũng cho phép tái sử dụng mã nguồn. Một API cung cấp một tập các hàm hoặc phương thức mà có thể được sử dụng để dùng triệu gọi các hành động, truy cập và lưu trữ dữ liệu. Việc truy cập vào một API có thể bị kiểm soát bởi phương thức truy cập cụ thể, ví dụ như là các khóa công khai, nếu được đảm bảo bởi ứng dụng.

Chúng ta cần một hướng tiếp cận tiêu chuẩn được xác định chính xác để giải quyết mọi thứ cho một DApp. Câu trả lời nằm trong các API. Trong trường hợp của ta, các API cụ thể thể hiện các dịch vụ của máy chủ blockchain dùng các hàm tiêu chuẩn. Khi ta phát triển một DApp, các tương tác với nút geth trên máy chủ blockchain được hoàn thành bằng cách triệu gọi các API này. Ví dụ, trong lệnh `miner.start()`, `miner` là API, và `start()` là hàm của `miner` API.

Công nghệ blockchain đã mở ra cả một nền văn hóa hoàn toàn mới trong việc phát triển và quản lý phần mềm và các hệ thống. Lúc ban đầu, các hệ thống phần mềm là các sản phẩm độc quyền. Cuối cùng thì một số hệ thống đã trở thành mã nguồn mở. Với công nghệ blockchain, các nhà phát triển không chỉ lập trình các phần mềm, rất nhiều người trong số họ còn giúp quản lý mạng lưới bằng cách thêm vào các nodes, hỗ trợ khai thác, v.v. Quan trọng hơn là, các nhà phát triển cũng hứa hẹn về những cải tiến và thay đổi đối với giao thức blockchain. Đây là văn hóa công nghệ ẩn sau cuộc cách mạng blockchain. Để cho phép ta có thể được hoạt động như là các developer trong trường hợp này, ta cần kiến thức cơ bản về API và các hoạt động bên trong đó.

Các Ethereum API có hai danh mục chủ yếu. Một là Management API, bao gồm `admin`, `debug`, `miner`, `personal`, và `txpool`. Chúng hỗ trợ các phương thức để quản lý nút geth. Loại thứ hai là `web3` API gồm `web3`, `eth` và `net`. Chúng hỗ trợ các phương thức để phát triển các DApp.

Admin API cho phép ta dùng các hàm để làm việc với Geth, bao gồm mạng ngang hàng và quản lý điểm cuối RPC. Ví dụ: `admin.addPeer()`, `admin.nodeInfo()`. Trong trường hợp này, `admin` là API, và `addPeer` và `nodeInfo` là các hàm của API.

API này. Ta có thể thấy API admin hỗ trợ các hàm để quản lý nút. Debug API, ví dụ: `Debug.dumpBlock(16)`. Lệnh này sẽ hiện thị các chi tiết tiêu đề (header) của block 16. Ta có thể thấy rằng Debug API giúp ta nhìn vào blockchain, tìm hiểu nó và sửa chữa bất cứ vấn đề gì khi nhìn thấy trong block.

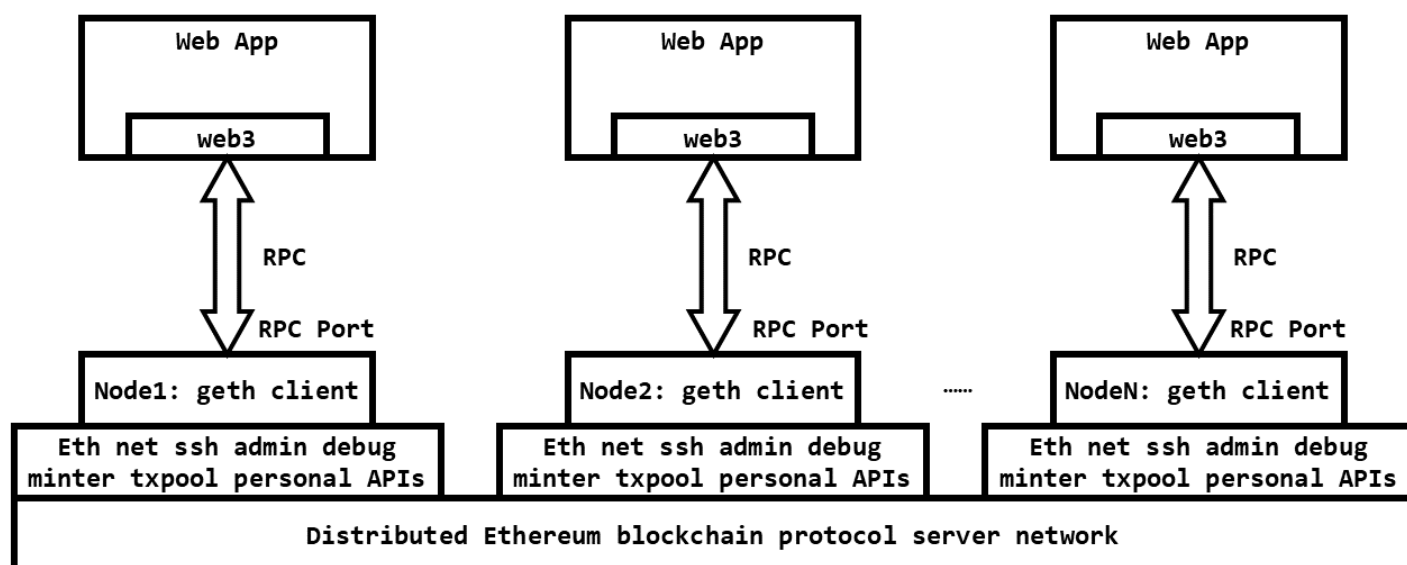
Miner API cho phép ta kiểm soát hoạt động khai thác các nút và thiết lập nhiều kiểu khai thác cụ thể khác nhau. Ví dụ: `miner.start()`, `miner.stop()` là các hàm mẫu sẽ khởi động và kết thúc việc khai thác của các thợ đào. Ta có thể gọi `miner.start(6)`, tức là có 6 luồng song song được gán cho hoạt động khai thác.

Personal API phụ trách việc tạo ra và quản lý các tài khoản trong một nút. Nó cũng quản lý các khóa bí mật trong kho khóa (key store), đây là lý do nó được gọi là Personal API. Ví dụ: `personal.newAccount()` sẽ tạo ra một tài khoản mới trong nút.

Txpool API, Txpool hoặc Transaction Pool API cho phép ta tiếp cận các phương thức RPC không tiêu chuẩn để kiểm tra nội dung của pool giao dịch, bao gồm tất cả các giao dịch hiện tại đang chờ xử lý, cũng như là các giao dịch xếp hàng đợi xử lý về sau. Ví dụ: `txpool.inspect()` liệt kê cho ta tất cả các giao dịch đang chờ xử lý để ta nghiên cứu và tập hợp khi xây dựng một khối các giao dịch.

Bây giờ ta xem xét các API Web3. Thư viện Web3.js, khi được đưa vào DApp, sẽ cho phép các ta dùng tất cả thư viện `web3.js`. Nó cũng giúp ta tiếp cận với một nút nội bộ thông qua cổng RPC. Nó cũng cung cấp truy cập tới đối tượng `eth` và các hàm của nó bằng cách dùng lệnh `web3.eth`, và đối tượng `net` thông qua `web3.net`, và các hàm tương ứng của chúng. Ta cũng có thể truy cập các Management API khác tới các đối tượng `web3`. Còn có Whisper API, `web3.ssh` Nó được dùng để bảo mật hội thoại và cho phép giao thức Whisper.

Web3 là một thư viện JavaScript được thiết kế chuyên biệt cho việc sử dụng ứng dụng web hoặc các ứng dụng Ethereum DApp. Nó là một cổng giúp triệu gọi tất cả các hoạt động của nút Ethereum trên máy chủ blockchain có thể được triệu gọi. Ví dụ như, việc triển khai một smart contract và triệu gọi hàm smart contract. Ta sẽ dùng `web3` trong phát triển front-end cho DApp và tương tác với blockchain.



Hình 13: Kiến trúc DApp với API đi kèm

Trên đây là một kiến trúc đơn giản của một DApp. Nó bao gồm nhiều nút đầy đủ với chỉ 3 trong số đó là Node 1, 2 và N. Ta cũng có thể thấy tất cả các APIs mà chúng ta đã thảo luận. Geth được dùng để thể hiện cổng RPC, cũng như là các API. Một đối tượng `web3` được khởi tạo trong một script trên trang web. Hãy nhớ lại rằng `web3.js` là một thư viện JavaScript. Các request hay các triệu gọi trên đối tượng `web3`. Request được truyền đi dưới dạng một JSON hoặc RPC giữa web client và geth client. Những request và hàm được thực thi khi dùng API và mã smart contract phù hợp, kết quả được trả lại client.

4.2 Truffle

4.2.1 Truffle IDE

Truffle là một trong những IDE hàng đầu cung cấp tất cả mọi thứ từ template khởi tạo ứng dụng đến một blockchain cục bộ để kiểm thử DApp hoàn chỉnh. Cũng tương tự như cách chúng ta dùng Remix IDE để kiểm thử hợp đồng thông minh (smart contract), chúng ta sẽ dùng Truffle để lắp ghép các thành phần khác nhau của một DApp. Thiết kế một ứng dụng phân tán xét tổng quan sẽ có những bước sau:

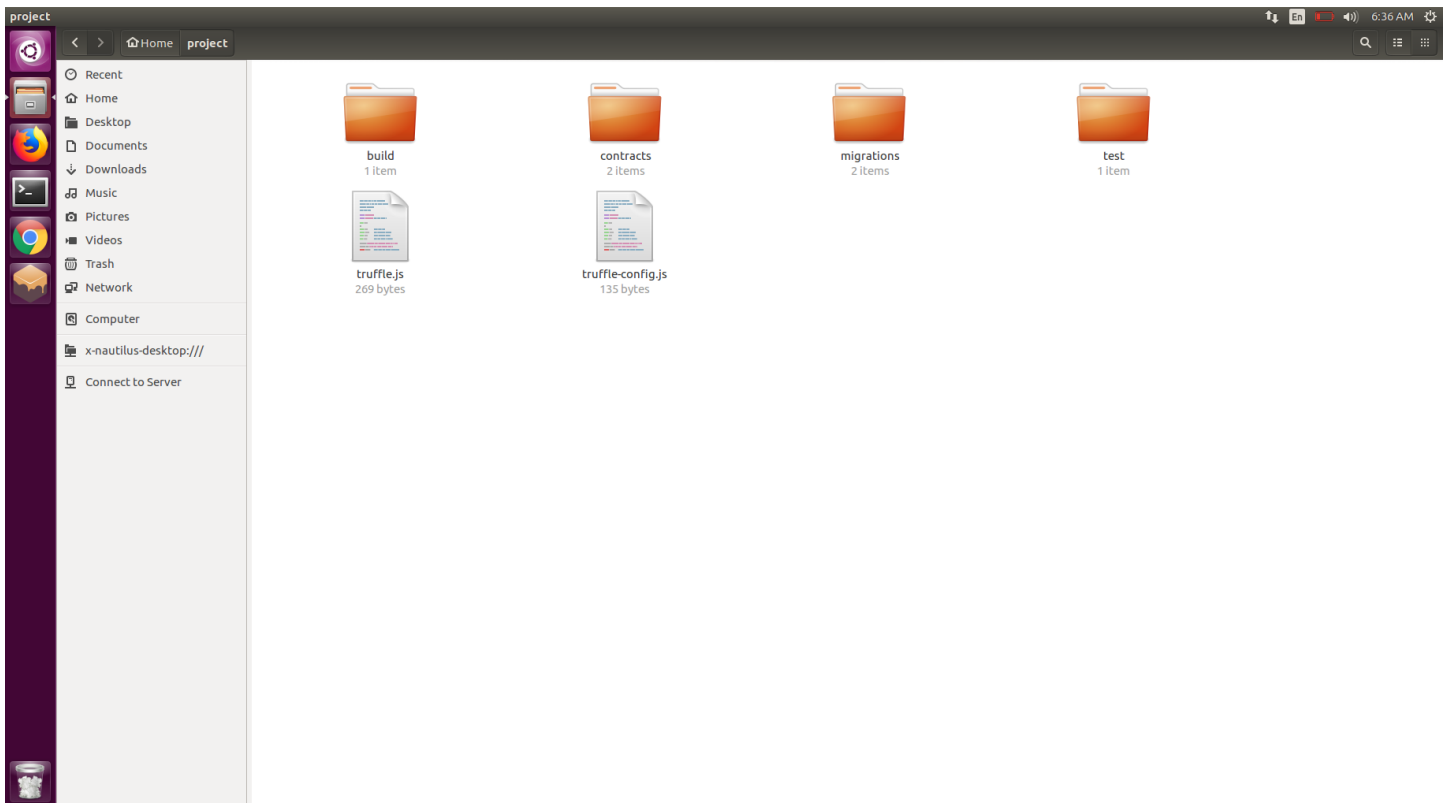
1. Thiết kế những hợp đồng thông minh cần thiết
2. Áp dụng modifier để cài đặt những điều kiện cần có.

3. Thêm vào mã nguồn kiểm thử (tester code) cho bài toán được yêu cầu và chạy kiểm thử để đảm bảo mọi trường hợp đều đạt.
4. Thêm giao diện người dùng.
5. Kiểm tra ứng dụng hoàn thiện bằng cách tương tác với giao diện vừa phát triển.

Những lệnh cơ bản nhất của Truffle là:

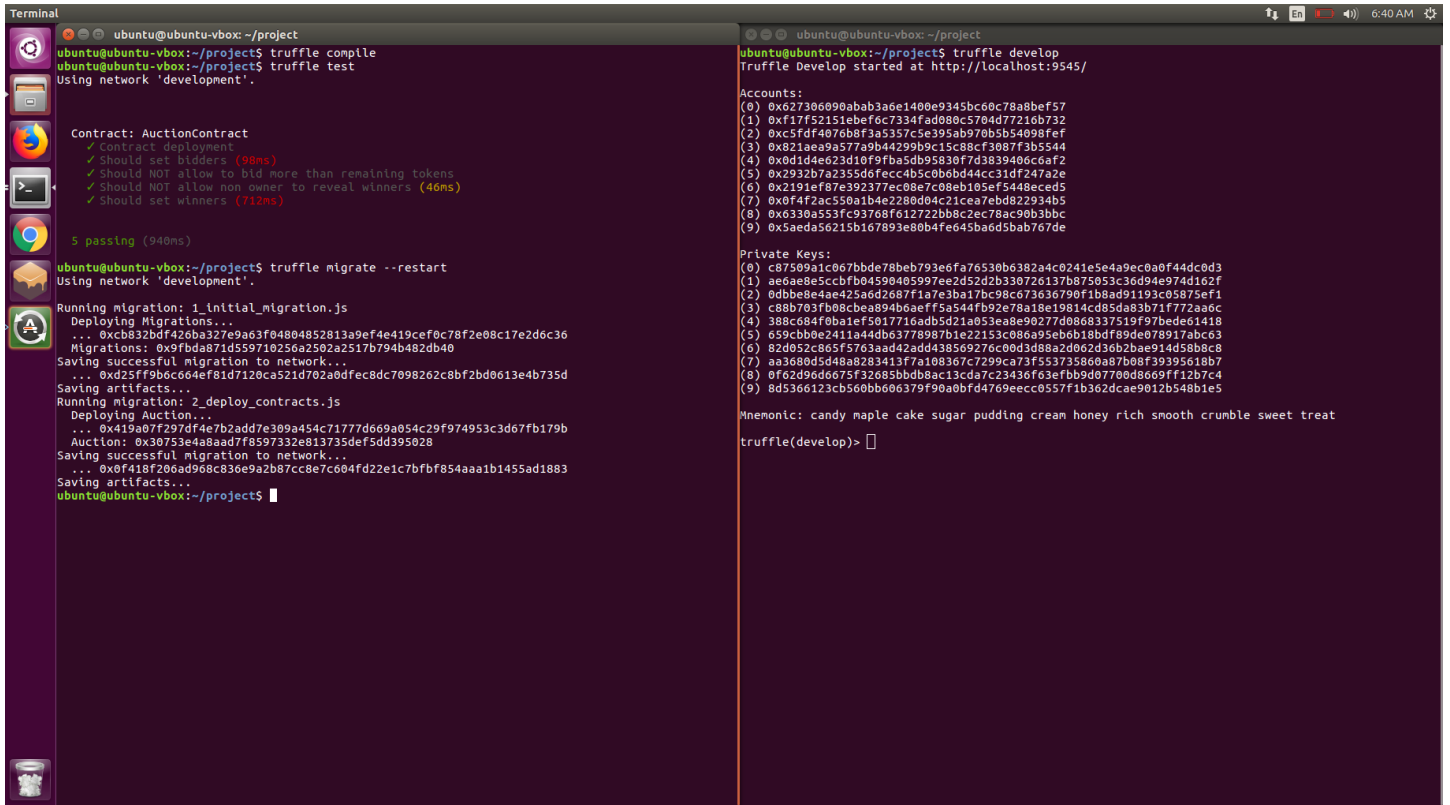
- **truffle init:** Tạo ra một template dự án mới với cấu trúc folder cơ bản cho ứng dụng phân tán.
- **truffle compile:** Biên dịch những hợp đồng thông minh
- **truffle develop:** Xây dựng blockchain cá nhân để kiểm thử trên console
- **truffle migrate:** Thực thi script triển khai hợp đồng thông minh
- **truffle test:** Môi trường kiểm thử cho những hợp đồng đã được triển khai

Đây chỉ là một vài hành động có thể thực hiện trong Truffle. Về sau ta có thể tìm hiểu sâu hơn những tính năng cao cấp mà Truffle cung cấp.



Hình 14: Cấu trúc thư mục dự án được tạo ra với lệnh truffle init

Ba thư mục con trong thư mục dự án là "contracts", "migrations" và "test". "contracts" bao gồm các tệp tin Solidity cho các hợp đồng thông minh. Có một hợp đồng quan trọng có tên là Migrations.sol, đó là một hợp đồng thông minh hỗ trợ cho việc triển khai. Tại thư mục "migrations", Truffle sử dụng một hệ thống migration để xử lý việc triển khai hợp đồng. Một migration là một hợp đồng bổ sung nhỏ để theo dõi các thay đổi. Thư mục "test" bao gồm cả code JavaScript lẫn code Solidity để kiểm thử cho hợp đồng thông minh.



Hình 15: Chạy các lệnh để triển khai ứng dụng

Lệnh **truffle compile** sẽ tạo ra một folder mới gọi là "build" chứa các artifacts được tạo mới. Nếu có lỗi thì lệnh này sẽ hiển thị trên terminal. Ở đây các hợp đồng thông minh không có lỗi nào, nên đầu ra là trống. Tiếp đến ta chạy lệnh **truffle develop** để triển khai một blockchain cục bộ với 10 địa chỉ tài khoản được hiện ra. Kèm theo đó là cụm 12 từ mnemonic được gọi là seed words để liên kết những tài khoản này với ví điện tử, ta cần copy hoặc ghi nhớ các cụm từ này. Tiếp đó là chạy lệnh **truffle test** để chạy những trường hợp kiểm thử hợp đồng thông minh. Ở đây ta đã chạy đúng đủ 5 trường hợp kiểm thử. Cuối cùng là chạy lệnh **truffle migrate --restart** để triển khai những hợp đồng thông minh mới, --restart là để xóa những hợp đồng cũ ở các lần chạy trước.

4.2.2 Phát triển theo kiểm thử

Việc kiểm thử là hoàn toàn bắt buộc cho các hợp đồng thông minh. Kiểm thử với smart contract có thể chạy cùng với các positive test để đảm bảo rằng với một đầu vào hợp lệ cho trước, nó sẽ hoạt động như kỳ vọng. Sau đó là negative test, đảm bảo là nó xử lý các đầu vào và các tình huống không hợp lệ một cách phù hợp. Thông thường thì, các kiểm thử được viết trong ngôn ngữ giống với ứng dụng chính được kiểm thử.

Truffle cho phép sử dụng cả hai ngôn ngữ, JavaScript và Solidity, chúng ta sẽ dùng JavaScript.

```
1 let Auction = artifacts.require("./Auction.sol");
2
3 let auctionInstance;
4
5 contract('AuctionContract', function (accounts) {
6
7   it("Contract deployment", function() {
8     return Auction.deployed().then(function (instance) {
9       auctionInstance = instance;
10      assert(auctionInstance !== undefined, 'Auction contract should be defined');
11    });
12  });
13
14  it("Should set bidders", function() {
15    return auctionInstance.register({from: accounts[1]}).then(function(result) {
16      return auctionInstance.getPersonDetails(0);
17    }).then(function(result) {
18      assert.equal(result[2], accounts[1], 'bidder address set');
19    });
20  });
21
22  it("Should NOT allow to bid more than remaining tokens", function() {
23    return auctionInstance.bid(0, 6, {from: accounts[1]})
24      .then(function (result) {
25        throw("Failed to check remaining tokens less than count");
```

```

26     }).catch(function (e) {
27         var a = e.toString();
28         if(e === "Failed to check remaining tokens less than count") {
29             assert(false);
30         } else {
31             assert(true);
32         }
33     })
34 });
35
36 it("Should NOT allow non owner to reveal winners", function() {
37     return auctionInstance.revealWinners({from: accounts[1]})
38     .then(function (instance) {
39         throw("Failed to check owner in reveal winners");
40     }).catch(function (e) {
41         if(e === "Failed to check owner in reveal winners") {
42             assert(false);
43         } else {
44             assert(true);
45         }
46     })
47 })
48
49
50 it("Should set winners", function() {
51     return auctionInstance.register({from: accounts[2]})
52     .then(function(result) {
53         return auctionInstance.register({from: accounts[3]})
54     }).then(function() {
55         return auctionInstance.register({from: accounts[4]})
56     }).then(function() {
57         return auctionInstance.bid(0, 5, {from: accounts[2]})
58     }).then(function() {
59         return auctionInstance.bid(1, 5, {from: accounts[3]})
60     }).then(function() {
61         return auctionInstance.bid(2, 5, {from: accounts[4]})
62     }).then(function() {
63         return auctionInstance.revealWinners({from: accounts[0]})
64     }).then(function() {
65         return auctionInstance.winners(0, {from: accounts[0]})
66     }).then(function(result) {
67         assert.notEqual(result, '0x0000000000000000000000000000000000000000', 'not equal')
68         return auctionInstance.winners(1, {from: accounts[0]})
69     }).then(function(result) {
70         assert.notEqual(result, '0x0000000000000000000000000000000000000000', 'not equal')
71         return auctionInstance.winners(2, {from: accounts[3]})
72     }).then(function(result) {
73         assert.notEqual(result, '0x0000000000000000000000000000000000000000', 'not equal')
74     })
75 });
76 });

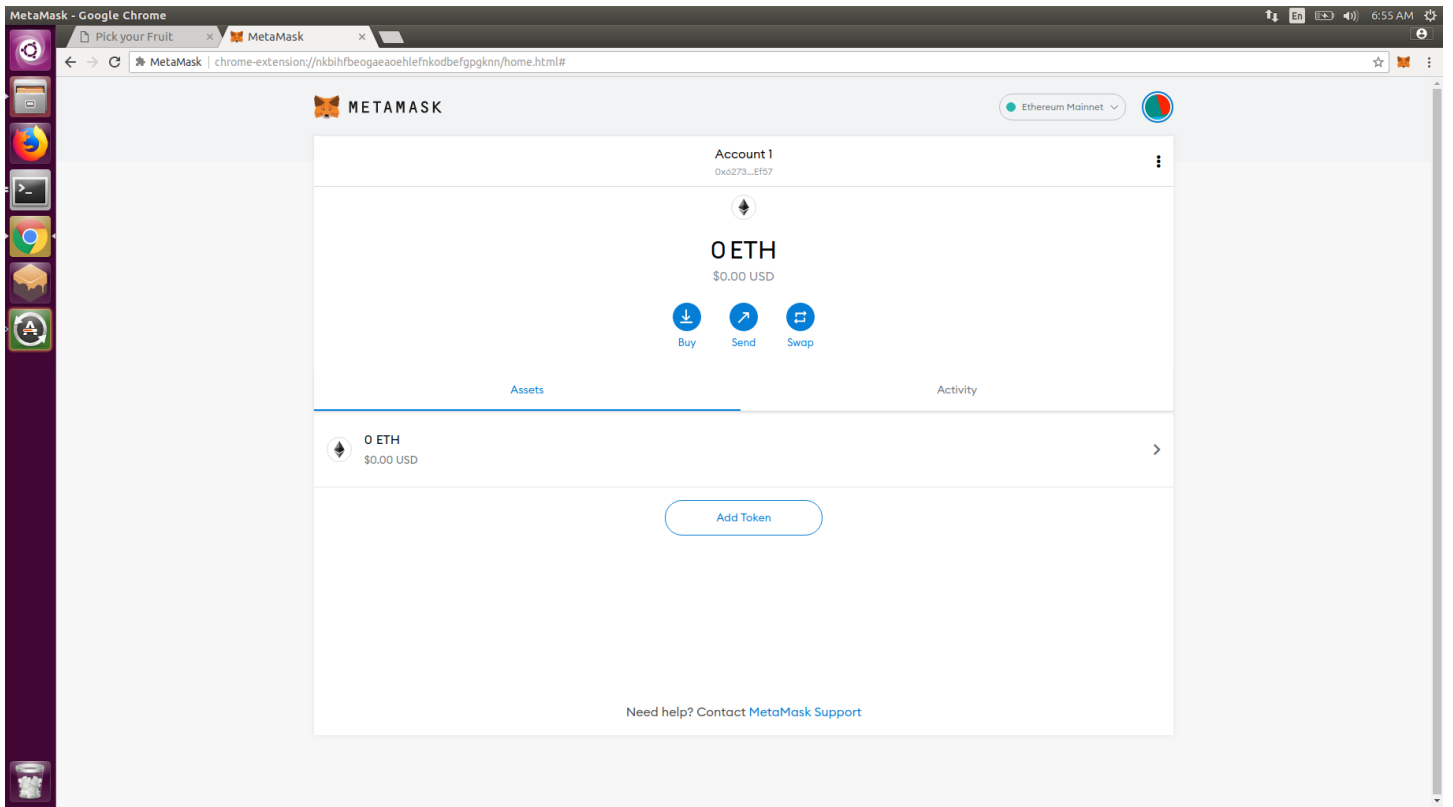
```

Listing 6: Mã kiểm thử cho hợp đồng thông minh Auction.sol

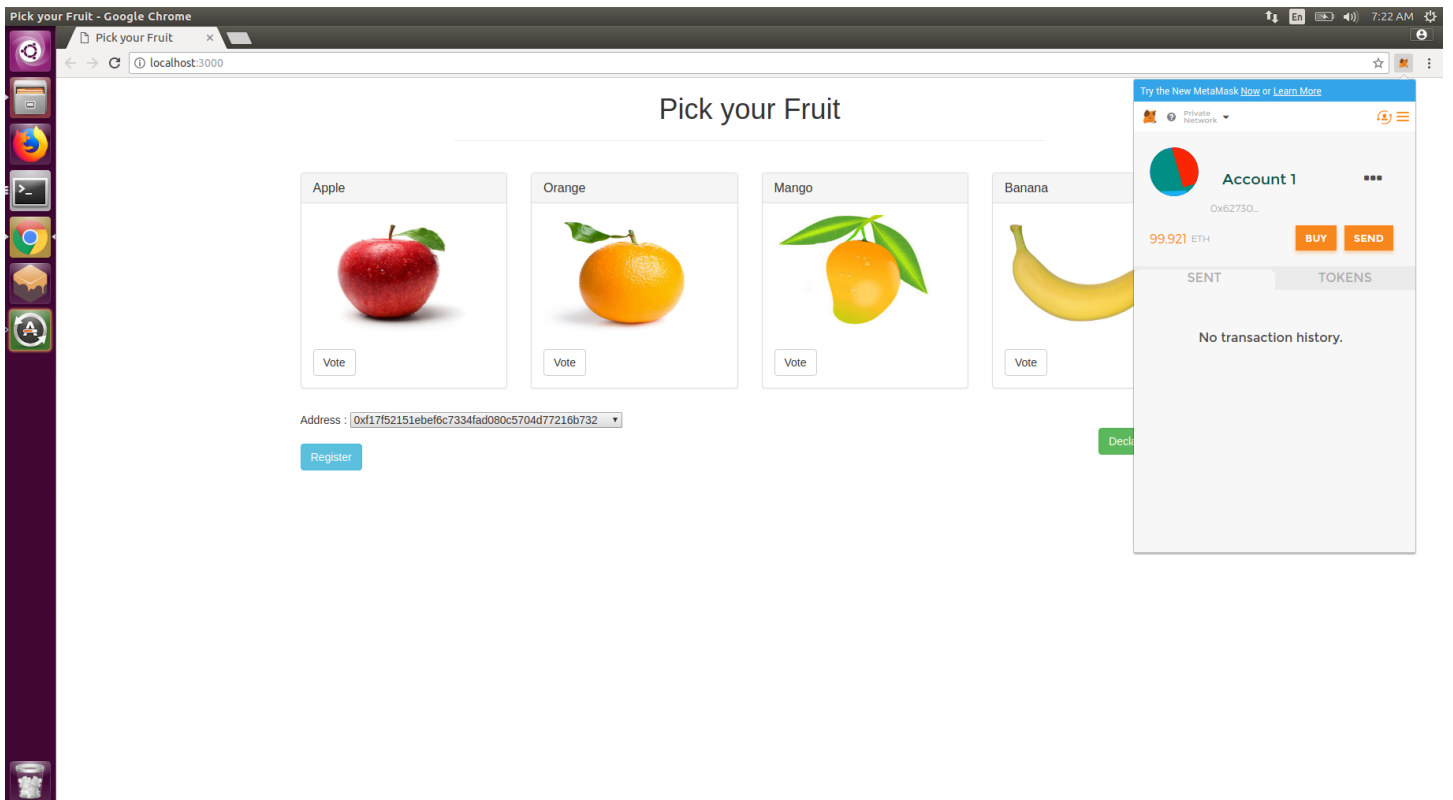
Tệp test.js trên có các positive test và negative test để kiểm thử

4.2.3 Giao diện web và kiểm thử

Ở phần này, ta sẽ thêm giao diện web cho Dapp. Trước đó ta cần cài đặt extension Metamask cho trình duyệt web. MetaMask là cầu nối giữa máy chủ blockchain và web của chúng ta. Nó quản lý các mạng blockchain, tài khoản cùng ví điện tử và thực thi các giao dịch trên giao diện web.



Hình 16: Tài khoản có được từ truffle develop có thể sử dụng trên metamask bằng cách tạo ví mới bằng 12 cụm từ đã có từ lệnh này



Hình 17: Giao diện ứng dụng web cho việc đấu thầu phân tán, các nút bấm trên web sẽ thực thi giao dịch được xác nhận bằng bên Metamask trung gian

4.3 Cải thiện thiết kế

Phần này sẽ nói về cách để cải thiện D-App tốt hơn, hiệu quả hơn và thiết thực hơn đối với blockchain. Có những cải tiến giúp cho thiết kế của ứng dụng phi tập trung thực tế xung quanh hợp đồng thông minh.

Thứ nhất, chúng ta cần đảm bảo rằng chúng ta không lưu những dữ liệu không cần thiết trên blockchain. Nhớ rằng blockchain không phải là một kho dữ liệu. Thứ hai, cần lưu tâm đến event và logging (ghi lại thông tin và sự kiện). Vấn đề thứ ba là hợp đồng thông minh không thể truy cập những liên kết ngoài, ta cần cách để truy cập những external data này.

4.3.1 Các tính năng của Solidity

Trong mạng máy tính, những cải tiến về hiệu năng và bảo mật luôn được đề ra. Những đề xuất này được nêu trong các RFC. Tương tự, Khung đề xuất cải tiến Ethereum, EIP (Ethereum Improvement Proposals), quản lý các cải tiến liên tục cho giao thức ethereum là một ví dụ điển hình về cải tiến quy trình lặp lại như RFC.

Vấn đề đầu tiên cần được nhắc đến là Để giảm thiểu footprint trên blockchain. chúng ta cần xem xét khả năng lưu trữ với khả năng bộ nhớ của dữ liệu. Mặc định, tất cả các thay đổi trạng thái đã xác định biến trong hợp đồng thông minh, đều được lưu lại như một trạng thái trên blockchain. Chúng ta chỉ cần một mảnh dữ liệu cần thiết như tình trạng xuất xứ, quản trị, và không thay đổi được được lưu trữ trên blockchain. Vậy, mục đích của chúng ta là giảm thiểu những trạng thái hoặc footprint trên hợp đồng thông minh và giảm chi phí. Một trong những cách mà ta có thể thực hiện điều này, là sử dụng chính xác các từ khóa **bộ nhớ - memory** và **lưu trữ - storage**. Bộ nhớ và lưu trữ thực sự là những từ khóa trong ngôn ngữ solidity và chúng có ý nghĩa giống như trong hệ tính toán thông thường của ta. Bộ nhớ là bộ nhớ tạm thời trong RAM còn lưu trữ liên quan đến bộ dữ trữ liên tục lưu trong thiết bị lưu trữ cố định như là ổ cứng. Dữ liệu trong bộ nhớ là tạm thời và là một cuộc đua giữa những lần gọi hàm. Bộ nhớ chỉ là dãy các bytes. Trong khi đó, storage lưu trữ ánh xạ nơi mà nó tốn 32 bytes cho mỗi cặp key - value. Tất cả dữ liệu lưu trữ đều được coi là một state và được sử dụng trong tính toán các hàm băm của header. Sử dụng bộ nhớ sẽ chỉ tiêu tốn một vài gas points, 1-3. trong khi chi phí của storage lên đến hàng nghìn điểm. 20 nghìn điểm để thiết lập, 5000 điểm để thay đổi giá trị.

Một hợp đồng thông minh được coi là toàn cầu (global) và một bản sao của nó được hiển thị trong mọi nút đầy đủ (full node). Các hàm băm, footprint và trạng thái được hiển thị trên mỗi khối. Chúng ta không muốn những chi tiết không cần thiết được lưu lại trên sổ cái vĩnh viễn. Có nhiều cách để quản lý bộ nhớ so với lưu trữ. Ở đây chúng ta chỉ xem xét cấu trúc và mảng dữ liệu tổng hợp. Trong một hợp đồng thông minh của ngôn ngữ Solidity, cấu trúc và mảng được mặc định chỉ định dùng lưu trữ thay vì bộ nhớ, ngay cả khi chúng là nội bộ đối với các hàm. Trong khi một cấu trúc hoặc mảng được sử dụng như một tham số hay một biến nội bộ trong một hàm, ta cần khai báo chúng dưới dạng biến bộ nhớ. Nếu thuộc tính bộ nhớ không có, biến tạm sẽ đóng vai trò là một biến lưu trữ gây lãng phí không gian biến trạng thái. Bởi vì chúng ta đã khai báo biến là loại bộ nhớ, không gian của nó chiếm 30x20 bit trên bộ nhớ tạm và một số lưu trữ vĩnh viễn trong hợp đồng thông minh. Chúng ta đều biết rằng một hợp đồng thông minh khi đã triển khai là cố định và vĩnh viễn. Chúng ta đã lưu 600 bit dung lượng lưu trữ trên EVM.

```
1 function findTheRichest() public return {address richest} {
2     address[30] memory investors = club.getInvestor();
3     //....
4 }
```

Listing 7: Cần sử dụng từ khóa memory cho các biến cần thiết để tiết kiệm bộ nhớ

Ta cần xem xét trường hợp nếu một hợp đồng thông minh đã triển khai bị vô hiệu. Nhớ rằng một hợp đồng thông minh được xác định bởi một địa chỉ. Đây là một khái niệm quan trọng. Nó có thể đại diện một token, một tổ chức, một người, con người hoặc bất kỳ một cái gì mà liên quan đến các hoạt động và quy tắc logic Một hợp đồng thông minh có một chủ sở hữu. Theo mặc định, nó chính là người tạo ra hợp đồng thông minh đó. Nó có một giá trị hoặc số dư ether. Quyền sở hữu của nó có thể thay đổi hoặc nó có thể bị xóa đi.

Để xóa hoặc hủy hợp đồng thông minh, Solidity cung cấp một hàm gọi là "tự hủy". Hiểu rằng hàm gọi này không thể đảo ngược theo giao thức blockchain mặc định. Khi đã bị hủy, hợp đồng thông minh không thể khởi động và truy cập lại được. Điều này giống như giữ một con tàu vũ trụ đã hết hạn đi vào hố đen không gian. Một cách lập trình tốt khác là hãy đảm bảo rằng chỉ 1 chủ sở hữu hoặc nhiều sở hữu của hợp đồng thông minh hoặc một người được ủy quyền có quyền tự hủy hợp đồng. Giả sử, chỉ định một khoảng thời gian cụ thể cho hợp đồng. Trình sửa đổi được sử dụng để thiết lập địa chỉ hủy và khoảng thời gian cho hợp đồng thông minh.

```
1 function kill() only (owner) onlyAfter(creationTime + 1 years) {
2     // viet code chuyen tien hoac chi ra dia chi dich
3     selfdestruct (toAddress); // gui so du cho dia chi toAddress
4 }
```

Listing 8: Mã nguồn tự hủy hợp đồng thông minh

Code của hợp đồng thông minh bị xóa khỏi trạng thái của blockchain, nếu thỏa mãn các điều kiện được chỉ định bởi trình sửa đổi. Trong trường hợp của chúng ta, các điều kiện là, hàm được gọi bởi địa chỉ được chỉ định và thời gian đã chỉ định đã kết thúc. Các ether còn lại sẽ được lưu trữ trong địa chỉ của hợp đồng thông minh này sẽ được chuyển đến địa chỉ chỉ định trong tham số của hàm gọi tự hủy.

Trong suốt thời gian tồn tại của một tài sản, quyền sở hữu có thể thay đổi. Nó có thể thay đổi do việc trao đổi tài sản, tặng một tài sản hoặc bằng cách kế thừa. Đây chỉ là một vài loại. Có nhiều trường hợp khác để chuyển quyền sở hữu.

Listing 9: Mã nguồn tự hủy hợp đồng thông minh

Hàm trên có thể thực hiện chuyển quyền sở hữu, nhưng phải đảm bảo rằng ta đủ điều kiện chuyển thông qua trình sửa đổi hoặc các quy tắc sửa đổi phù hợp được thực thi từ việc quản lý quyền sở hữu và thời gian tồn tại của hợp đồng.

Chúng ta sẽ chuyển sang phần quản lý các mối quan hệ giữa các hợp đồng thông minh. Ta đã tạo ra một hợp đồng thông minh đơn lẻ. Một ứng dụng phức tạp có thể được tạo ra bởi nhiều hợp đồng thông minh, một cái tạo ra cái khác hoặc sở hữu những cái khác. Một biểu tượng hay vùng tên cho một tổ chức cũng có thể được xác định trong một hợp đồng thông minh thông thường nếu cần thiết. Lệnh `import` có thể được sử dụng để bao hàm các tệp trong hợp đồng thông minh của ta. Tại thời điểm biên dịch, các liên kết được tạo ra giữa các tệp quan trọng và hợp đồng thông minh hiện tại và mã byte được tạo dựa trên liên kết này. Điều này dẫn chúng ta đến cách thực hành lập trình thứ ba. Tính tương thích, chuyên môn hóa, và khái quát hóa là các nguyên lý quan trọng từ lập trình hướng đối tượng, mà có thể giúp thiết kế các giải pháp hợp đồng thông minh cho các vấn đề phức tạp. Điều này cũng thúc đẩy khả năng sử dụng lại mã và tiêu chuẩn giữa tổ chức và các bên tham gia.

```
1 import "FederalLaws.sol";
2 import "StateLaws.sol";
3 contract FieldLaws is FederalLaws, StateLaws {
4
5 }
```

Listing 10: Ví dụ kế thừa liên kết các tệp hợp đồng thông minh sử dụng `import`

Một chức năng khác của mối quan hệ giữa hợp đồng thông minh là có thể tạo thư viện và nhập các thư viện hiện có vào hợp đồng thông minh Solidity. Thư viện là hợp đồng thông minh đặc biệt không cần đến ether, không có hàm thanh toán và không có trạng thái được lưu trữ trong chuỗi khối. Một ví dụ điển hình là `SafeMath.sol` của Zeppelin, nó có thể được đưa vào hợp đồng thông minh của ta để dùng bất kỳ phép tính toán nào.

4.3.2 Điều khiển sự kiện

Các sự kiện hữu ích trong việc đẩy thông báo để biết sự hoàn thành của một hoạt động smart contract, nói là việc thực thi của một giao dịch. Các sự kiện có thể được dùng để thông báo bất cứ sự xảy ra sự kiện nào với một smart contract và blockchain. Các sự kiện hữu ích cho việc ghi nhật ký các hoạt động của một smart contract. Trong vai trò này, nhật ký sự kiện giống như là `printf` của chúng ta hoặc nhật ký bảng điều khiển. Nhật ký có thể được xem lại để kiểm tra chức năng của một smart contract. Sự kiện cho phép các hoạt động không đồng bộ giữa người gửi và người nhận sự kiện. Khung sự kiện là một thay thế hiệu quả cho việc kiểm tra xem một yêu cầu đã được hoàn thành hay chưa.

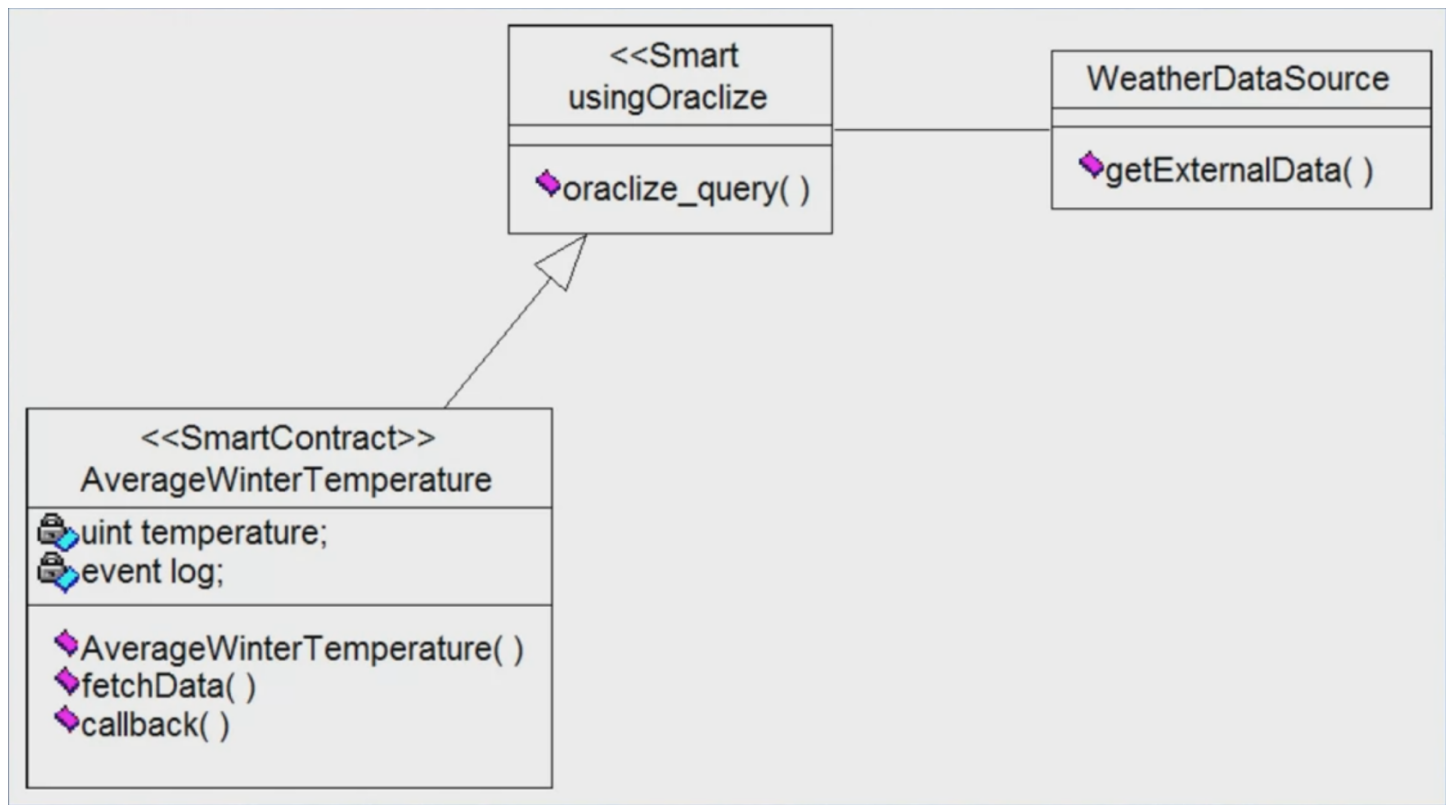
Ta xác định một sự kiện bằng tên và các tham số của nó. Một sự kiện có thể được tham số hóa như là một hàm. Ví dụ như, trong `Coin.sol`, ta sẽ thấy sự kiện được gửi kèm các tham số từ, tới và số lượng. Ta triệu gọi một sự kiện bằng tên và các giá trị tham số thực của nó. Điều này sẽ đẩy hoặc ghi nhật ký diễn ra sự kiện cho những người theo dõi nó. Ví dụ, trong `coin.sol` ta sẽ thấy nó được gửi với các tham số người gửi, người nhận và số lượng.

4.3.3 Oraclize

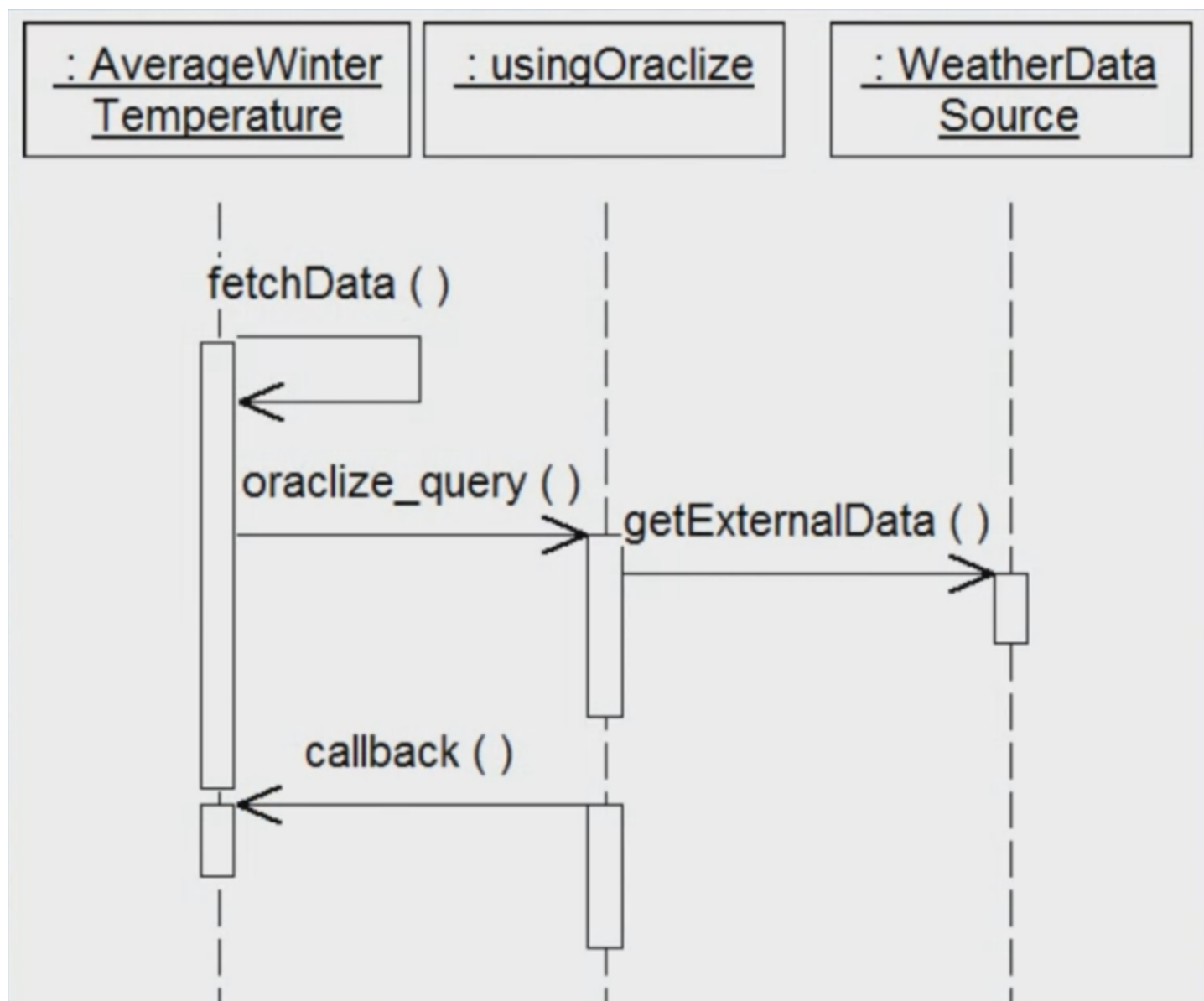
Nhắc lại rằng smart contract không thể gọi bên ngoài hàm hoặc gắn với các tài nguyên bên ngoài. Vì vậy ta phải dùng một hợp đồng thông minh đặc biệt là `Oracilize` để truy cập được nguồn tài nguyên đó.

Khái niệm `oraclize` chỉ ra vấn đề tìm nạp dữ liệu ngoài từ hợp đồng thông minh. `Oraclize` được mô tả như một người cung cấp dữ liệu giữa các tài nguyên web, APIs và URLs, và smart contract. Đương nhiên rằng `oraclize` nằm ngoài giao thức blockchain. Tuy nhiên, nó là một thành phần hữu dụng của Dapps mà hỗ trợ việc thu thập các thông tin thực tế cần thiết cho việc triển khai các smart contracts cụ thể.

`UsingOraclize` là một smart contract cung cấp tối thiểu một chức năng query để truy cập các nguồn bên ngoài. Nó không chỉ thu thập dữ liệu mà còn cung cấp các minh chứng và tính xác thực về các nguồn nếu cần bằng cách gọi smart contract. Dữ liệu đã yêu cầu được trả lại thông qua chức năng callback vì việc truy cập dữ liệu và xác thực có thể mất chút thời gian.



Hình 18: Sơ đồ lớp với hợp đồng thông minh, Oraclize cùng dữ liệu bên ngoài

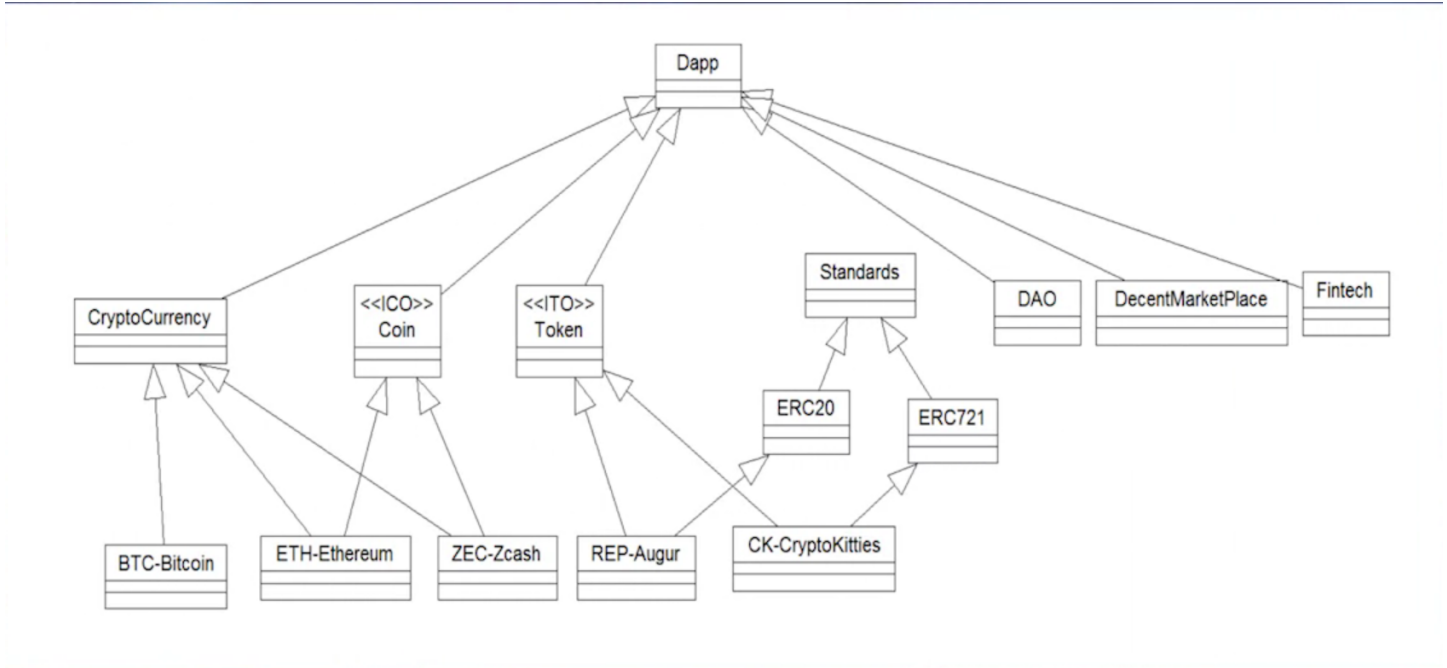


Hình 19: Biểu đồ tuần tự thể hiện dòng thời gian thực thi nhiều hàm khác nhau với Oraclize

Sử dụng hợp đồng thông minh `usingOraclize` rất đơn giản. Ta import hợp đồng đó và để hợp đồng ta đang viết kế thừa `usingOraclize`. Nó cung cấp nhiều hàm ví dụ như `oraclize_query()`, và cho ta chỉ định nguồn dữ liệu cần lấy trong truy vấn. Ngoài ra ta có thể cần có những phương pháp để xác thực nguồn thông tin vừa lấy là chính xác.

4.4 Mô hình và Tiêu chuẩn cho ứng dụng

4.4.1 Mô hình của DApp



Hình 20: Các mô hình của ứng dụng phân tán

Initial Coin Offering (ICO) - phát hành coin lần đầu (tương tự như một đợt phát hành cổ phiếu lần đầu IPO của các công ty) sử dụng một blockchain để ghi lại sự phân phối đồng tiền mã hóa, nhận quỹ, chỉ ra luật lệ và thực thi bất kỳ điều kiện tiên quyết và chính sách nào.

Token giống như Coin của ứng dụng phân tán, nhưng gắn liền với một tài sản hoặc công cụ.

Decentralized Autonomous Organization (DAO) - một "tổ chức tự động phân tán" là một cuộc cách mạng sớm trong Dapp. Nó là một công cụ đầu tư được triển khai như một hợp đồng thông minh trên chuỗi khối Ethereum với mục tiêu chính là giới thiệu một tổ chức không có cấu trúc quản trị công ty truyền thống để phi tập trung, ẩn danh gây quỹ và đầu tư tự động.

Mô hình DApp tiếp theo là **thị trường phi tập trung (decentralized marketplace)**. Đây là một mô hình rất đơn giản của ứng dụng phân tán. Chúng ta xác định thị trường rất rộng. Nó có thể là DApp của bất cứ cái gì từ buôn bán hàng hóa đến chứng chỉ giáo dục. Ta biết rằng một thị trường cho phép các cuộc gặp của người bán và người mua. Ta có thể xây dựng một nền tảng thị trường nơi mà những người mua và bán phi tập trung, những người không quen biết nhau, có thể gặp và trao đổi hàng hóa với việc chi trả và giao dịch được ghi nhận lại trên bất cứ công nghệ blockchain đã công bố nào như là Bitcoin hoặc Ethereum. Ta luôn có thể dùng Bitcoin như một hệ thống thanh toán cho thị trường toàn cầu mà ta xây dựng.

Một ví dụ là smart contract mua bán an toàn từ xa trong tệp Purchase.sol viết bằng ngôn ngữ Solidity. Thêm một DApp để thảo luận là FinTech. FinTech, viết tắt của công nghệ tài chính, có tiềm năng rất lớn để phát triển DApp. Lĩnh vực này nằm trong phạm vi từ công cụ đầu tư phi tập trung tới các kênh micropayment.

4.4.2 Tiêu chuẩn cho DApp

Các tiêu chuẩn thiết lập trật tự, sự an toàn, và rất nhiều các thuộc tính định tính khác nữa cho lĩnh vực này. Các tiêu chuẩn có thể đưa ra các quy định và sự rõ ràng. Chúng đặc biệt bắt buộc cho một công nghệ mới và lãi suất cao như là một blockchain.

EIP hoặc Ethereum Improvement Proposal là một phương tiện để quản lý đặc tả giao thức, sự cải thiện, cập nhật, client APIs và các tiêu chuẩn hợp đồng. EIP xử lý các vấn đề trong bốn thư mục khác nhau bao gồm: core (lỗi), hoặc core Ethereum protocol (lỗi giao thức Ethereum), network (mạng lưới) hoặc network level improvement (sự cải thiện cấp độ mạng), interface (giao diện) hoặc các giao diện như là ABI RPC, và ERC, hoặc các quy ước và tiêu chuẩn cấp ứng dụng.

ERC là Ethereum Request for Comments, giống như là RFCs của mạng máy tính.

5 Blockchain Platform

Trong phần này chúng ta dần đi sâu tìm hiểu về Blockchain và xem qua về hệ sinh thái của nó. Chúng ta cũng sẽ khám phá những hệ sinh thái khác ngoài nền tảng Blockchain. Các framework, công cụ, các giao thức thay thế Proof-Of-Work và làm việc với nền tảng ứng dụng phi tập trung.

5.1 Blockchain cấp quyền

Mục đích chính của Bitcoin blockchain là để hỗ trợ một hệ thống thanh toán ngang hàng phi tập trung. Nó tạo ra sự minh bạch, không cần cấp quyền và là một hệ thống công cộng, nơi bất cứ ai cũng có thể truy cập vào và rời đi khi họ muốn, giống như trong bất kỳ hệ thống thanh toán nào khác như giao dịch tiền mặt. Tuy nhiên, khi các tình huống sử dụng blockchain được mở rộng ra ngoài cả hệ thống thanh toán đơn giản vào các lĩnh vực kinh doanh như như hệ thống chăm sóc sức khỏe cá nhân và hệ thống tài chính, quyền riêng tư và quyền truy cập bị giới hạn trở thành bắt buộc và được bảo đảm.

Những suy nghĩ như vậy đã dẫn đến việc tạo ra Blockchain cấp quyền, nơi chỉ các nút có quyền mới có thể giao dịch, và tham gia vào hoạt động của blockchain. Vì vậy, chúng ta có đặc tính gọi là *permissioned blockchain*. Blockchain cấp quyền được biết đến như một blockchain liên kết dựa trên các tình huống sử dụng phổ biến của nó trong miền kinh doanh theo chiều dọc cụ thể, chẳng hạn như tập đoàn ô tô hay dịch vụ thực phẩm. Trong phần này, ta sẽ khám phá những đóng góp của hai tổ chức công nghệ lớn. Hyperledger Fabric của Linux Foundation, và dịch vụ blockchain Azure của Microsoft. Cái đầu tiên là một nền tảng *permissioned blockchain*, còn cái thứ hai sử dụng sức mạnh của điện toán đám mây cho phép người dùng làm việc trên một số nền tảng blockchain. Đây thực sự là những công nghệ đa dạng.

5.1.1 Hyperledger

Linux Foundation khởi xướng dự án Hyperledger vào năm 2015 để thúc đẩy liên hợp tác giữa các ngành công nghiệp. Mục đích là để duy trì và kết nối các bên liên quan, nhà cung cấp công nghệ và nhà phát triển để thúc đẩy sự phát triển và áp dụng các giải pháp blockchain. Ta có thể nhớ một tình huống tương tự khi Unix trở thành một hệ điều hành phổ biến dẫn đến xuất hiện nhiều phiên bản thương mại phi tiêu chuẩn như Solaris, AIX, BSD, v.v... Cuối cùng, lên đến đỉnh điểm là phiên bản mã nguồn mở của hệ điều hành Linux. Một điều tương tự cũng đang xảy ra đối với blockchain với đó là Hyperledger. Kiểu hợp tác này giữa các đối tác công nghiệp và cộng đồng các nhà phát triển cũng giúp hiểu nhu cầu tập thể của người dùng và giảm thiểu sự trùng lặp trong thiết kế và kỹ thuật. Nó cũng có thể cải thiện tốc độ thăng tiến, đặc biệt là đối với nền công nghệ non trẻ này, nơi nhiều thách thức vẫn đang chờ để được giải quyết.

Hyperledger là một hệ sinh thái hỗ trợ không chỉ giao thức blockchain, sổ kế toán phân phối, và hợp đồng thông minh, mà còn hỗ trợ các framework và công cụ cho sự tham gia và cộng tác tích cực của các nhà phát triển, doanh nghiệp và các bên liên quan khác. Các mục tiêu bao quát là để thúc đẩy sự phát triển của các thành phần mã nguồn mở hướng tới sự an toàn, đáng tin cậy, hiệu quả, sáng tạo, chất lượng nền tảng để hỗ trợ việc áp dụng công nghệ blockchain cho doanh nghiệp. Linux Foundation cùng các thành viên và các cộng sự đã thiết lập một ngôi nhà, và một môi trường để hỗ trợ những mục tiêu này. Framework cơ bản cho Hyperledger được định nghĩa bởi Linux Foundation. Sau đó, tổ chức thành viên có thể thiết kế blockchain của họ bằng cách mở rộng định nghĩa này. Dự án này cũng giúp xây dựng các công cụ và framework hỗ trợ các hoạt động phát triển giáo dục và ứng dụng. Người ta hy vọng rằng dưới sự bảo trợ của các thông số kỹ thuật cụ thể, mô-đun blockchain được tạo bởi các thành viên khác nhau sẽ tương thích với môi trường công nghệ của mỗi thành viên.

Tính đến thời điểm này, đã có hơn 191 thành viên, 4 công cụ, 5 framework và sự đóng góp vào codebase đến từ khắp nơi trên thế giới. Năm frameworks là **Fabric**, **Sawtooth**, **Indy**, **Iroha**, và **Burrows**, tất cả đều được nằm trong Hyperledger framework. Trong năm framework, một framework đã được đưa ra và đi vào sản xuất. Nó là phiên bản Hyperledger Fabric 1.0 từ IBM. Các công cụ là rất cần thiết để tạo mẫu và kiểm thử nhanh chóng, và 4 công cụ đang trong thời kỳ phát triển đó là, Cello, Quilt, Composer và Explorer.

Ta nhớ lại rằng giao thức Bitcoin đã định nghĩa các giao dịch của một blockchain trong các điều khoản của UTXO (Unspent Transaction Outputs) - Các đầu ra giao dịch chưa sử dụng. Giao thức Ethereum giới thiệu hợp đồng thông minh và khái niệm về một tài khoản. Hyperledger Fabric còn vượt xa hơn nữa trong việc xác định toàn bộ mạng lưới kinh doanh với vai trò và tài sản và đưa những giao thức đó gần hơn với ứng dụng trong thế giới thực. Lưu ý rằng Hyperledger Fabric 1.0 là sản phẩm framework duy nhất nằm ngoài Linux Foundation tại thời điểm này.

Trong giao thức Hyperledger không có tiền điện tử. Hyperledger được thiết kế để chỉ xử lý logic nghiệp vụ, chẳng hạn như chức năng hợp đồng thông minh. Mã hợp đồng thông minh được gọi là *chaincode* trong Hyperledger. Thậm chí còn có sự khác biệt quan trọng hơn giữa Bitcoin và Hyperledger là Fabric là một Blockchain cấp quyền và các bên không xác định không thể tham gia và rời khỏi mạng như họ muốn. Hyperledger hướng đến kiến tạo ra giải pháp cho B2B, B2C, hơn là bất kỳ các bên tham gia nào không xác định, không đáng tin cậy và phi tập trung. Khai thác các khối cũng khác nhau đáng kể vì chỉ được chỉ định các bên tham gia hợp lệ sẽ thực hiện hoạt động này trong Hyperledger. Tính năng này giúp cho việc triển khai một cách đồng thuận. Ở đây thuật toán PBFT - Practical Byzantine Fault Tolerance sẽ được sử dụng, khác so với đào Bitcoin Blockchain.

5.1.2 Dịch vụ của Fabric

Phần này sẽ liệt kê các dịch vụ dựa trên kiến trúc framework Hyperledger, liệt kê các dịch vụ và API được cung cấp bởi framework Hyperledger, giải thích các dịch vụ khác nhau được cung cấp bởi nó.

Theo whitepaper của giải thích các dịch vụ của HyperLedger Framework, thiết kế của Hyperledger tập trung nhấn mạnh vào tính mô-đun và cấu hình, mô-đun có thể cắm được. Một ứng dụng blockchain Hyperledger xem tên miền của nó được tạo thành từ nhiều blockchain tương tác ở nhiều mức dung lượng khác nhau. Kiến trúc tham chiếu chỉ định bốn nhóm dịch vụ khác nhau và API tương ứng để các ứng dụng truy cập chúng.

Những dịch vụ được cung cấp bởi Fabric là:

- Dịch vụ định danh, nhận diện.

- Dịch vụ chính sách.
- Dịch vụ blockchain.
- Dịch vụ hợp đồng thông minh.

Mô-đun dịch vụ nhận diện quản lý danh tính của các thực thể, người tham gia, đối tượng sở cái, chẳng hạn như hợp đồng thông minh. Ở Fabric thì nó được gọi là chaincode.

Mô-đun dịch vụ chính sách, quản lý việc kiểm soát truy cập, chi tiết bảo mật, quy tắc liên kết và quy tắc đồng thuận.

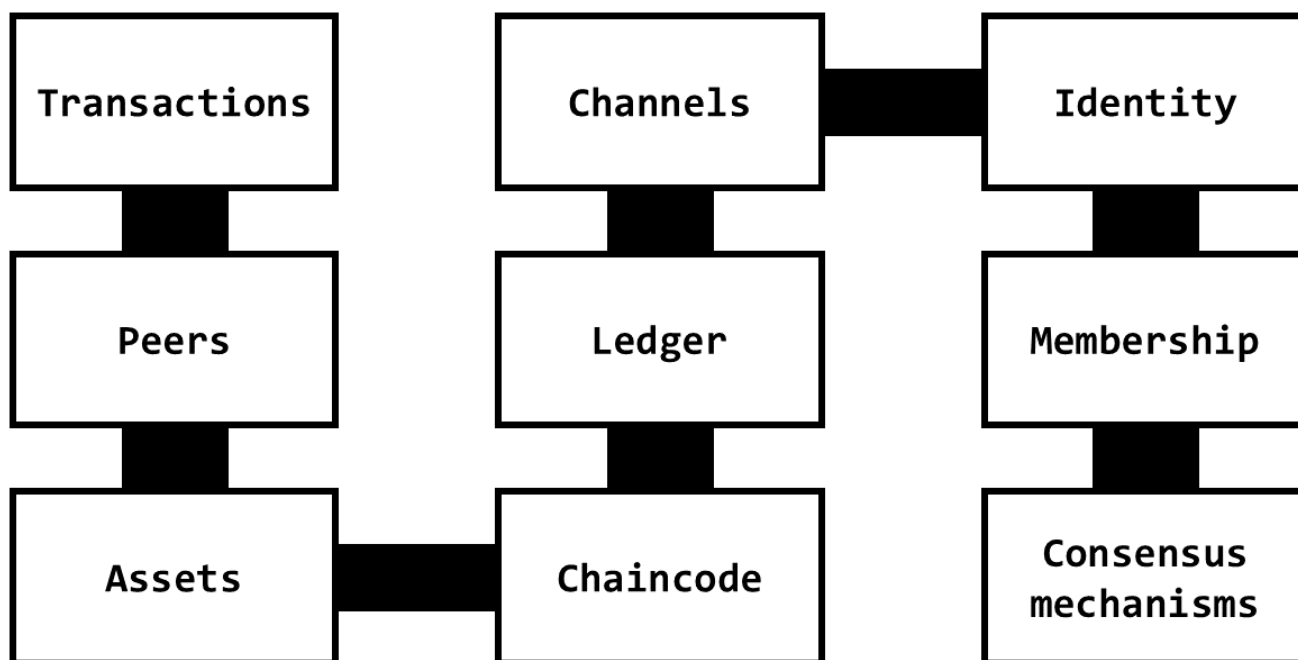
Mô-đun dịch vụ blockchain quản lý giao thức truyền thông ngang hàng, sở cái phân tán duy trì trạng thái toàn cầu, trạng thái toàn cầu được nhân rộng tại nhiều người tham gia, thuật toán đồng thuận cho phép kết nối được, gồm PBFT hoặc POW.

Các mô-đun dịch vụ hợp đồng thông minh cung cấp bảo mật và lightweight sandbox và thời điểm cho mã chuỗi để thực thi.

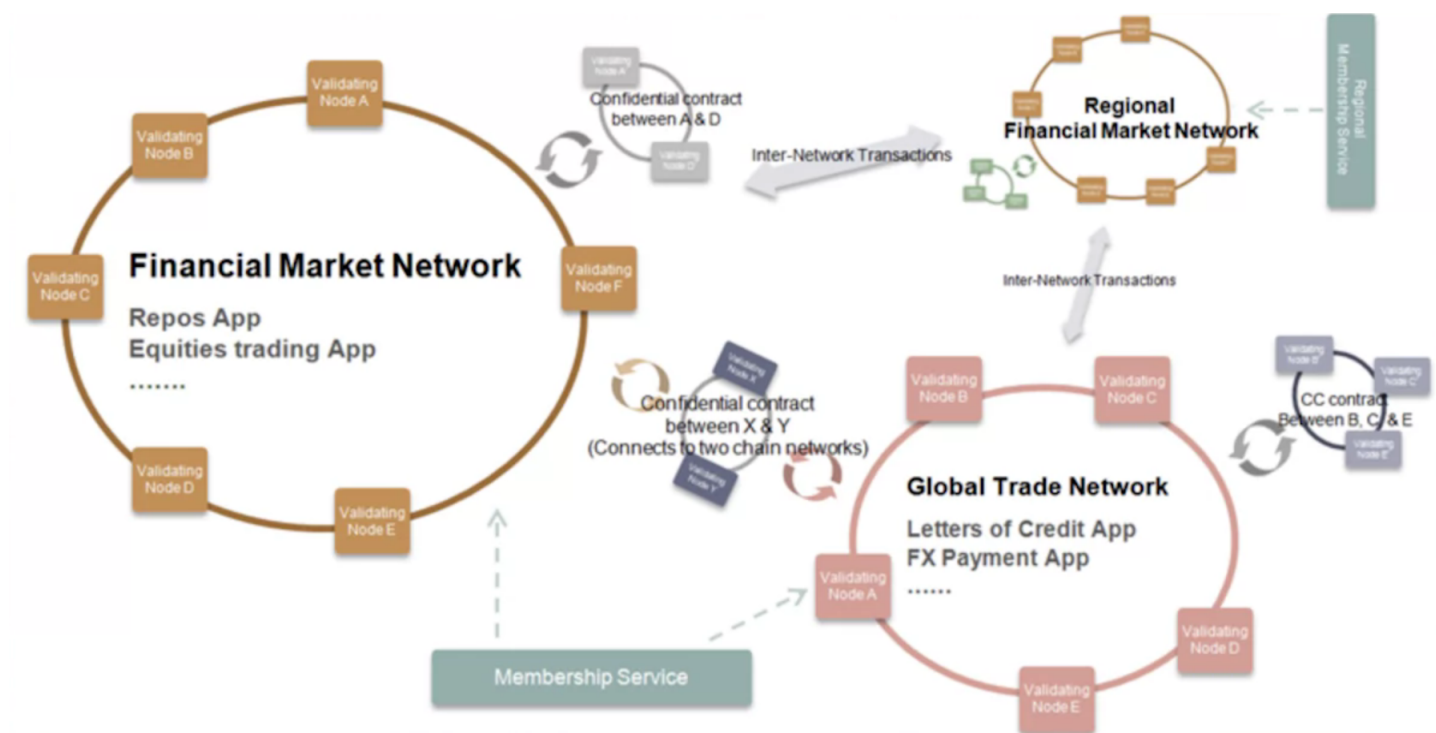
API cho phép các chương trình ứng dụng gọi vào các dịch vụ cơ bản. SDK giúp phát triển mã dựa trên các API này. CLI là giao diện dòng lệnh để gọi các API này cho mục đích kiểm thử.

5.1.3 Mô hình và chức năng của Fabric

Mô hình Fabric bao gồm các giao dịch, peer, asset, chaincode, ledger, channel, identity, membership và cơ chế đồng thuận (consensus mechanism).



Hình 21: Mô hình Fabric



Hình 22: Ứng dụng sử dụng fabric

Trên đây là một cái nhìn về một ứng dụng mà được hỗ trợ bởi Hyperledger Fabric. Nhiều blockchain kinh doanh mỗi cái đều có kênh riêng với những nút xác nhận, dịch vụ thành viên để quản lý danh tính và các thành viên. Tương tác giữa các doanh nghiệp độc lập khác nhau trong mạng lưới được kích hoạt bởi hợp đồng thông minh bí mật. Giao dịch có thể ở bên trong một mạng đơn hoặc giữa các mạng với nhau.

Peer là các nút khởi đầu giao dịch và duy trì trạng thái sổ cái. Có ba loại nút Peer là:

- **Endorsing peer (peer xác nhận)** nhận và kiểm tra giao dịch hợp lệ, ký và trả chứng về ứng dụng tạo ra giao dịch.
- **Ordering peer (peer sắp xếp)** thu tập các giao dịch đã được ký, sắp xếp chúng thành các khối và gửi chúng cho committing peer ().
- **Committing peer** nhận những khối được tạo bởi ordering service. Kiểm tra điều kiện như chi tiêu gấp đôi, ký và đưa chúng vào sổ cái.

Asset đại diện cho những mục giá trị hữu hình mà được giao dịch trong blockchain, ví dụ như cung cấp thực phẩm và tài sản tài chính. Assets được thể hiện trong chương trình dưới dạng cặp <key,value> trong JSON trên định dạng nhị phân.

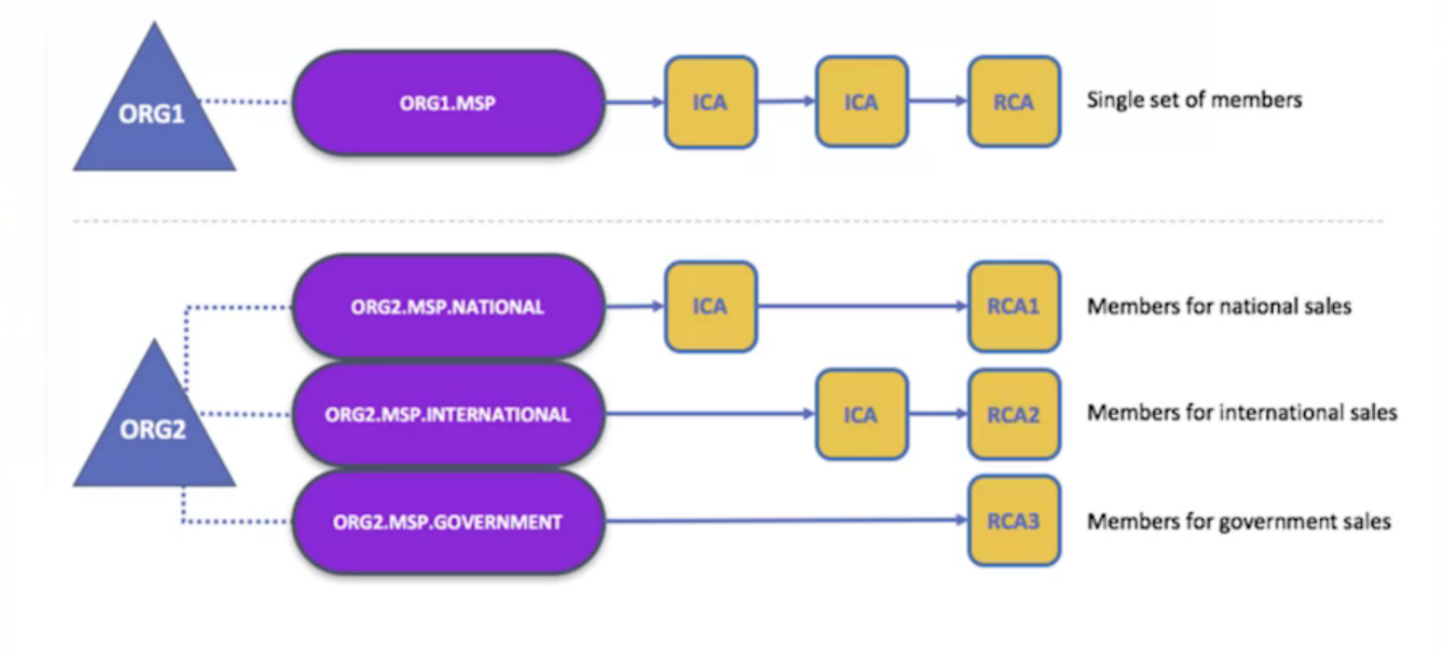
Chaincode là một hợp đồng thông minh định nghĩa một tập hợp asset và cung cấp các hàm để thực hiện trên các asset và thay đổi trạng thái. Nó cũng thực hiện quy tắc và chính sách ứng dụng cụ thể. Thực thi các hàm có thể dẫn đến thay đổi trạng thái được ghi trên sổ cái. Sổ cái này tương tự như các sổ cái ở các blockchain khác rằng nó là một hồ sơ chứng minh giả mạo của giao dịch trong Fabric, thay đổi trạng thái bị ảnh hưởng trong các hàm chaincode bởi các giao dịch được khởi động từ các bên tham gia trên Fabric. Giao dịch được chỉ định và tất cả các quy tắc kiểm soát truy cập được thiết lập bởi các permission service một giao dịch phải thực thi và bao gồm một khối để ghi lại. Mỗi một giao dịch tạo ra cặp asset mà được lưu lại trên sổ cái là create, update và delete. Bởi vì một sổ cái lưu theo cặp <key,value> có thể dễ dàng truy vấn để phân tích và kiểm định sau này.

Một thành phần khác nữa của mô hình Fabric là channel. Fabric yêu cầu quyền riêng tư và bảo mật thông qua khái niệm channel. Channel định nghĩa một permissioned network của các thực thể với một cái sổ cái duy nhất cho tất cả các giao dịch và thay đổi trạng thái của nó. Channel cung cấp Fabric riêng biệt cho một nhóm các thực thể để giao dịch riêng, ngay cả trong channel, dữ liệu và bảo mật giao dịch có thể đạt được bởi skating và sử dụng các phương pháp mã hóa. Channel cũng cung cấp khả năng để hỗ trợ giao dịch nhiều bên giữa doanh nghiệp cạnh tranh và ngành công nghiệp đã được điều chỉnh thông qua liên kết chéo giữa các chaincode. Trong trường hợp này, từng tập đoàn kinh doanh tư nhân hoạt động trên mạng lưới riêng của chính nó với giao dịch chéo giữa các chuỗi. được kiểm soát bởi chính sách được thực hiện trong chaincode.

Bây giờ, chúng ta chuyển sang phần membership service provider của Fabric. Nhớ lại blockchain công khai ví dụ như Bitcoin, có nghĩa là các đối tượng dù không được cấp phép và không xác định vẫn có thể tham gia và rời đi khi họ muốn. Các đối tượng muốn tham gia vào mạng lưới Fabric, cần đăng ký thông qua Membership Service Provider(MSP), một nơi đáng tin cậy. Một tổ chức quản lý tư cách thành viên, và vai trò của các đối tượng tham gia thông qua MSP.

Các đối tượng tham gia phải có danh tính có thể kiểm chứng được. Để nhận dạng sự tin cậy, triển khai mặc định của MSP sử dụng chứng chỉ X.509 để nhận dạng kỹ thuật số. Các nút ngang hàng, ứng dụng khách, các thực thể nghiệp vụ, và các quản trị cần định danh đồng nhất trong Fabric. Chúng được gán một định danh là một chứng chỉ X.509 Những nhận

dạng này xác định vai trò của các đối tượng, và các quyền mà họ có để truy cập các tài nguyên trong mạng blockchain. Có một cơ quan xác thực chứng chỉ gốc và cơ quan cấp chứng chỉ trung gian do MSP quản lý.



Hình 23: Ví dụ về cơ quan cấp quyền chứng chỉ

Trên đây ta thấy hai tổ chức. ORG1 không có bất kỳ tổ chức con nào. ORG2 có hai tổ chức con, mỗi tổ chức có thẩm quyền cấp chứng chỉ gốc và cơ quan cấp chứng chỉ trung gian cung cấp chứng chỉ nhận dạng.

Bây giờ chúng ta chuyển sang thành phần cuối cùng của mô hình Fabric, mô hình đồng thuận (consensus model). Sự đồng thuận ở mức cao là thỏa thuận về việc khối tiếp theo của giao dịch sẽ được thêm vào chuỗi, và xác nhận mở rộng và xác minh yêu cầu và tính chính xác của giao dịch bao gồm double-spend và các điều kiện khác. Fabric cho phép mô hình đồng thuận 'có thể plugin được'. Điều đó có nghĩa là, tùy thuộc vào đặc tính của channel, định thức con của khối tiếp theo được quyết định bởi một chính sách vòng tròn hoặc thuật toán "Practical Byzantine Fault Tolerant" hoặc bất kỳ thứ gì trung gian. Nếu channel được coi là có mức độ tin cậy cao, một mô hình đồng thuận đơn giản sẽ thực hiện. Nếu đó là một channel B2C (Business To Customer) nơi có tiềm năng cho những hoạt động không đáng tin cậy thậm chí là một thuật toán PoW cũng có thể được plug-in như một mô hình đồng thuận.

Tại trung tâm, Fabric cung cấp Công nghệ Sổ cái Phân tán (DLT) và thông qua các thành phần khác nhau mà ta đã thảo luận trong mô hình của nó, nó cung cấp các chức năng sau đây: bảo mật và quyền riêng tư thông qua việc xây dựng mạng permission blockchain giữa các peer kinh doanh đáng tin cậy. Fabric cũng cung cấp sự riêng biệt qua các channel quản lý danh tính thông qua các membership service. Nó cung cấp hiệu quả cho các loại nút ngang hàng khác nhau hoạt động song song, xác nhận, yêu cầu, và cam kết những giao dịch trên chuỗi. Chức năng chaincode giúp thực hiện các phép lô-gic dành riêng cho ứng dụng cụ thể, và tính cấu hình thông qua kiến trúc mô-đun của Fabric. Cho phép các mô-đun có thể kết nối với bất kỳ thứ gì từ mô hình đồng thuận với sự quốc tế hóa và mô-đun policy cho các vùng khác nhau trên thế giới.

5.1.4 Composer

Dự án Hyperledger của Linux foundation không chỉ tạo ra các framework blockchain bởi các đối tác khác nhau như IBM và Intel, mà còn phát triển một số công cụ để phát triển các ứng dụng. Ta sẽ xem xét một số công cụ. Công cụ Yeoman, có thể được sử dụng để tạo bộ khung cho Business Network, ngôn ngữ Composer Modeling được sử dụng để định nghĩa các asset của ứng dụng kinh doanh, và công cụ Composer để triển khai các ứng dụng Composer cũng tương tự như Truffle mà chúng ta đã sử dụng trong mục trước để phát triển DApp.

Ta có một cái nhìn tổng quan về các kỹ thuật bằng cách sử dụng các công cụ trong việc phát triển một mạng blockchain kinh doanh đơn giản. Các bước phát triển là: sử dụng Yeoman để tạo mã khung cho business network. Trong tệp .cto được tạo ra, ta sẽ thấy định nghĩa lớp cho tất cả các asset, người tham gia và các giao dịch trong business network. Tệp này được viết bằng ngôn ngữ Hyperledger Composer Modeling. Trong tệp JavaScript là các hàm giao dịch. Trong tệp kiểm soát truy cập (.acl), là những quy tắc kiểm soát quyền truy cập đơn giản. Sau khi cập nhật tệp .cto, .js và .acl, toàn bộ thư mục được đóng gói bằng Composer, để đóng gói mã vào một kho lưu trữ business network có thể triển khai. Composer được sử dụng để thiết lập thông tin xác thực chạy REST server và triển khai ứng dụng. Composer Playground được thực hiện để cho phép tương tác với business network đã được triển khai.

5.1.5 Microsoft Azure

Sự đổi mới blockchain đã tác động tới mọi ngành công nghiệp. Phần này sẽ nói về những cống hiến của gã khổng lồ công nghệ Microsoft tới sự phát triển blockchain. Microsoft đang phát triển nền tảng của riêng nó trên CoCo blockchain với sự kết hợp cùng Ethereum và các công ty tài chính. Cùng với đó, nó cũng đã tạo ra một khung công việc để quản lý các blockchain công nghiệp khác nhau trong kho lưu trữ đám mây của nó. Ta tìm hiểu blockchain của Microsoft Azure như là một mô hình dịch vụ.

Mục tiêu chính của Microsoft Azure là tăng tốc sự triển khai blockchain. Nó thực hiện việc này bằng cách đưa ra các công cụ và môi trường phát triển để nhanh chóng triển khai các giải pháp end-to-end. Cung cấp các khả năng tích hợp thông qua plug-and-play giữa nhiều nền tảng. Cho phép việc triển khai nhanh chóng của các sổ cái và mạng lưới và cung cấp blockchain trên đám mây hoặc tại chính cơ sở hạ tầng nội bộ của người dùng. Azure blockchain là một tính năng dịch vụ bao gồm:

- một tập các sổ cái sẵn sàng triển khai
- một mạng blockchain với nhiều nodes đã băm nhỏ, được đào, sự hài hòa giữa các nodes và sự phân phối sổ cái nhân rộng tới tất cả các node
- một mạng đã được cấu hình sẵn để phát triển lô-gic nghiệp vụ
- các công cụ và cơ sở hạ tầng trong một không gian độc lập
- bảo mật dữ liệu và khả năng mở rộng của nền tảng đám mây
- Single Node Ledger và Multi Node Ledger

Sự cung cấp của Azure blockchain tính đến bây giờ bao gồm Ethereum và Enterprise Ethereum, Hyperledger fabric, R3 Corda, Chain và Quorum. Azure tạo ra hạ tầng cơ sở cần thiết cho bất cứ nền tảng blockchain nào và duy trì mạng lưới cho chúng. Cấu trúc mạng khác nhau tùy thuộc vào nền tảng của mỗi blockchain. Ethereum sẽ có các nodes minor và non-minor, trong khi R3 Corda sẽ có các nodes notary và các nodes participating. Cấu trúc mạng cũng phụ thuộc vào cấu hình node mà ta chỉ định. Tất cả các thành phần mạng sẽ phụ thuộc vào nhóm tài nguyên độc lập.

Để sử dụng Azure, Đầu tiên, ta cần lựa chọn nền tảng. Ví dụ như là, Ethereum, Corda hoặc Hyperledger. Tiếp theo, dựa trên mô hình ứng dụng của mình, ta cần chọn loại sổ cái, single hay multi-node. Một khi đã chọn, ta cần điền vào cấu hình mạng cần thiết. Ví dụ cho Ethereum, số lượng các nodes, loại lưu trữ máy ảo, kích thước máy ảo, v.v. Sau khi điền các chi tiết cấu hình, Azure sẽ tạo ra một nhóm tài nguyên bao gồm các tài nguyên cần thiết cho mạng lưới của ta dựa trên những cấu hình đầu vào mà ta đã cung cấp. Kết nối, duy trì, và cơ sở hạ tầng sẽ được Azure cung cấp. Ta có thể ssh vào các nodes để tương tác và tất cả các nodes được kết nối sẽ tự động đồng bộ. Thêm nữa, ta có thể thêm các công cụ bổ sung vì mục đích phát triển như Truffle IDE.

5.2 Nền tảng ứng dụng phi tập trung

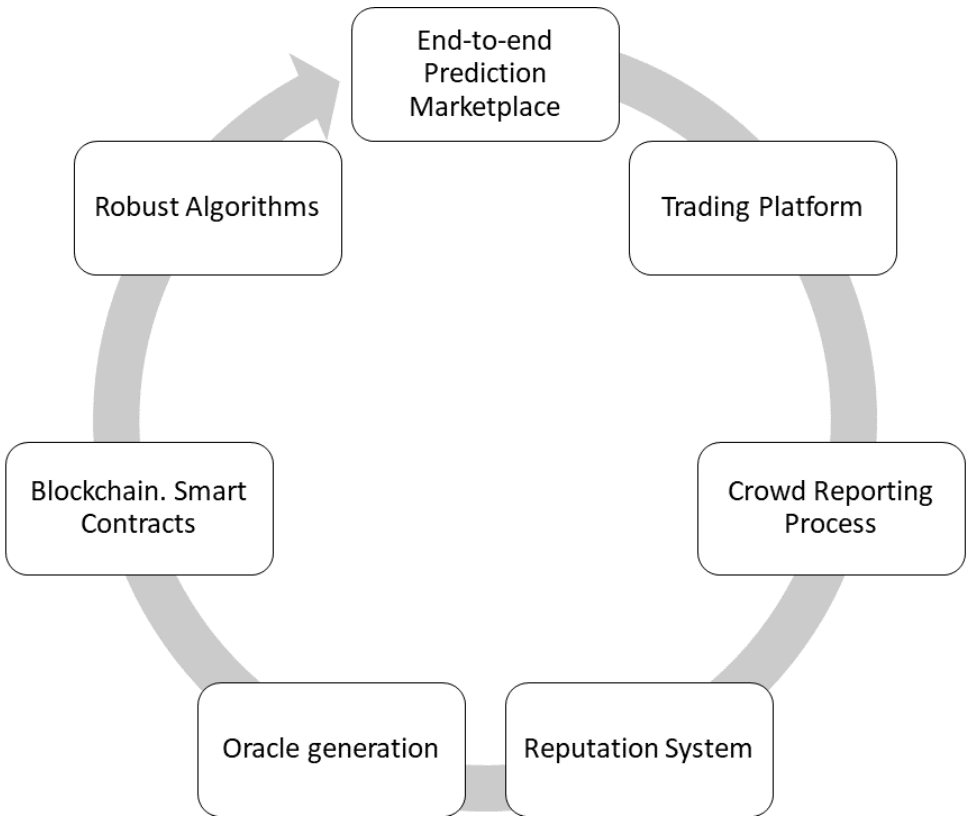
Ta đã nói nhiều về nền tảng Blockchain, giờ phần này sẽ bàn về các ứng dụng thực hành để minh họa những tính năng đặc biệt của công nghệ blockchain. Đó là Decentralization (phân tán/phi tập trung), Disintermediation (phi trung gian), và Distributed Immutable Ledger (sổ cái phân phối không thay đổi). Chúng ta sẽ khám phá hai nền tảng Dapp, Augur và Grid Plus, cả hai cái này đều được triển khai trên Ethereum. Các ý tưởng và kế hoạch cho các ứng dụng này đã song hành cùng sự phát triển của Ethereum. Augur và Grid Plus là các lớp phía trên smart contract, mỗi cái mở ra một blockchain và lớp smart contract với sự mở rộng các miền dọc. Trong trường hợp này, nó tương ứng là các thị trường dự báo và bán lẻ năng lượng. Augur và Grid+ được chỉ định để minh họa hai cách dùng rất đặc trưng của blockchain. Những ứng dụng này có lịch sử phát triển, các thành phần phức tạp và sự hoạt động. Việc chứng minh giao dịch và rất quan trọng trong cả hai ứng dụng. Chúng đang giải quyết những vấn đề thực tế, không phải là giả thuyết. Chúng cung cấp các mô hình sáng tạo cho các miền khác để thích ứng.

5.2.1 Augur

Ta nói về Decentralized Prediction Markets (các thị trường dự báo phân cấp/phi tập trung). Các thị trường dự báo đã không còn xa lạ. Chúng đã xuất hiện từ những ngày đầu tiên trong lịch sử. Tất cả các khảo sát mà ta thấy trong thư, kết quả các cực (poles) và cá cược mà ta có thể đưa ra trong các môn thể thao là ví dụ của các thị trường dự báo. Ý tưởng cốt lõi của những thị trường này là khi ta đặt cược một kết quả của một sự kiện. Mặc dù những cái này đã tồn tại từ rất lâu, hầu hết các thị trường dự báo này đều tập trung (centralized), cũng luôn có một sổ cái, nhưng thông thường thì nó được nội bộ hóa cho một tổ chức. Phi tập trung với một sổ cái phân phối không thay đổi đối với blockchain mở ra cho thị trường dự báo rất nhiều cơ hội khổng lồ.

Augur thúc đẩy sự phân quyền tin cậy của blockchain và mở ra các thị trường dự báo theo quyết định của số đông. Đây là một cải tiến cần thiết. Augur là một nền tảng thị trường dự báo phi tập trung trustless dựa trên công nghệ blockchain. Người tham gia có thể vào và rời đi bất cứ khi nào muốn. Đây là lý do nó gọi là trustless. Nó phi tập trung vì nền tảng sau nó dựa trên đặc tính phi tập trung của blockchain. Những người tham gia không bị ràng buộc bởi bất cứ cơ quan trung ương nào. Trong một mẫu điển hình của hoạt động, người tạo ra thị trường đăng lên một sự kiện mà cô ấy thăm dò ý kiến của

những người tham gia. Các phóng viên thị trường suy đoán về kết quả của sự kiện này. Một khi kết quả được biết, những người tham gia, những người dự báo kết quả chính xác, sẽ thắng một phần tiền và những người khác mất số tiền mà họ đã cược. Quá trình end-to-end được thực hiện một cách tự động bởi Ethereum Smart Contract. Đây là một cách lý giải rất đơn giản.



Hình 24: Một số ý tưởng chính được cài đặt trong Augur

5.2.2 Grid+

Phần này ta cần nhắc đến khái niệm **DER - Distributed Energy Resources - tài nguyên năng lượng phân tán**. Sự phi tập trung đang xuất hiện nhiều trong sản xuất năng lượng. Grid+ là một lớp tính toán hoặc nền tảng Dapp cho hệ thống tiết kiệm năng lượng trên lớp smart contracts. Chuỗi cung ứng năng lượng gồm có 4 pha: sản xuất, truyền tải, phân phối và bán lẻ. Trong số này, Grid+ tập trung vào việc cải tiến mảng bán lẻ năng lượng với sự tham gia của công nghệ blockchain. Có nhiều cơ hội để hiện đại hóa, cải thiện tính hiệu quả, cập nhật hệ thống thanh toán, và cắt giảm chi phí cho người dùng cuối. Những chuyên gia trong lĩnh vực này đề xuất rằng khoảng 50% chi phí năng lượng của người dùng cuối không dành cho điện mà dành cho việc vận hành hành chính. Mục tiêu của Grid+ là giảm thiểu chi phí này với công nghệ tốt hơn và tạo ra chi phí lưới điện hiệu quả, đáng tin cậy và mạnh mẽ.

Grid+ là một nền tảng Dapp blockchain triển khai trên Ethereum blockchain. Grid+ đã tạo ra hệ thống tiết kiệm năng lượng bằng cách tích hợp blockchain và AI, do đó tạo ra sự nhảy vọt trong nền kinh tế năng lượng sang kỷ nguyên hiện đại. Grid+ hướng tới dịch chuyển việc chuyển giao năng lượng và giao dịch thanh toán trên cấu trúc blockchain mới nổi. Grid+ thúc đẩy các tín hiệu thị trường tới người dùng cuối, những người có thể nắm giữ việc điều khiển các tài sản năng lượng của mình và đưa ra những quyết định thông minh. Điều này cải thiện hiệu quả sử dụng năng lượng và cũng giảm chi phí cho người dùng cuối. Grid+ cung cấp một hệ thống thanh toán liền mạch, thúc đẩy crypto tokens (mã thông báo mật mã), do đó cải thiện hiệu quả trong hệ thống thanh toán năng lượng. Ta có thể dễ dàng thấy, nó đều về các khâu phi trung gian của hệ thống năng lượng.

Trong một thị trường điện deregulated, người tham gia thị trường không chỉ là những công ty tiện ích sở hữu nhà máy điện và đường dây truyền tải. Trong những bối cảnh như vậy, các công ty có thể sản xuất điện, bán điện cho thị trường bán buôn, và những nhà cung cấp năng lượng lẻ giao dịch điện này để bán nó cho khách hàng. Thị trường điện deregulated này cung cấp cơ hội cho Grid+ để kiểm tra khả năng hoạt động và tính khả thi thương mại của nó. Sau đây là những ý tưởng chính của Grid+:

- **Người bán lẻ năng lượng (Energy Retailer):** Grid+ sẽ hoạt động như một người bán lẻ điện thương mại trong các thị trường deregulated.
- **Smart Agent:** kiểu một đại lý thông minh trong gia đình người dùng hoặc khách hàng. Một Grid+ Smart Agent là một thiết bị tính toán có thể quản lý một phần mềm dành cho giao dịch blockchain, ví tiền mã hóa nhiều chữ ký với bảo mật PKI, và thanh toán ngoài chuỗi đối với các xác minh nhanh hơn.

- **Intelligent electricity usage - Cách dùng điện thông minh:** Giao dịch điện là một quá trình hoàn chỉnh với nhiều sự phức tạp, Grid+ quản lý những việc này bằng cách lập trình các tùy chọn giá hiệu quả cho khách hàng sử dụng smart contract.
- **Thanh toán ERC20 Token:** một ERC20 compliant token đặc biệt gọi là BOLT đã được tạo ra vì mục đích thanh toán.
- **Intergration to IOT devices - tích hợp trong các thiết bị IoTs:** một smart agent có thể được tích hợp bên trong các smart agents khác trong nhà như là NEST và pin điện tử (vd: Tesla Power Wall)
- **Remote control - điều khiển từ xa:** Grid+ cho phép tích hợp trong các thiết bị điện thoại di động và máy tính để cho phép điều khiển các hoạt động của nó từ xa. Giao diện người dùng trực quan cho phép việc tương tác với hệ thống một cách dễ dàng. Mặc dù blockchain và crypto token có liên quan, người dùng cuối không cần biết bất cứ kiến thức kỹ thuật nào về cơ sở hạ tầng cơ bản đây.

5.3 Các thách thức và giải pháp

Bất cứ công nghệ mới nổi nào cũng sẽ gặp phải những thách thức khi nó đang trưởng thành/hoàn thiện. Blockchain không phải một ngoại lệ. Lĩnh vực này đang diễn ra sôi động với nhiều hoạt động và sáng kiến để liên tục cải tiến công nghệ blockchain. Có rất nhiều thách thức cản trở việc áp dụng nó rộng rãi. Chúng ta sẽ tìm hiểu một số những thách thức quan trọng và các giải pháp đang liên tục đổi mới công nghệ mới nổi này.

5.3.1 Tính đồng thuận

Consensus nghĩa là sự đồng thuận chung. Trong bối cảnh của hoạt động blockchain, consensus là sự đồng thuận giữa các nodes đầy đủ về khối tiếp theo sẽ được thêm vào chuỗi. Điều này đảm bảo tính nhất quán của chuỗi. Ngay cả giữa những thực thể đã được cho phép, chúng ta cũng cần sự đồng thuận. Các giao thức đồng thuận khác nhau được dùng trong các blockchains khác nhau. Bitcoin dùng bằng chứng về công việc (Proof Of Work - POW) cho sự đồng thuận. Nó được tính toán chuyên sâu, và kết quả là lượng tiêu thụ điện năng khổng lồ trong các kệ lớn của máy tính ASIC dùng trong xử lý các PoW puzzle, cho quyền khai thác khối tiếp theo. Người ta ước tính việc khai thác Bitcoin mỗi ngày tiêu thụ nhiều năng lượng ngang với cả đất nước Ireland. Có nhiều thống kê cảnh báo khác về việc khai thác Bitcoin. Vấn đề là ở năng lượng khổng lồ dùng bởi POW cho việc khai thác một khối.

Proof of Work hoạt động như sau. Tính toán việc băm các yếu tố tiêu đề khối đã cố định và nonce là một biến số. H là hash của header và nonce. Nếu hash nhỏ hơn 2 mũ 128 của Bitcoin và ít hơn hàm độ khó của Ethereum, thợ đào đã giải quyết được câu đố. Nếu không thì, thay đổi nonce và lặp lại các bước trên. Trong khi nó khá khó để tìm được kết hợp giữa header và nonce để giải quyết vấn đề mà chúng ta đã mô tả, nó lại khá dễ để xác minh. Ta xác minh hash đó ít hơn hoặc bằng 2 mũ 128 bằng cách kiểm tra nếu 128 bits dẫn đầu của hash H là 0.

Bitcoin đã dùng Proof of Work (bằng chứng công việc) hoặc gọi là PoW, và Ethereum cũng bắt chước với một số điều chỉnh nhỏ. Trong phiên bản hiện nay, Ethereum Metropolis, nó vẫn sử dụng PoW. Tuy nhiên, giao thức Ethereum Metropolis sử dụng bộ nhớ dựa trên POW mà không hao phí năng lượng.

Ethereum Constantinople đã chuyển sang sử dụng PoS, hoặc Proof of Stake (bằng chứng cổ phần). Trong PoS, các node đầy đủ với cổ phần nhiều nhất hoặc coin nhiều nhất được chọn để thêm vào khối tiếp theo. Đây là lý do nó được gọi là Proof of Stake. Ý tưởng là, node đầy với nhiều cổ phần nhất sẽ không độc hại. Để tránh bị độc quyền bởi node nhiều cổ phần nhất, một thời gian dựa trên quy tắc robin được dùng thêm vào PoS cơ bản. Thực tế, vì Ethereum đang chuyển sang sử dụng PoS của nó, phí khai thác đã giảm từ 5 ether xuống 3 ether. Hướng tiếp cận POS là thân thiện với môi trường và hiệu quả, và được chấp nhận khá phổ biến bởi nhiều nền tảng blockchain.

Các thuật toán đồng thuận tiếp theo mà chúng ta sẽ xem xét là Practical Byzantine Fault Tolerance (dung sai lỗi Byzantine thực tế), PBFT. Thuật toán này đã được chứng minh để xử lý các ngẫu nhiên hoặc sai sót của Node Byzantine, bao gồm các nodes độc hại. Trong PBFT, các nodes bỏ phiếu để cử ra leader và leader đẩy thêm khối tiếp theo vào chuỗi. Đây là khối của các giao dịch đã được xác minh. PBFT khá phổ biến trong các blockchains được cấp phép, như là Hyperledger Fabric.

5.3.2 Tính mở rộng

Hãy so sánh các hệ thống blockchain phi tập trung với các hệ thống tập trung. Blockchain đã đảm nhận trách nhiệm của các bên trung gian dưới hình thức xác nhận, xác minh và ghi lại các giao dịch, và tất nhiên, quá trình đồng thuận cho sự toàn vẹn của chuỗi. Tất cả các chức năng này mất thời gian và dẫn đến chi phí đáng kể trong thời gian xác nhận của các giao dịch, khi so sánh với hệ thống tập trung. Giao dịch trong Ethereum, ví dụ, thực hiện trên tất cả các nút và thực hiện tuần tự, không song song. Mỗi nút đầy đủ lưu trữ toàn bộ chuỗi các khối. Tất cả những điều này cản trở khả năng mở rộng của các ứng dụng blockchain. Tỷ lệ giao dịch không đạt yêu cầu so với ứng dụng tập trung hiện tại. Đây là thử thách.

Khả năng mở rộng là khả năng của một hệ thống để thực hiện thỏa đáng ở tất cả các mức tải thực tế. Tải trong bối cảnh của blockchain có thể là giao dịch, số nút, số lượng người tham gia và tài khoản, và các thuộc tính khác của blockchain. Trong trường hợp của blockchain, các nhà nghiên cứu đang quan tâm đến tỷ lệ giao dịch hoặc giao dịch mỗi giây. Đây là một số liệu quan trọng đối với nhiều ứng dụng từ hệ thống thanh toán đến quản lý chuỗi cung ứng để thực hiện tốt. Vì vậy, chúng ta tập trung vào các giao dịch mỗi giây như là một số liệu cho khả năng mở rộng.

Phương pháp rõ ràng nhất để cải thiện tỷ lệ giao dịch là tăng số lượng giao dịch trên mỗi khối. Bitcoin giải quyết nó theo hai cách, tách biệt nhân chứng và tăng giới hạn kích thước khối. Trong nhân chứng tách biệt (segregated witness) hoặc gọi là Segret, các giao dịch và chữ ký được tách biệt để cho phép nhiều giao dịch hơn trên mỗi khối. Đây là một soft fork mà được hiện thực hóa vào năm 2017. Nó đang hoạt động trong phiên bản hiện tại của blockchain Bitcoin. Tuy nhiên, phiên bản hiện tại, giới hạn kích thước khối được cố định tại một megabyte.

Bây giờ, chúng ta tìm hiểu cách Ethereum giải quyết kích thước khối. Trong Ethereum, kích thước khối có thể thay đổi, và bị giới hạn bởi các giới hạn gas được chỉ định trong tiêu đề khối. Nó có thể được tăng lên bởi các thợ mỏ làm việc trên nó. Số lượng giao dịch trong một khối quy định kích thước khối. Kích thước khối biến đổi này cùng với thuật toán đồng thuận "ASIC-resistant Pure W" đã cải thiện tỷ lệ giao dịch. Nó đã được cải thiện hơn nữa khi Ethereum chuyển sang Proof Of State trong một hard fork vào năm 2018.

Ta vừa tìm hiểu về các giải pháp trên chuỗi để cải thiện tỉ lệ giao dịch. Giờ đến lượt những giải pháp ngoài chuỗi. Một trong những giải pháp để giải quyết tỷ lệ giao dịch là dỡ bỏ một số giao dịch off-chain. Điều này được thực hiện giữa các bên giao dịch đáng tin cậy. Một khi việc dỡ bỏ đã kết thúc, một giao dịch xác nhận hoạt động off-chain này được ghi lại trên chuỗi trên blockchain DLT. Tính năng này trong Bitcoin được gọi là kênh thanh toán. Các giao dịch thanh toán có thể được thực hiện với mức phí tối thiểu với mức giá cao hơn đáng kể giữa các bên đáng tin cậy. Một giải pháp Ethereum mở rộng khái niệm kênh thanh toán này sang Hợp đồng thông minh. Giao dịch của các kênh off-chain diễn ra ở tốc độ cao hơn nhiều so với mạng blockchain vì không cần sự đồng thuận hoặc ghi âm theo DLT. Do đó, cung cấp một giải pháp có thể mở rộng. Mạng lưới radar là một giải pháp off-chain cho chuỗi chính dựa trên Ethereum.

5.3.3 Tính bảo mật và riêng tư

Privacy (quyền riêng tư) là quyền để bảo mật dữ liệu, các thuộc tính, và tài sản của một thực thể khỏi sự quan sát bởi các bên không liên quan. Việc bảo mật dữ liệu đảm bảo rằng chỉ những bên có liên quan mới có thể truy cập vào dữ liệu. Có hai khái niệm quan trọng đặc biệt là cho blockchain khi mà các giao dịch có thể được xem xét và phân tích sử dụng các công cụ người dùng tự thiết kế. Chúng ta sẽ xem xét các phương pháp để xử lý quyền riêng tư và bảo mật. Ta đều đã biết rõ là Bitcoin là một public blockchain. Ta có thể thấy tất cả các giao dịch xảy ra trên Bitcoin blockchain. Rất nhiều blockchain khác bao gồm Enterprise Ethereum và Hyperledger Fabric là các blockchains cần cấp phép. Chúng giới hạn truy cập vào chuỗi cho những thực thể được cấp phép. Điều này đảm bảo sự riêng tư của mạng lưới ở một mức độ lớn. Trong mạng lưới, đặc quyền truy cập dữ liệu như tạo ra, đọc, cập nhật được giao cho các thực thể khác nhau. Những điều này là bắt buộc với mỗi giao dịch. Điều này thêm vào quyền riêng tư cơ bản cung cấp bởi các mạng lưới cần cấp phép. Một số nền tảng blockchain như là MultiChain đảm bảo rằng các nodes giao dịch thực sự nằm trong danh mục được cấp phép. Nó cũng xác thực người gửi và người nhận bằng cách dùng trao đổi khóa cho mỗi giao dịch.

Đối với bảo mật, một phương thức trực tiếp là mã hóa dữ liệu được giao dịch và ký điện tử cho nó. Cũng có thể làm xáo trộn dữ liệu bằng cách thêm vào các thành phần bổ sung. Chúng ta minh họa việc mã hóa và làm xáo trộn với một dữ liệu smart contract tên là blindedBid.

```
1 blindedBid = keccak(value, fake, secret);
```

Listing 11: Ví dụ về một phương pháp để bảo mật

Ở đây bid thực sự ở trong smart contract đầu giá là value, nhưng chúng ta đã thêm vào hai tham số khác nữa gọi là fake và secret. Trong trường hợp này, blindedBid được gửi bởi bidder bị xáo trộn với giá trị fake và giá trị secret như là các tham số, và sau đó được mã hóa với thuật toán băm bảo mật Keccak. Điều này cung cấp bảo mật của bid trong ứng dụng đấu giá.

5.3.4 Escrow & Multi-sig

Rất nhiều hợp đồng và tài liệu pháp lý bao gồm các tài liệu kỹ thuật số dễ bị lỗi. Ví dụ như, một dấu phẩy bị đặt nhầm chỗ trong một hợp đồng giấy có thể khiến 1 công ty viễn thông mất đến cả triệu đô la. Escrow và multisig là hai kỹ thuật có tiềm năng khổng lồ trong việc xử lý hợp đồng kinh doanh có liên quan các vấn đề trong các quá trình chuyển đổi kinh doanh.

Wikipedia định nghĩa Escrow là một thỏa thuận hợp đồng trong đó bên thứ ba nhận và giải ngân tiền hoặc tài liệu cho các bên giao dịch chính, với việc giải ngân phụ thuộc vào các điều kiện đã được các bên giao dịch đồng thuận. Một smart contract có thể thực hiện tất cả những việc này. Nó có thể giữ tiền và tất cả các artifacts trong một Escrow, và cho phép các bên giao dịch thương lượng trên đó, và xác minh các điều kiện cần đáp ứng trước khi thực hiện việc giải ngân tài sản trong Escrow. Sự cải tiến nằm ở chỗ, nó có thể được thực hiện giữa các bên mà đang hoạt động vượt ngoài ranh giới của niềm tin trong một thế giới phi tập trung. Hãy nhớ lại rằng một smart contract có một địa chỉ có thể giữ một số dư. Ta có thể giao dịch, gửi hoặc nhận tiền mã hóa từ smart contract. Ta có thể dùng ba tính năng cụ thể của smart contract để phát triển ứng dụng dựa trên Escrow. Bao gồm trong đó có:

- Chứa số lượng escrow như là một biến trạng thái, ví dụ: bản thân ta và số lượng Escrow.
- Sử dụng modifier để xác định điều kiện cho việc giải ngân của Escrow.
- Xác định các hàm cần áp dụng công cụ điều chỉnh để xác minh các điều kiện cho việc giải ngân số lượng.

Giữ quỹ Escrow thường hoạt động với nhiều chữ ký. Điều này được biết đến như là tính năng Multisig. Nhiều hợp đồng thực tế có việc ký nhiều chữ ký để đảm bảo an toàn, xác thực và bảo vệ tài sản, việc kinh doanh và các cá nhân tham gia

vào hợp đồng. Trong nhiều trường hợp, nó có thể bị yêu cầu bởi luật định rằng nhiều bên phải ký một tài liệu. Một chữ ký trong thế giới mã hóa là một khóa bí mật của địa chỉ, các giao dịch được ký bởi địa chỉ này để xác thực tính đích thật của người gửi. Bảo mật tài sản bằng địa chỉ này chỉ tốt như bảo mật khóa bí mật. Nếu khóa bị xâm phạm, bất cứ ai có khóa bí mật có thể rút quỹ ở địa chỉ tương ứng. Các vấn đề này có thể được xử lý bằng cách bắt buộc nhiều chữ ký khi thực thi những giao dịch quan trọng cụ thể. Ý tưởng multikey hoặc multisig không mới. Chúng ta đã thấy giải pháp hai khóa trong kết tiền gửi an toàn của ngân hàng. Ta cần hai khóa để mở nó. Một khóa do ta giữ, và cái còn lại do ngân hàng giữ. Một tên trộm không thể lấy được nội dung của hộp tiền gửi an toàn khi không chiếm được cả hai khóa này. Giống như vậy, bitcoin đã sử dụng ý tưởng này thành công từ 2012.

Để sử dụng Multisig, ta tạo ra một địa chỉ multisig, nó có n khóa, m trong đó được yêu cầu để kích hoạt một hành động. Sau đó nạp tiền cho địa chỉ multisig mới với một khoản tiền gửi. Bây giờ ta đã sẵn sàng để giao dịch. Tạo một giao dịch multisig. Giao dịch này sẽ được ký với chữ ký đầu tiên là MS 1, kết quả là một chuỗi hex, HS 1. Sau đó chúng có thể được ký bởi chữ ký thứ hai MS 2, kết quả là chuỗi hex HS 2. Điều này lặp lại m lần cho m chữ ký, kết quả là HS m . Đây là khóa multisigned bắt buộc. Nó được xác thực bởi giao thức và các quỹ được chuyển. Điều này giải thích hoạt động multi-sig.

5.4 Các giải pháp phi tập trung khác

Kể từ khi ra đời blockchain, một số giải pháp khác cho hệ thống phi tập trung đã được đề xuất nhằm các mục đích khác nhau. Sự hỗ trợ mạnh mẽ và nhiệt tình từ cộng đồng đang thúc đẩy khái niệm về hệ thống phi tập trung. Dưới đây là một số đóng góp nổi bật.

5.4.1 Hệ thống file liên kết

Một blockchain cơ bản là một hệ thống phi tập trung. Nói cách khác, một hệ thống phi tập trung ngang hàng có thể được nhận diện độc lập với một blockchain. Interplanetary Files System (hệ thống tệp tin liên hành tinh), IPFS, là ví dụ hay ho của kiểu hệ thống như vậy. IPFS là một mô hình phi tập trung cho việc chuyển tệp tin trái ngược với không gian tên tập trung kèm việc chuyển do các giao thức HTTP thực hiện.

Giống với Bitcoin, IPFS thúc đẩy, kết hợp nhiều ý tưởng hệ thống ngang hàng thành công. Những ý tưởng này là:

- **Hệ thống tệp phân phối toàn cầu.** IPFS là về việc phân phối phi tập trung.
- Xác định dựa trên nội dung có sử dụng hash an toàn về nội dung như công cụ xác định vị trí tệp, và giải quyết các vị trí sử dụng bằng hash được phân phối.
- Trao đổi Khối sử dụng Bittorrent phổ biến dựa trên giao thức phân phối tệp ngang hàng.
- Trao đổi khối được khuyến khích bằng cách dùng giao thức Bitswap
- Merkel DAG, Directed Acyclic Graph, tổ chức các tệp dựa theo phiên bản, giống với hệ thống kiểm soát phiên bản Git.
- Tự chứng nhận các máy chủ node lưu trữ để bảo mật.

5.4.2 Hashgraph

Hashgraph nhằm mục đích giải quyết một trong những các vấn đề gây tranh cãi về blockchain công khai hiện tại mô hình đồng thuận mà đảm bảo niềm tin trong một hệ thống phi tập trung. Nhớ lại rằng trong trường hợp của BitCoin và Môi trường đồng thuận Ethereum đạt được là nhờ bằng chứng công việc, PoW. Chi phí đồng thuận với POW là khá cao. Mất khoảng 78 phút để xác nhận giao dịch tại thời điểm ghi này. Và để thêm một giao dịch vào blockchain Bitcoin tốn khoảng 250 kilowatt năng lượng điện mà có thể một căn nhà điển hình trong chín ngày.

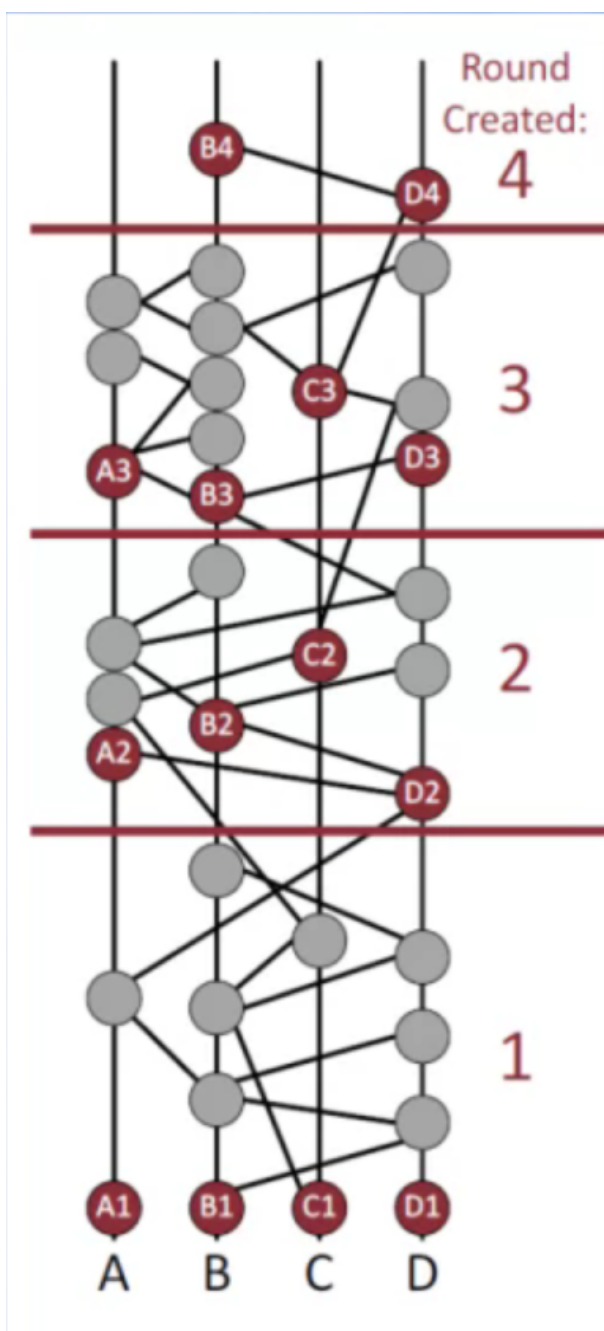
Hashgraph là một mô hình tin cậy cung cấp một lớp đồng thuận giải quyết thời gian chờ giao dịch và năng lượng lãng phí rõ ràng và cũng cung cấp thuật toán tính toán mạnh mẽ cho tính chịu lỗi, và sự nhất quán cuối cùng. Nhớ lại rằng trong một giao thức blockchain, thợ mỏ có thể thu thập bất kỳ giao dịch nào từ member pool theo thứ tự nào họ mong muốn và tạo thành một khối để xác nhận và sau đó thêm khối đó vào blockchain. Điều này dẫn đến cuộc cạnh tranh hoặc cuộc đua được giải quyết bởi bằng chứng công việc, PoW. Điều đó tiêu thụ rất nhiều năng lượng điện để tính toán. Tất cả những công việc này được thực hiện để đạt được sự đồng thuận trên một tập các giao dịch và thứ tự của các giao dịch.

Mục tiêu của hashgraph là sắp xếp giao dịch trong một mạng phi tập trung, giải quyết sự công bằng, an ninh, độ trễ và tập trung năng lượng và đồng thời phần trăm và lỗi, đồng thời cung cấp một thuật toán mạnh cho Byzantine Fault Tolerance và sự nhất quán cuối cùng. Các thành phần của Hashgraph bao gồm:

- Sự kiện - event
- Transactions - giao dịch
- Directed Acyclic Graph: DAG - The hashgraph
- Witness - nhân chứng
- Famous - Witness nhân chứng nổi

- Round - vòng/lượt: Round tạo ra và round nhận được
- Đồng thuận bằng cách bầu ra những nhân chứng ở lượt tiếp theo
- Gossip protocol

Về mặt lý thuyết, một đồ thị được xác định bởi một tập hợp các đỉnh, các nút và các cạnh để nối chúng. Hashgraph có một yếu tố bổ sung của thời gian. Có một dòng thời gian cho mọi người tham gia vào hashgraph. Điều này được chỉ định bởi một đường dọc từ dưới lên trên mọi người tham gia vào hashgraph. **Node** của đồ thị là cấu trúc dữ liệu được gọi là sự kiện và sự truyền của một sự kiện từ người tham gia này đến người khác đại diện cho các **cạnh**. Việc truyền này được gọi là truyền gossip - những gì được biết đến, bởi một nút này đến một nút khác. Gossip / thông tin này được truyền lại dẫn đến khái niệm về "gossip about gossip". Thông tin này được sử dụng cho quan sát các thuộc tính của đồ thị. Điều này hỗ trợ việc biểu quyết ảo cuối cùng và sắp xếp các event và giao dịch trong một tập hợp các sự kiện. Đồ thị cũng được xem là được tạo thành các vòng / round của các sự kiện. Một vòng xác định và phân tách một tập hợp liên kế các sự kiện hashgraph mà có thể được sắp xếp độc lập. Khi ta kết hợp các vòng sự kiện theo thứ tự độc lập, chúng vẫn sẽ giữ lại thứ tự toàn cầu của các giao dịch và sự kiện. Một vòng bao gồm tất cả các sự kiện giữa sự kiện lâu đời nhất và sự kiện mới nhất của vòng.



Hình 25: Một ví dụ về Hashgraph

6 Kết luận - Nhu cầu xã hội

Hiệu ứng cánh bướm - butterfly effect được định nghĩa là một chút hỗn loạn của một trạng thái ban đầu mà ảnh hưởng đáng kể đến kết quả thu được. Bitcoin đã có một hiệu ứng cánh bướm đến công nghệ, theo nghĩa đen. Bitcoin được tổng hợp một cách sáng tạo các kết quả nghiên cứu từ bốn thập kỷ qua để phát hành và làm việc theo mô hình hệ thống thanh toán kỹ thuật số ngang hàng. Ý tưởng kì dị này đã mở ra những nỗ lực và khả năng cho một cuộc cách mạng xã hội. Cuộc cách mạng Blockchain về Bitcoin rút ra từ nghiên cứu về mật mã học, hash, ví dụ như PKI, ECDSA, SHA, và sự tiến bộ to lớn trong công nghệ Internet phương thức chuyển giao ngang hàng P2P. Điều này cho phép phân cấp, phi tập trung hóa, và DLT của blockchain. Máy trạng thái hữu hạn trong ngôn ngữ lập trình hướng đối tượng được sử dụng để phát triển hợp đồng thông minh. Công nghệ web đã được thừa hưởng để phát triển DApp cho blockchains. Từ đó, ta có thể thấy sự biến chuyển Blockchain Bitcoin mang lại.

Blockchain có nhiều ứng dụng đa dạng. Ta có thể tham gia vào phát triển ứng dụng blockchain và giáo dục, thiết kế, phát triển công cụ và framework, tham gia vào nghiên cứu kỹ thuật trong điều tra các ứng dụng của blockchain, về pháp lý, y tế, chính phủ, FinTech, v.v... Ta có thể đề xuất và phát triển các cải tiến giao thức Blockchain, khám phá các giải pháp blockchain khác cho hệ thống phân tán, thực hiện nghiên cứu cơ bản trong thuật toán blockchain. Chúng ta đang chứng kiến sự biến đổi từ ứng dụng tập trung đến ứng dụng phi tập trung, được hỗ trợ bởi công nghệ blockchain và tiền mã hóa. Những công nghệ mới nổi này được kỳ vọng lên đến đỉnh điểm trong các ứng dụng mới hơn, điều đó vượt qua các rào cản về nhân khẩu học và quốc gia. Đó là một nhu cầu xã hội mà ta giáo dục và cho phép tạo ra một xã hội bao gồm mọi nơi đều có được những lợi ích từ ứng dụng blockchain.

Báo cáo đã tập hợp các thông tin cần thiết về blockchain để đóng góp cho một xã hội phi tập trung, toàn diện.

7 Tài liệu tham khảo

- [1] Bitcoin whitepaper
<https://bitcoin.org/bitcoin.pdf>
- [2] Bitcoin's Academic Pedigree
<https://queue.acm.org/detail.cfm?id=3136559>
- [3] What is Blockchain Technology? A Step-by-Step Guide For Beginners
<https://blockgeeks.com/guides/what-is-blockchain-technology/>
- [4] Blockchain: The Invisible Technology That's Changing the World
<https://www.pcmag.com/news/blockchain-the-invisible-technology-thats-changing-the-world>
- [5] Unspent Transaction Output, UTXO
<https://smithandcrown.com/glossary/unspent-transaction-outputs-utxo/>
- [6] How a Bitcoin Transaction Works
<https://www.ccn.com/bitcoin-cash-pools-the-majority-of-bitcoin-sv-blocks-are-mined-by-unknown-yes-really/>
- [7] How does the Blockchain Work? (Part 1)
<https://medium.com/blockchain-review/how-does-the-blockchain-work-for-dummies-explained-simply-9f94d386e093>
- [8] How Does the Blockchain Work?
<https://onezero.medium.com/how-does-the-blockchain-work-98c8cd01d2ae>
- [9] How Do Bitcoin Nodes Verify Transactions?
<https://smartereum.com/8970/how-do-bitcoin-nodes-verify-transactions/>
- [10] A Gentle Introduction to Blockchain Technology
<https://bitsonblocks.net/2015/09/09/gentle-introduction-blockchain-technology/>
- [11] On Public and Private Blockchains
<https://blog.ethereum.org/2015/08/07/on-public-and-private-blockchains/>
- [12] What is Cryptocurrency. Guide for Beginners
<https://cointelegraph.com/bitcoin-for-beginners/what-is-cryptocurrency#accept-as-payment-for-business>
- [13] 2017 Was Bitcoin's Year. 2018 Will Be Ethereum's
<https://www.coindesk.com/2017-bitcoins-year-2018-will-ethereums>
- [14] What is Cryptocurrency: Everything You Need To Know
<https://blockgeeks.com/guides/what-is-cryptocurrency/>
- [15] What is the Difference Between Public and Permissioned Blockchains? - <https://www.coindesk.com/learn/blockchain-10-what-is-blockchain-technology>
- [16] What is Ethereum?
<https://ethdocs.org/en/latest/introduction/what-is-ethereum.html>
- [17] Smart Contracts: The Blockchain Technology That Will Replace Lawyers
<https://blockgeeks.com/guides/smart-contracts/>
- [18] Introduction to Smart Contracts
<https://docs.soliditylang.org/en/develop/introduction-to-smart-contracts.html>
- [19] Ethereum Whitepaper: A Next-Generation Smart Contract and Decentralized Application Platform
<https://ethereum.org/en/whitepaper/>

[20] Account Management
<https://ethdocs.org/en/latest/account-management.html>

[21] Native: Account management
<https://geth.ethereum.org/docs/dapp/native-accounts>

[22] How Ethereum Works
<https://www.coindesk.com/learn/ethereum-101/how-ethereum-works>

[23] What Is Meant By The Term “Gas”?
<https://ethereum.stackexchange.com/questions/3/what-is-meant-by-the-term-gas>

[24] Vitalik Buterin Doubles Down on Ethereum Incentive Strategy
<https://www.coindesk.com/vitalik-buterin-doubles-ethereum-incentive-strategy>

[25] Ether
<https://ethereum.org/en/dapps/>

[26] Proof of Work vs Proof of Stake: Basic Mining Guide
<https://blockgeeks.com/guides/proof-of-work-vs-proof-of-stake/>

[27] What Is Public-Key Cryptography?
<https://www.globalsign.com/en/ssl-information-center/what-is-public-key-cryptography/>

[28] Asymmetric Cryptography (Public-Key Cryptography)
<https://searchsecurity.techtarget.com/definition/asymmetric-cryptography>

[29] Public Key Cryptography - Computerphile
https://www.youtube.com/watch?v=GSIDS_1vRv4

[30] Basic Intro to Elliptic Curve Cryptography
<https://qvault.io/cryptography/elliptic-curve-cryptography/>

[31] What Is Hashing? Under The Hood of Blockchain
<https://blockgeeks.com/guides/what-is-hashing/>

[32] SHA: Secure Hashing Algorithm - Computerphile
<https://www.youtube.com/watch?v=DMtFhACPnTY>

[33] Hash Functions
<https://www.cs.hmc.edu/~geoff/classes/hmc.cs070.200101/homework10/hashfuncs.html>

[34] Hashing demo
<https://andersbrownworth.com/blockchain/hash>

[35] How Safe Are Blockchains? It Depends.
<https://hbr.org/2017/03/how-safe-are-blockchains-it-depends>

[36] Blockchains: Embedding Integrity
<https://infospectives.co.uk/blockchains-embedding-integrity/>

[37] Securing the Chain
<https://assets.kpmg/content/dam/kpmg/xx/pdf/2017/05/securing-the-chain.pdf>

[38] Is It Chain of Headers Rather Than a Chain of Blocks?
<https://bitcoin.stackexchange.com/questions/35448/is-it-chain-of-headers-rather-than-a-chain-of-blocks>

[39] What is a Block Header in Bitcoin?
<https://www.cryptocompare.com/coins/guides/what-is-a-block-header-in-bitcoin/>

[40] Blockchain Based Trust & Authentication For Decentralized Sensor Networks
<https://arxiv.org/pdf/1706.01730.pdf>

[41] How the Blockchain will Radically Transform the Economy
https://www.ted.com/talks/bettina_warburg_how_the_blockchain_will_radically_transform_the_economy?utm_campaign=tedsread--b&utm_medium=referral&utm_source=tedcomshare

[42] A (Short) Guide to Blockchain Consensus Protocols
<https://www.coindesk.com/short-guide-blockchain-consensus-protocols>

[43] Review of blockchain consensus mechanisms
<https://medium.com/wavesprotocol/review-of-blockchain-consensus-mechanisms-f575afae38f2>

[44] Blockchain Expert Explains One Concept in 5 Levels of Difficulty | WIRED
https://www.youtube.com/watch?v=hYip_Vuv8J0

[45] How The Blockchain Is Redefining Trust
<https://www.youtube.com/watch?v=8uF7RVF2osk>

HowTheBlockchainIsRedefiningTrust

[46] Have Blockchain Forks Shown Hayek to be Right or Wrong?
<https://www.trustnodes.com/2017/12/02/blockchain-forks-shown-hayek-right-wrong>

[47] Split on Forks? Blockchain Leaders Learn Tough Lessons from Bitcoin Scaling - <https://www.coindesk.com/split-forks-blockchain-leaders-learn-tough-lessons-bitcoin-scaling>

[48] Bitcoin, Blockchain Forks & Lightning
<https://www.youtube.com/watch?v=8uF7RVF2osk>

[49] Smart Contract: Building blocks for digital markets
https://www.fon.hum.uva.nl/rob/Courses/InformationInSpeech/CDROM/Literature/L0Twinterschool2006/szabo.best.vwh.net/smart_contracts_2.html

[50] How to Learn Solidity: The Ultimate Ethereum Coding Guide
<https://blockgeeks.com/guides/solidity/>

[51] Remix- Solidity IDE
<https://remix.readthedocs.io/en/latest/>

[52] Structure of a Contract
<https://docs.soliditylang.org/en/develop/structure-of-a-contract.html>

[53] Camel Case
https://en.wikipedia.org/wiki/Camel_case

[54] Introduction to Smart Contracts
<https://docs.soliditylang.org/en/develop/introduction-to-smart-contracts.html>

[55] Account Types, Gas, and Transactions
<http://ethdocs.org/en/latest/contracts-and-transactions/account-types-gas-and-transactions.html>

[56] Ethereum, Tokens, and Smart Contracts
<https://k3no.medium.com/ethereum-tokens-smart-contracts-80f639f5c46b>

[57] Decoding the Enigma of Bitcoin Mining
<https://medium.com/all-things-ledger/decoding-the-enigma-of-bitcoin-mining-f8b2697bc4e2>

[58] What is Solidity? Our Guide to the Language of Ethereum Smart Contracts
<https://blockonomi.com/solidity-guide/>

[59] Intro to Solidity 2017 Edition
<https://www.youtube.com/watch?v=KkN108TChbM>

[60] Ethereum Smart Contracts In Solidity 1
 State, Functions, Modifiers and Events - <https://www.youtube.com/watch?v=xWKq86PWG0o>

[61] Solidity Frequently Asked Questions

Types

[62] Solidity Types Documentation
<https://docs.soliditylang.org/en/develop/types.html>

[63] Learning Solidity : Tutorial 6 Data Types (Array, Mapping, Struct)
<https://www.youtube.com/watch?v=8Uh03IKApSg>

[64] Units and Globally Available Variables
<https://docs.soliditylang.org/en/develop/units-and-global-variables.html>

[66] Solidity Tutorials
https://ethereumbuilders.gitbooks.io/guide/content/en/solidity_tutorials.html

[67] Solidity Enums Documentation
<https://docs.soliditylang.org/en/develop/types.html#enums>

[68] Liquid Democracy uses Blockchain to Fix Politics, and Now You Can Vote for It
<https://techcrunch.com/2018/02/24/liquid-democracy-uses-blockchain/>

[69] Contracts
<http://solidity.readthedocs.io/en/develop/contracts.html>

[70] Voting Examole
<https://soliditycookbook.com/voting/>

[71] Time units
<https://docs.soliditylang.org/en/develop/units-and-global-variables.html#time-units>

[72] Solidity Learning: Revert(), Assert(), and Require() in Solidity, and the New REVERT Opcode in the EVM
<https://medium.com/blockchannel/the-use-of-revert-assert-and-require-in-solidity-and-the-new-revert-opcode->

[73] Technical Introduction to Events and Logs in Ethereum
<https://media.consensys.net/technical-introduction-to-events-and-logs-in-ethereum-a074d65dd61e>

[74] Capturing Smart Contract Events in our User Interface (Solidity)
<https://www.youtube.com/watch?v=L5Au5DY8eL4>

[76] Ethereum Smart Contract Security Best Practices
<https://consensys.github.io/smart-contract-best-practices/>

[77] Beyond Smart Contract Best Practices for UX and Interoperability
<https://medium.com/@maurelian/beyond-smart-contract-best-practices-for-ux-and-interoperability-6d94d27c1e0f>

[78] Solidity Smart Contract Security Best Practices
<https://lightrains.com/blogs/smart-contract-best-practices-solidity>

[79] World's First DAPI: Decentralized Application Programming Interface
<https://cointelegraph.com/news/worlds-first-dapi-decentralized-application-programming-interface>

[80] What is the Difference Between a Blockchain and a Database?
<https://www.coindesk.com/learn/blockchain-101/what-is-blockchain-technology>

[81] DEVCON1: Understanding the Ethereum Blockchain Protocol
 Vitalik Buterin
<https://www.youtube.com/watch?v=gjwr-7PgpN8&t=10s>

[82] Decentralizing Everything with Ethereum's Vitalik Buterin | Disrupt SF 2017
<https://www.youtube.com/watch?v=WSN5BaCzsbo&t=5s>

[83] What is and how to use the Ethereum Name Service (ENS)
<https://www.cryptocompare.com/coins/guides/what-is-and-how-to-use-the-ens/>

[84] Ethereum-ens
<https://www.npmjs.com/package/ethereum-ens>

[85] Decentralized Applications-dApps
<https://blockchainhub.net/decentralized-applications-dapps/>

[86] The Future will be Decentralized
<https://www.youtube.com/watch?v=97ufCT6lQcY>

[87] Web3 JavaScript app API for 0.2x.x
<https://github.com/ethereum/wiki/wiki/JavaScript-API>

[88] Ethereum CLI Tools
https://gavofyork.gitbooks.io/turboethereum/content/cli_tools.html

[89] TRUFFLE TUTORIAL INDEX
<https://www.trufflesuite.com/tutorial>

[90] Linux command sheet
https://sites.tufts.edu/cbi/files/2013/01/linux_cheat_sheet.pdf

[91] Testing of Smart Contracts in the Blockchain world
<https://www.capgemini.com/2017/01/testing-of-smart-contracts-in-the-blockchain-world/>

[92] Smart Contracts and Truffle 101. Part 4 - Testing Functions and Errors
<https://www.youtube.com/watch?v=LvNOeDGG0y8>

[93] Smart Contracts and Truffle 101. Part 3 - Simple functions and tests
<https://www.youtube.com/watch?v=YhO-Uzp7Apw>

[94] Solidity Events Tutorial - Using Web3.js to Listen for Smart Contract Events
<https://coursetro.com/posts/code/100/Solidity-Events-Tutorial---Using-Web3.js-to-Listen-for-Smart-Contract-Events>

[95] Calls, transactions, events, filters and topics
<https://netherium.readthedocs.io/en/latest/contracts/calling-transactions-events/>

[96] Technical Introduction to Events and Logs in Ethereum
<https://media.consensys.net/technical-introduction-to-events-and-logs-in-ethereum-a074d65dd61e>

[97] Getting data from the internet with Oraclize
<https://ethereumdev.io/getting-data-internet-oraclize/>

[98] Oraclize Documentation
<https://docs.provable.xyz>

[99] What is An Initial Coin Offering? Raising Millions In Seconds
<https://blockgeeks.com/guides/initial-coin-offering/>

[100] The Ether Thief
<https://www.bloomberg.com/features/2017-the-ether-thief/>

[101] The difference between App Coins and Protocol Tokens
<https://blog.0xproject.com/the-difference-between-app-coins-and-protocol-tokens-7281a428348c>

[102] If you don't understand blockchain, maybe these cats can explain it to you - <https://www.cnbc.com/2017/12/17/cryptokitties-makes-it-easy-to-understand-blockchain-and-genetics.html>

[103] ERC 20 Token Standard
<https://github.com/ethereum/EIPs/blob/master/EIPS/eip-20.md>

[104] ERC 721 Token Standard
<https://github.com/ethereum/EIPs/blob/master/EIPS/eip-721.md>

[105] EIP Ethereum Improvement Proposal
<https://github.com/ethereum/EIPs/blob/master/EIPS/eip-1.md>

[106] About Hyperledger
<https://www.hyperledger.org/about>

[107] Hyperledger tutorial
<https://hyperledger.github.io/composer/latest/tutorials/tutorials.html>

[108] What is Hyperledger? Brian Behlendorf Explains the Linux Foundation's Blockchain Initiative
<https://www.youtube.com/watch?v=6rUsARjV4To>

[109] Hyperledger Fabric
<https://hyperledger-fabric.readthedocs.io/en/release-1.1/>

[110] An introduction to Hyperledger Fabric from The Linux Foundation
<https://www.youtube.com/watch?v=irCIDxA5asc>

[111] Microsoft And The Blockchain: MSFT's Big Projects
<https://www.nasdaq.com/articles/microsoft-and-blockchain-msfts-big-projects-2018-01-17>

[112] Main three components of Hyperledger Fabric
<https://www.youtube.com/watch?v=CsauV-9zHAK>

[113] Peer channel-based event services
https://hyperledger-fabric.readthedocs.io/en/release-1.1/peer_event_services.html

[114] Architecture Explained
<https://hyperledger-fabric.readthedocs.io/en/release-1.1/arch-deep-dive.html>

[115] APIs - CLI, REST, and Node.js
<https://openblockchain.readthedocs.io/en/latest/API/CoreAPI/>

[116] Blockchain Development on Hyperledger Fabric using Composer : What is Hyperledger?
<https://www.youtube.com/watch?v=oGbcdToJa7w>

[106] Hyperledger Fabric Model
https://hyperledger-fabric.readthedocs.io/en/release-1.1/fabric_model.html

[107] Hyperledger Fabricdocs Documentation
<buildmedia.readthedocs.org/media/pdf/hyperledger-fabric/latest/hyperledger-fabric.pdf>

[108] Hyperledger Architecture, Volume 1
www.hyperledger.org/wp-content/uploads/2017/08/Hyperledger_Arch_WG_Paper_1_Consensus.pdf

[109] Hyperledger Composer
<https://www.hyperledger.org/use/composer>

[110] What is Hyperledger Composer from The Linux Foundation with Dan Selman - <https://www.youtube.com/watch?v=PvrLJTGFje0>

[111] Hyperledger Composer Modeling Language
https://hyperledger.github.io/composer/latest/reference/cto_language

[112] Model and test your blockchain network
[- developer.ibm.com/tutorials/ibm-blockchain-platform-vscode-smart-contract/](https://developer.ibm.com/tutorials/ibm-blockchain-platform-vscode-smart-contract/)

[113] What is Microsoft Azure, Anyway?
<https://www.howtogeek.com/337961/what-is-microsoft-azure/>

[114] Tour of Microsoft Azure
azure.microsoft.com/en-us/resources/videos/tour-of-microsoft-azure/

[115] Simplifying blockchain app development with Azure Blockchain Workbench - <https://azure.microsoft.com/en-us/blog/simplifying-blockchain-app-development-with-azure-blockchain-workbench-2/>

[116] What Is Augur?
<https://www.weusecoins.com/what-is-augur/>

[106] Augur white paper
medium.com/@AugurProject/the-augur-white-paper-a-decentralized-oracle-and-prediction-market-platform-ed89074

[107] Augur: The World's Most Undervalued Crypto Project
[/medium.com/@Cryptokeeper/augur-the-worlds-most-undervalued-crypto-project-62934686a016](https://medium.com/@Cryptokeeper/augur-the-worlds-most-undervalued-crypto-project-62934686a016)

[108] Augur in 2 Minutes
https://www.youtube.com/watch?v=579SRoK_kdQ

[109] GridPlus - What is Grid+?
https://www.youtube.com/watch?v=rwkjAXom_6U

[110] Basic Primer: Blockchain Consensus Protocol
<https://blockgeeks.com/guides/blockchain-consensus/>

[111] Consensus Achieved Using Proof-of-Work
<https://mastanbtc.github.io/blockchainnotes/consensustypes/>

[112] Consensus and Mining on the Blockchain
<https://www.youtube.com/watch?v=2HcmwFVzPEU>

[113] How nodes reach a consensus on a blockchain
<https://www.youtube.com/watch?v=DqtzxJP6Y9k>

[114] Blockchain Scalability: When, Where, How?
<https://blockgeeks.com/guides/blockchain-scalability/>

[115] Scalability Tradeoffs: Why "The Ethereum Killer" Hasn't Arrived Yet - <https://medium.com/loom-network/scalability-tradeoffs-why-the-ethereum-killer-hasnt-arrived-yet-8f60a88e46c0>

[116] Can Blockchain Solve Its Scalability Problem?
<https://www.youtube.com/watch?v=LxuLhPtIiHA>

[117] Brock Pierce from Blockchain Capital explains the scalability problems for Bitcoin
<https://www.youtube.com/watch?v=gMFp66FRjZo>

[118] SOLVING BLOCKCHAIN'S PRIVACY PROBLEM
www.newsweek.com/solving-blockchain-privacy-problem-643368

[119] Two party contracts - <https://dappsforbeginners.wordpress.com/tutorials/two-party-contracts/>

[120] How A Smart Contract replaced An Escrow Company in a \$60k deal
<https://hackernoon.com/how-a-smart-contract-replaced-an-escrow-company-in-a-60k-deal-551ff7839044>

[121] Ultimate Intro to Ethereum Dapp Development P8 - Smart Contracts - Escrow
<https://www.youtube.com/watch?v=EbWktDPFPz8>

[122] BURST Decentralize Escrow Service Smart Contracts
<https://www.youtube.com/watch?v=nxQSeXk0Ezw>

[123] Toward an Ethereum Multisig Standard
medium.com/@asmiller1989/toward-an-ethereum-multisig-standard-c566c7b7a3f6

[124] What is the InterPlanetary File System?
themerikle.com/what-is-the-interplanetary-file-system/

[125] IPFS Alpha Demo
<https://www.youtube.com/watch?v=8CMxDNuuAiQ>

[126] IPFS white paper
ipfs.io/ipfs/QmR7GSQM93Cx5eAg6a6yRzNde1FQv7uL6X1o4k7zrJa3LX/ipfs.draft3.pdf

[127] An Introduction to IPFS
<https://medium.com/@ConsenSys/an-introduction-to-ipfs-9bba4860abd0>

[128] An Introduction to The Interplanetary File System - <https://www.youtube.com/watch?v=BA2rHlbB5i0>

[129] The next Internet Revolution | Juan Benet | TEDxSanFrancisco
www.youtube.com/watch?v=2RCwZDRwk48

[130] What's Yeoman?
<https://yeoman.io>

[131] Hashgraph Consensus: Detailed Examples
<https://www.swirlds.com/downloads/SWIRLDS-TR-2016-02.pdf>

[132] What Is Hedera Hashgraph?
<https://www.youtube.com/watch?v=MzWiiOLv96I>

[133] Hashgraph wants to give you the benefits of blockchain without the limitations
<https://techcrunch.com/2018/03/13/hashgraph-wants-to-give-you-the-benefits-of-blockchain-without-the-limitations/>

[134] Demystifying Hashgraph: Benefits and Challenges
<https://hackernoon.com/demystifying-hashgraph-benefits-and-challenges-d605e5c0cee5>

[135] Hedera Hashgraph Thinks It Can One-Up Bitcoin And Ethereum With Faster Transactions
<https://www.forbes.com/sites/jeffkauflin/2018/03/13/hedera-hashgraph-thinks-it-can-one-up-bitcoin-and-ethereum/#5feba94aabcbb>

[136] What is Hashgraph and how will it replace the Blockchain?!
<https://www.youtube.com/watch?v=SPdUAW-Fpco>

[137] Commentary: How Blockchain Could Replace Social Security Numbers
<https://fortune.com/2018/01/11/blockchain-technology-social-security-number-cybersecurity-identity-theft/>

[138] Blockchain and U.S. state governments: An initial assessment
<https://www.brookings.edu/blog/techtank/2018/04/17/blockchain-and-u-s-state-governments-an-initial-assessment/>